



# last time

pipeline idea

instruction A in stage 3, while instruction B in stage 2, while instruction C in stage 3

register delays, uneven split limiting pipeline benefits

control and data hazards

stalling — hardware inserted no-ops

forwarding — shortcut for value that hasn't made it to register file

## some quiz notes (1)

some question writing issues on cryptography questions

Q1B: As written: send duplicate message to forum A — no problem, since no counters or similar. Meant to write “same text they intended to go to forum B” — would be prevented because it would require forging  $\text{MAC}(k, \text{forum name} + \text{msg})$ .

Q3A: Depends whether C can act as machine-in-the-middle (not specified in question)

Q3E: Probably should have written ‘without B being able to detect it’, but does not change answer

## some quiz notes (2)

on Q6E:

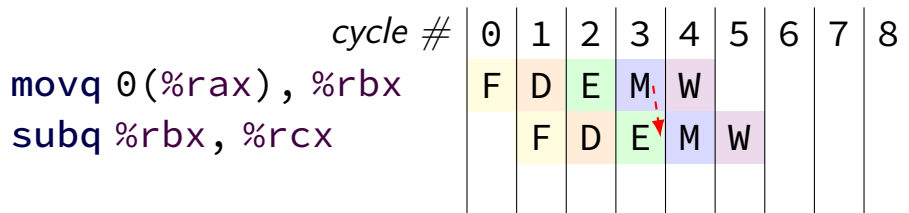
```
movq %r9, (%r10) // memory[r10] <- r9
```

```
movq %r10, %r11 // r11 <- r10
```

first movq does not modify register %r10 (or any registers)

...so no way for second movq to read 'wrong version' of '%r10'

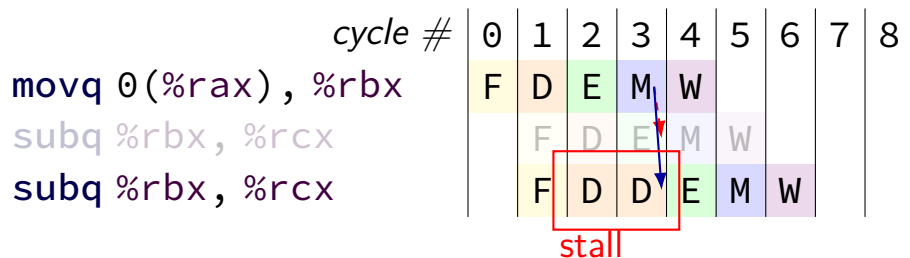
# unsolved problem



**combine** stalling and forwarding to resolve hazard

assumption in diagram: hazard detected in subq's decode stage  
(since easier than detecting it in fetch stage)

# unsolved problem



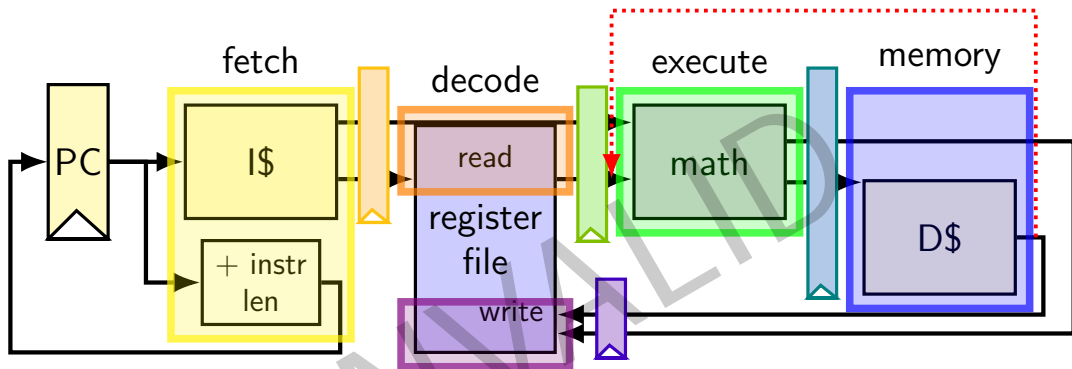
**combine** stalling and forwarding to resolve hazard

assumption in diagram: hazard detected in subq's decode stage  
(since easier than detecting it in fetch stage)

# solveable problem

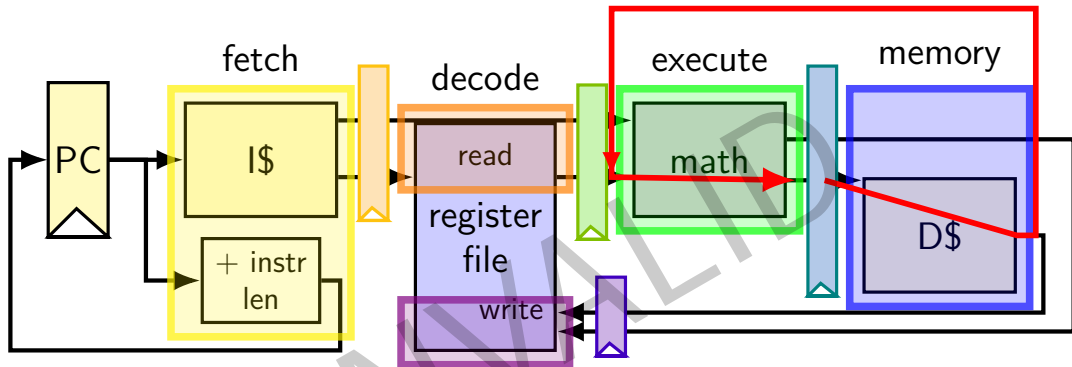
|                    | cycle # | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|--------------------|---------|---|---|---|---|---|---|---|---|---|
| movq 0(%rax), %rbx |         | F | D | E | M | W |   |   |   |   |
| movq %rbx, 0(%rcx) |         |   | F | D | E | M | W |   |   |   |

# why can't we...



clock cycle needs to be long enough  
to go through data cache AND  
to go through math circuits!  
(which we were trying to avoid by putting them in separate stages)

## why can't we...



clock cycle needs to be long enough  
to go through data cache AND  
to go through math circuits!  
(which we were trying to avoid by putting them in separate stages)

# hazards versus dependencies

dependency — X needs result of instruction Y?

has potential for being messed up by pipeline  
(since part of X may run before Y finishes)

hazard — will it not work in some pipeline?

**before** extra work is done to “resolve” hazards

multiple kinds: so far, *data hazards*, *control hazards*

## ex.: dependencies and hazards (1)

**addq**        %rax,     %rbx

**subq**        %rax,     %rcx

**movq**        \$100,     %rcx

**addq**        %rcx,     %r10

**addq**        %rbx,     %r10

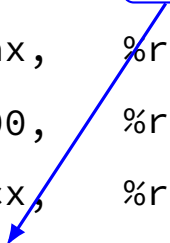
where are dependencies?

which are hazards in our pipeline?

which are resolved with forwarding?

## ex.: dependencies and hazards (1)

|      |        |      |
|------|--------|------|
| addq | %rax,  | %rbx |
| subq | %rax,  | %rcx |
| movq | \$100, | %rcx |
| addq | %rcx,  | %r10 |
| addq | %rbx,  | %r10 |



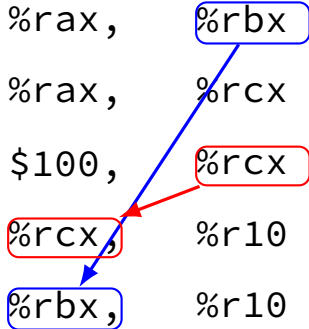
where are dependencies?

which are hazards in our pipeline?

which are resolved with forwarding?

## ex.: dependencies and hazards (1)

|      |        |      |
|------|--------|------|
| addq | %rax,  | %rbx |
| subq | %rax,  | %rcx |
| movq | \$100, | %rcx |
| addq | %rcx,  | %r10 |
| addq | %rbx,  | %r10 |



where are dependencies?

which are hazards in our pipeline?

which are resolved with forwarding?

## ex.: dependencies and hazards (1)

|      |        |      |
|------|--------|------|
| addq | %rax,  | %rbx |
| subq | %rax,  | %rcx |
| movq | \$100, | %rcx |
| addq | %rcx,  | %r10 |
| addq | %rbx,  | %r10 |

```
graph TD; I1[addq %rax, %rbx] -- blue --> I5[addq %rbx, %r10]; I3[movq $100, %rcx] -- red --> I4[addq %rcx, %r10]; I4 -- red --> I5;
```

where are dependencies?

which are hazards in our pipeline?

which are resolved with forwarding?

# pipeline with different hazards

example: 4-stage pipeline:

fetch/decode/execute+memory/writeback

|                 | <i>// 4 stage</i> | <i>// 5 stage</i> |
|-----------------|-------------------|-------------------|
| addq %rax, %r8  | <i>//</i>         | <i>// W</i>       |
| subq %rax, %r9  | <i>// W</i>       | <i>// M</i>       |
| xorq %rax, %r10 | <i>// EM</i>      | <i>// E</i>       |
| andq %r8, %r11  | <i>// D</i>       | <i>// D</i>       |

# pipeline with different hazards

example: 4-stage pipeline:

fetch/decode/execute+memory/writeback

|                              | <i>// 4 stage</i> | <i>// 5 stage</i> |
|------------------------------|-------------------|-------------------|
| <code>addq %rax, %r8</code>  | <i>//</i>         | <i>// W</i>       |
| <code>subq %rax, %r9</code>  | <i>// W</i>       | <i>// M</i>       |
| <code>xorq %rax, %r10</code> | <i>// EM</i>      | <i>// E</i>       |
| <code>andq %r8, %r11</code>  | <i>// D</i>       | <i>// D</i>       |

(assuming register file does not read while writing)

`addq/andq` is hazard with 5-stage pipeline

`addq/andq` is **not** a hazard with 4-stage pipeline

# pipeline with different hazards

example: 4-stage pipeline:

fetch/decode/execute+memory/writeback

|                              | <i>// 4 stage</i> | <i>// 5 stage</i> |
|------------------------------|-------------------|-------------------|
| <code>addq %rax, %r8</code>  | <i>//</i>         | <i>// W</i>       |
| <code>subq %rax, %r9</code>  | <i>// W</i>       | <i>// M</i>       |
| <code>xorq %rax, %r10</code> | <i>// EM</i>      | <i>// E</i>       |
| <code>andq %r8, %r11</code>  | <i>// D</i>       | <i>// D</i>       |

more hazards with more pipeline stages

## exercise: different pipeline

split execute into two stages: F/D/E1/E2/M/W

result only available near end of second execute stage

where does forwarding, stalls occur?

|                                    | <i>cycle #</i> | 0 | 1 | 2  | 3  | 4 | 5 | 6 | 7 | 8 |
|------------------------------------|----------------|---|---|----|----|---|---|---|---|---|
| (1) <code>addq %rcx, %r9</code>    |                | F | D | E1 | E2 | M | W |   |   |   |
| (2) <code>addq %r9, %rbx</code>    |                |   |   |    |    |   |   |   |   |   |
| (3) <code>addq %rax, %r9</code>    |                |   |   |    |    |   |   |   |   |   |
| (4) <code>movq %r9, 8(%rbx)</code> |                |   |   |    |    |   |   |   |   |   |
| (5) <code>movq %rcx, %r9</code>    |                |   |   |    |    |   |   |   |   |   |

## exercise: different pipeline

split execute into two stages: F/D/E1/E2/M/W

|                                | <i>cycle #</i> | 0 | 1 | 2  | 3  | 4 | 5 | 6 | 7 | 8 |
|--------------------------------|----------------|---|---|----|----|---|---|---|---|---|
| <code>addq %rcx, %r9</code>    |                | F | D | E1 | E2 | M | W |   |   |   |
| <code>addq %r9, %rbx</code>    |                |   |   |    |    |   |   |   |   |   |
| <code>addq %rax, %r9</code>    |                |   |   |    |    |   |   |   |   |   |
| <code>movq %r9, 8(%rbx)</code> |                |   |   |    |    |   |   |   |   |   |

## exercise: different pipeline

split execute into two stages: F/D/E1/E2/M/W

|                                | <i>cycle #</i> | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7 | 8 |
|--------------------------------|----------------|---|---|----|----|----|----|----|---|---|
| <code>addq %rcx, %r9</code>    |                | F | D | E1 | E2 | M  | W  |    |   |   |
| <code>addq %r9, %rbx</code>    |                |   | F | D  | E1 | E2 | M  | W  |   |   |
| <code>addq %rax, %r9</code>    |                |   |   | F  | D  | E1 | E2 | M  | W |   |
| <code>movq %r9, 8(%rbx)</code> |                |   |   |    | F  | D  | E1 | E2 | M | W |

## exercise: different pipeline

split execute into two stages: F/D/E1/E2/M/W

|                          | <i>cycle #</i> | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8 |   |
|--------------------------|----------------|---|---|----|----|----|----|----|----|---|---|
| <b>addq %rcx, %r9</b>    |                | F | D | E1 | E2 | M  | W  |    |    |   |   |
| <b>addq %r9, %rbx</b>    |                |   | F | D  | E1 | E2 | M  | W  |    |   |   |
| <b>addq %r9, %rbx</b>    |                |   | F | D  | D  | E1 | E2 | M  | W  |   |   |
| <b>addq %rax, %r9</b>    |                |   |   | F  | D  | E1 | E2 | M  | W  |   |   |
| <b>addq %rax, %r9</b>    |                |   |   | F  | F  | D  | E1 | E2 | M  | W |   |
| <b>movq %r9, 8(%rbx)</b> |                |   |   |    | F  | D  | E1 | E2 | M  | W |   |
| <b>movq %r9, 8(%rbx)</b> |                |   |   |    |    | F  | D  | E1 | E2 | M | W |

## exercise: different pipeline

split execute into two stages: F/D/E1/E2/M/W

|                          | <i>cycle #</i> | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8 |   |
|--------------------------|----------------|---|---|----|----|----|----|----|----|---|---|
| <b>addq %rcx, %r9</b>    |                | F | D | E1 | E2 | M  | W  |    |    |   |   |
| <b>addq %r9, %rbx</b>    |                |   | F | D  | E1 | E2 | M  | W  |    |   |   |
| <b>addq %r9, %rbx</b>    |                |   | F | D  | D  | E1 | E2 | M  | W  |   |   |
| <b>addq %rax, %r9</b>    |                |   |   | F  | D  | E1 | E2 | M  | W  |   |   |
| <b>addq %rax, %r9</b>    |                |   |   | F  | F  | D  | E1 | E2 | M  | W |   |
| <b>movq %r9, 8(%rbx)</b> |                |   |   |    | F  | D  | E1 | E2 | M  | W |   |
| <b>movq %r9, 8(%rbx)</b> |                |   |   |    |    | F  | D  | E1 | E2 | M | W |

## exercise: different pipeline

split execute into two stages: F/D/E1/E2/M/W

|                   | cycle # | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  |   |   |
|-------------------|---------|---|---|----|----|----|----|----|----|----|---|---|
| addq %rcx, %r9    |         | F | D | E1 | E2 | M  | W  |    |    |    |   |   |
| addq %r9, %rbx    |         |   | F | D  | E1 | E2 | M  | W  |    |    |   |   |
| addq %r9, %rbx    |         |   | F | D  | D  | E1 | E2 | M  | W  |    |   |   |
| addq %rax, %r9    |         |   |   | F  | D  | E1 | E2 | M  | W  |    |   |   |
| addq %rax, %r9    |         |   |   | F  | F  | D  | E1 | E2 | M  | W  |   |   |
| movq %r9, 8(%rbx) |         |   |   |    | F  | D  | E1 | E2 | M  | W  |   |   |
| movq %r9, 8(%rbx) |         |   |   |    |    | F  | D  | E1 | E2 | M  | W |   |
| movq %rcx, %r9    |         |   |   |    |    |    | F  | D  | E1 | E2 | M | W |

# beyond pipelining: multiple issue

start **more than one instruction/cycle**

multiple parallel pipelines; many-input/output register file

**hazard handling much more complex**

|                 | <i>cycle #</i> | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|-----------------|----------------|---|---|---|---|---|---|---|---|---|
| addq %r8, %r9   |                | F | D | E | M | W |   |   |   |   |
| subq %r10, %r11 |                | F | D | E | M | W |   |   |   |   |
| xorq %r9, %r11  |                |   | F | D | E | M | W |   |   |   |
| subq %r10, %rbx |                |   | F | D | E | M | W |   |   |   |
| ...             |                |   |   |   |   |   |   |   |   |   |

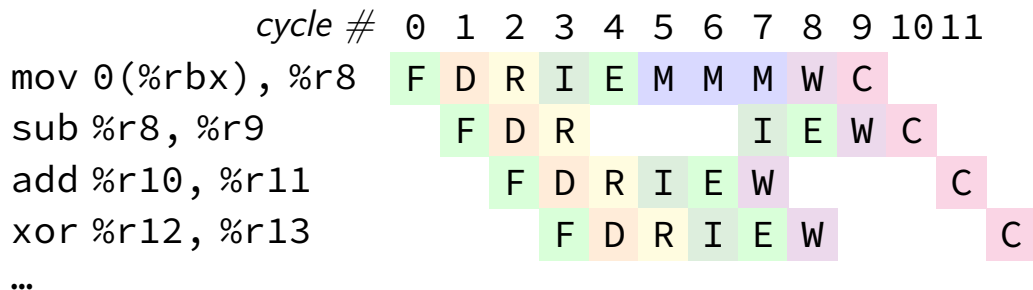


# beyond pipelining: out-of-order

find **later instructions to do** instead of stalling

lists of available instructions in pipeline registers  
take any instruction with available values

provide **illusion that work is still done in order**  
much more complicated hazard handling logic



# interlude: real CPUs

modern CPUs:

execute multiple instructions at once

execute instructions out of order — whenever values available

# out-of-order and hazards

out-of-order execution makes hazards harder to handle

problems for forwarding:

- value in last stage may not be most up-to-date

- older value may be written back before newer value?

problems for branch prediction:

- mispredicted instructions may complete execution before squashing

which instructions to dispatch?

- how to quickly find instructions that are ready?

# out-of-order and hazards

out-of-order execution makes hazards harder to handle

problems for forwarding:

- value in last stage may not be most up-to-date

- older value may be written back before newer value?

problems for branch prediction:

- mispredicted instructions may complete execution before squashing

which instructions to dispatch?

- how to quickly find instructions that are ready?

# read-after-write examples (1)

| cycle #          | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|------------------|---|---|---|---|---|---|---|---|---|
| addq %r10, %r8   | F | D | E | M | W |   |   |   |   |
| movq %r8, (%rax) |   | F | D | E | M | W |   |   |   |
| movq \$100, %r8  |   |   | F | D | E | M | W |   |   |
| addq %r12, %r8   |   |   |   | F | D | E | M | W |   |

normal pipeline: two options for %r8?

choose the one from *earliest stage*

because it's from the most recent instruction

# read-after-write examples (1)

out-of-order execution:

%r8 from earliest stage might be from *delayed instruction*  
can't use same forwarding logic

addq %r8, (%rax)

movq \$100, %r8

addq %r12, %r8

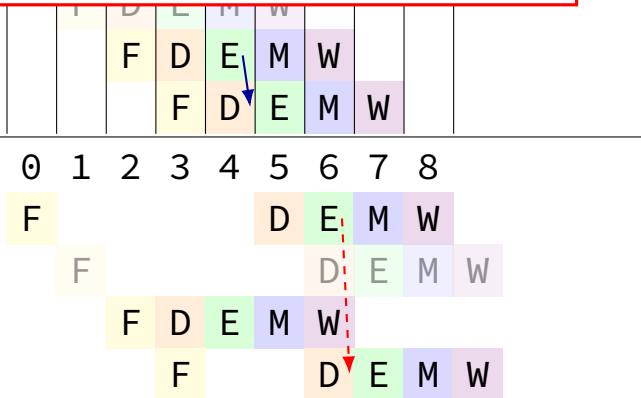
addq %r10, %r8

movq %r8, (%rax)

movq \$100, %r8

addq %r13, %r8

cycle # 0 1 2 3 4 5 6 7 8



# register version tracking

goal: track **different versions of registers**

out-of-order execution: may compute versions at different times

only forward the **correct version**

strategy for doing this: preprocess instructions represent version info

makes forwarding, etc. lookup easier

## rewriting hazard examples (1)

|                |  |                                                  |
|----------------|--|--------------------------------------------------|
| addq %r10, %r8 |  | addq %r10, %r8 <sub>v1</sub> → %r8 <sub>v2</sub> |
| addq %r11, %r8 |  | addq %r11, %r8 <sub>v2</sub> → %r8 <sub>v3</sub> |
| addq %r12, %r8 |  | addq %r12, %r8 <sub>v3</sub> → %r8 <sub>v4</sub> |

---

read different version than the one written

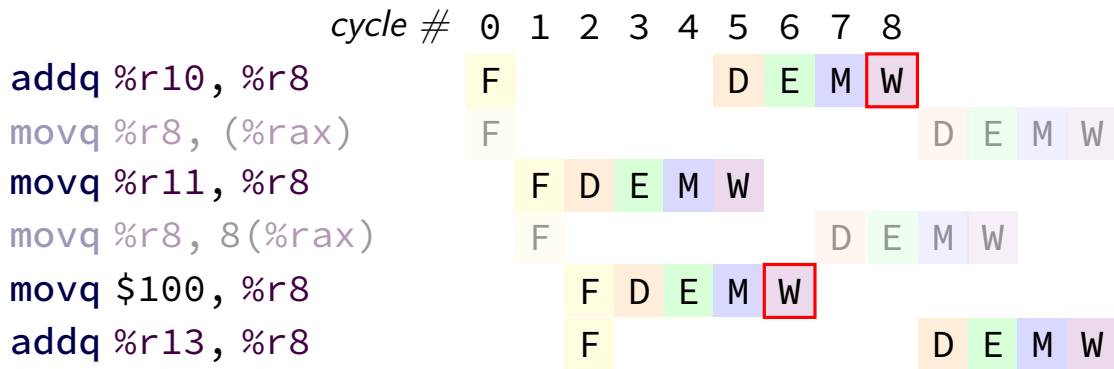
represent with three argument psuedo-instructions

forwarding a value? must match version *exactly*

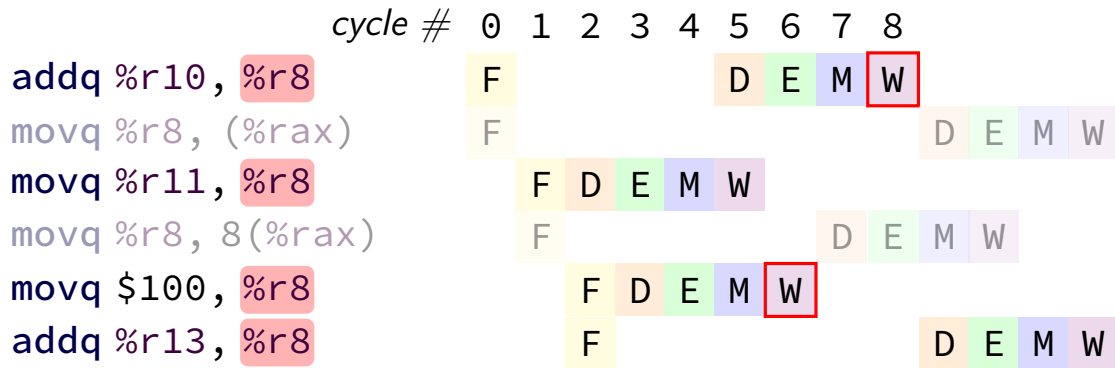
for now: version numbers

later: something simpler to implement

# write-after-write example



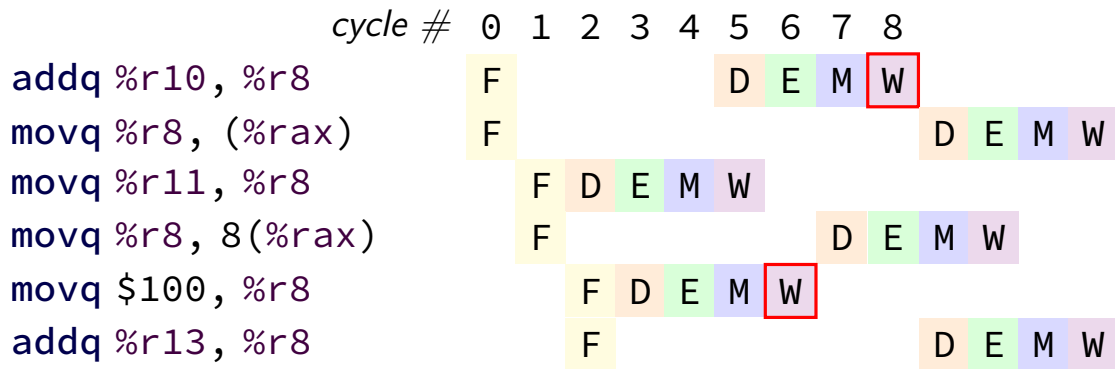
# write-after-write example



out-of-order execution:

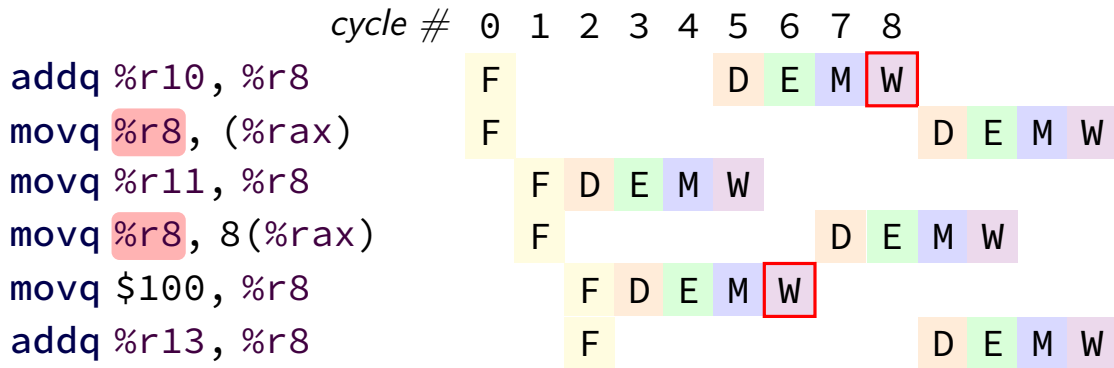
if we don't do something, newest value could be overwritten!

# write-after-write example



multiple instructions that haven't been started  
could need *different versions* of %r8!

# write-after-write example



# keeping multiple versions

for write-after-write problem: need to keep copies of multiple versions

both the new version and the old version needed by delayed instructions

for read-after-write problem: need to distinguish different versions

solution: have lots of extra registers

...and assign each version a new 'real' register

called register renaming

# register renaming

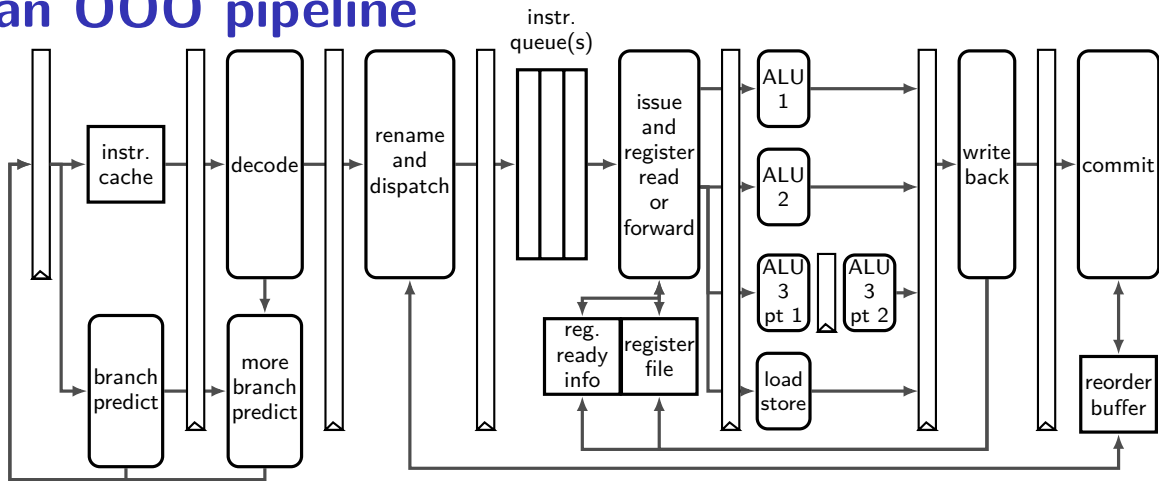
rename *architectural registers* to *physical registers*

different physical register for each version of architectural

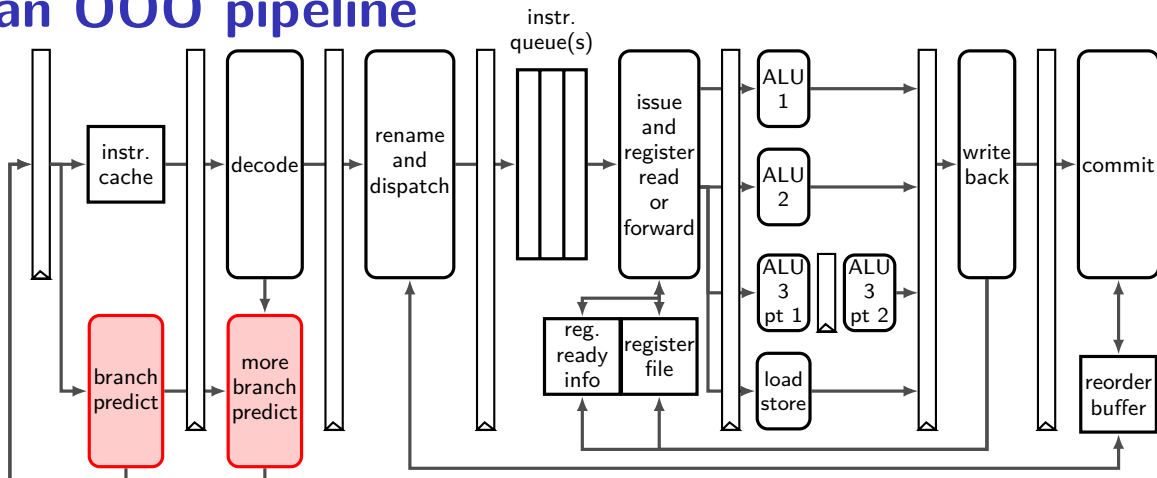
track which physical registers are ready

compare physical register numbers to do forwarding

# an OOO pipeline

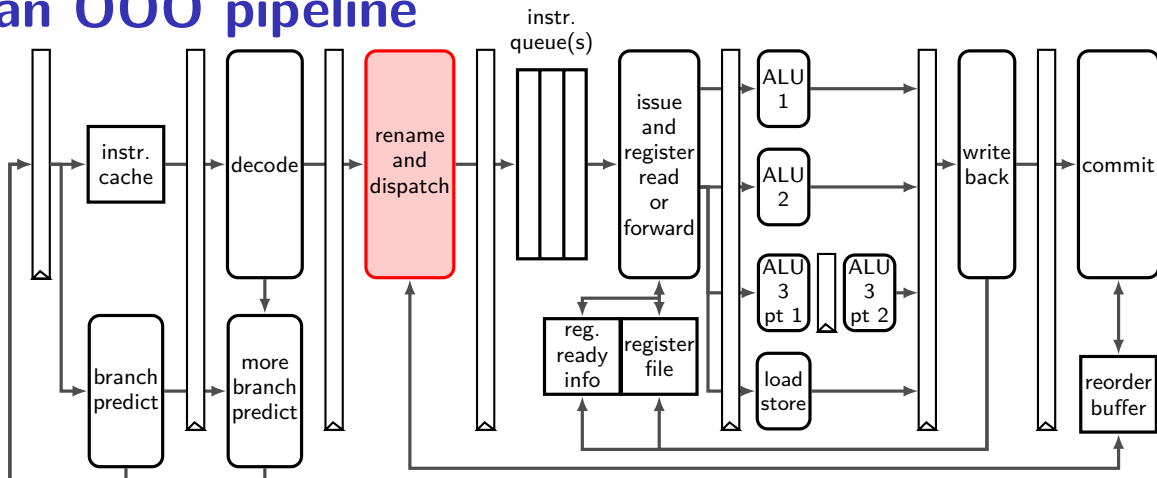


# an OOO pipeline



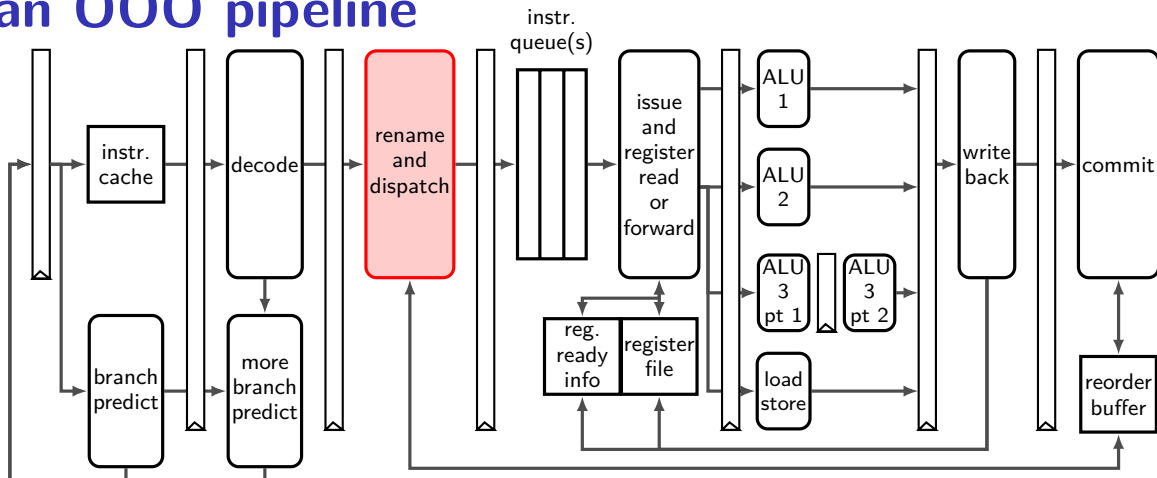
branch prediction needs to happen before instructions decoded  
done with cache-like tables of information about recent branches

# an OOO pipeline



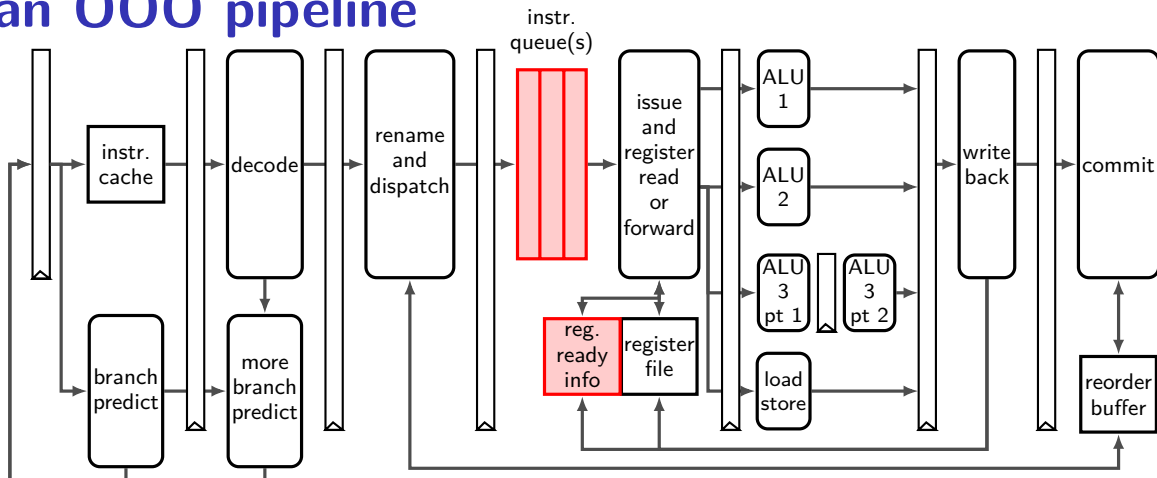
register renaming done here  
stage needs to keep mapping from architectural to physical names

# an OOO pipeline



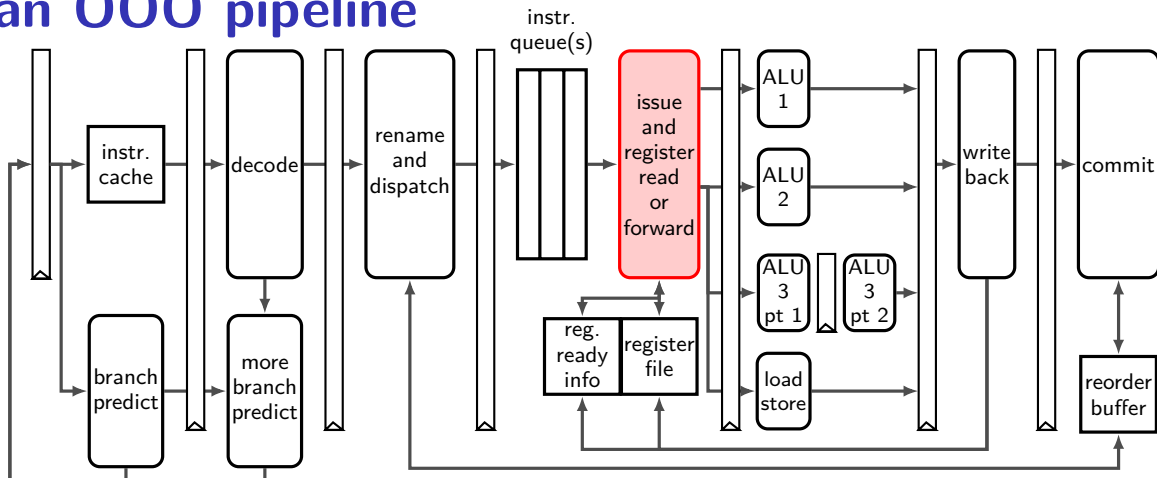
'dispatch' instructions: add to instruction queue  
prepare reorder buffer entries for any squashing

# an OOO pipeline



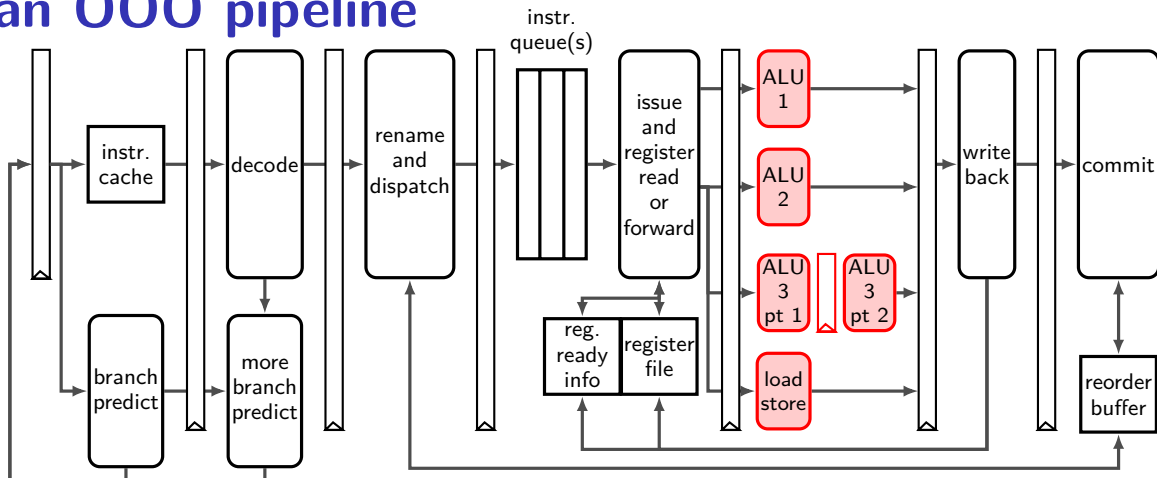
instruction queue holds pending renamed instructions combined with register-ready info to *issue* instructions (issue = start executing)

# an OOO pipeline



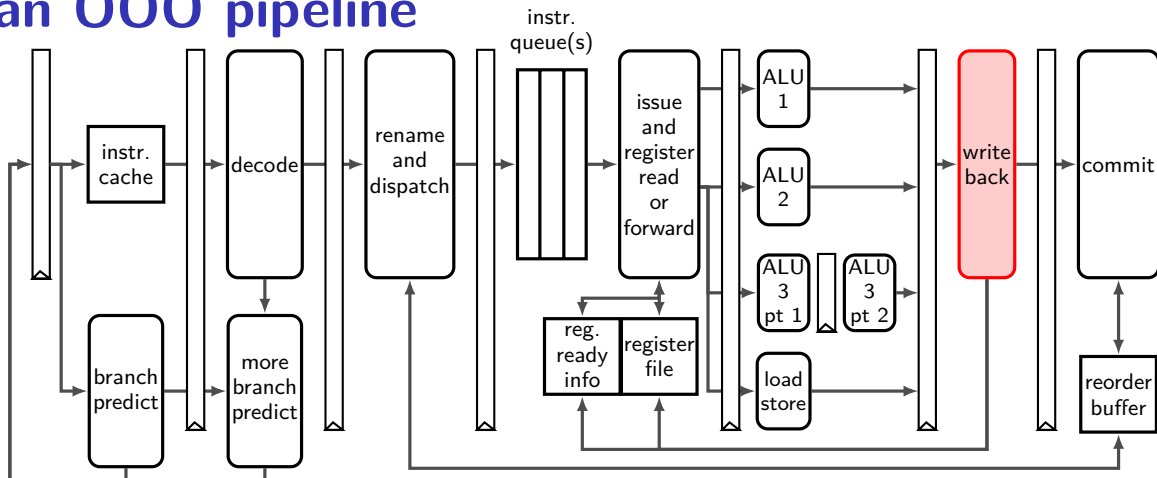
read from much larger register file and handle forwarding  
register file: typically read 6+ registers at a time  
(extra wires for forwarding not shown)

# an OOO pipeline



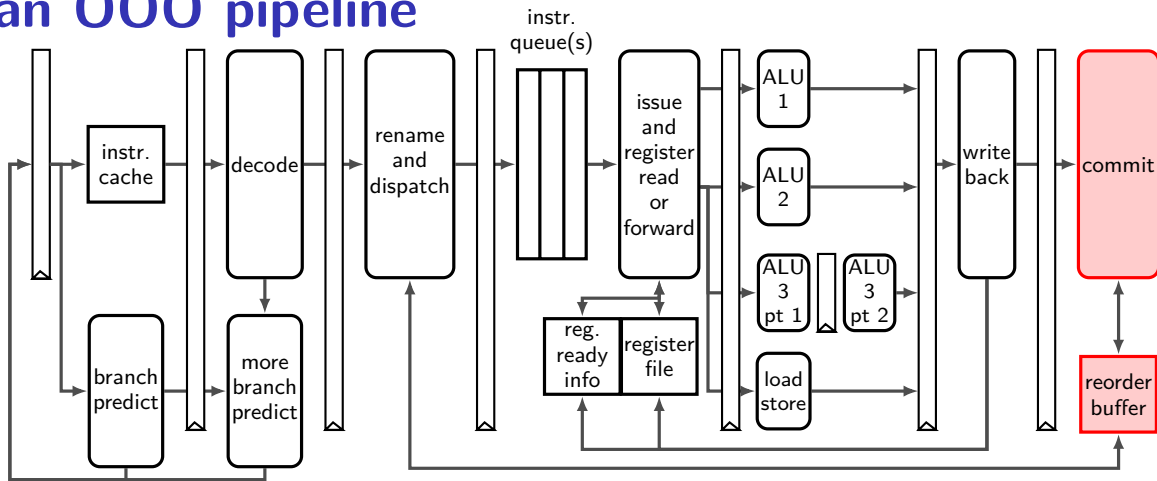
many *execution units* actually do math or memory load/store  
some may have multiple pipeline stages  
some may take variable time (data cache, integer divide, ...)

# an OOO pipeline



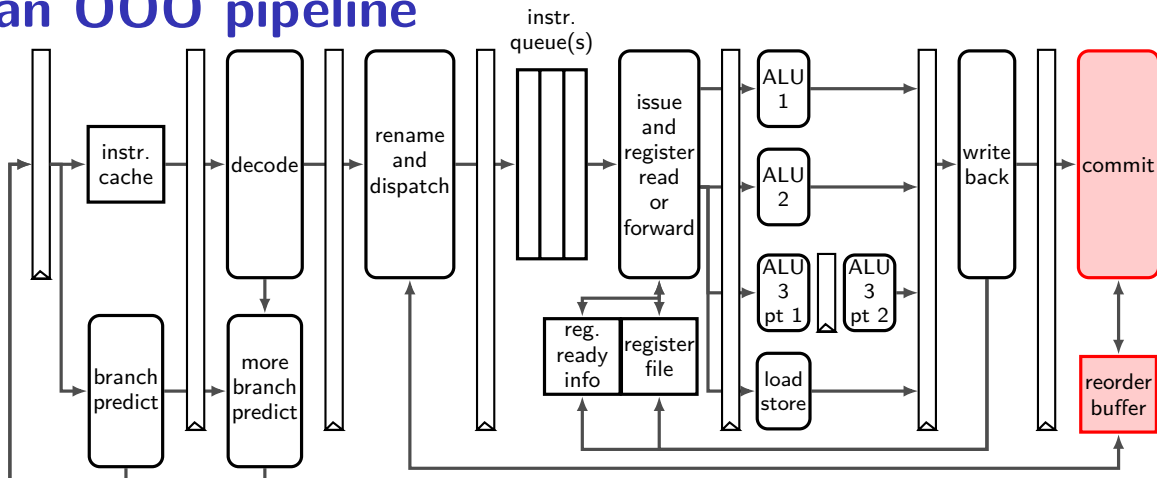
writeback results to physical registers  
register file: typically support writing 3+ registers at a time

# an OOO pipeline



new commit (sometimes *retire*) stage finalizes instruction figures out when physical registers can be reused again

# an OOO pipeline

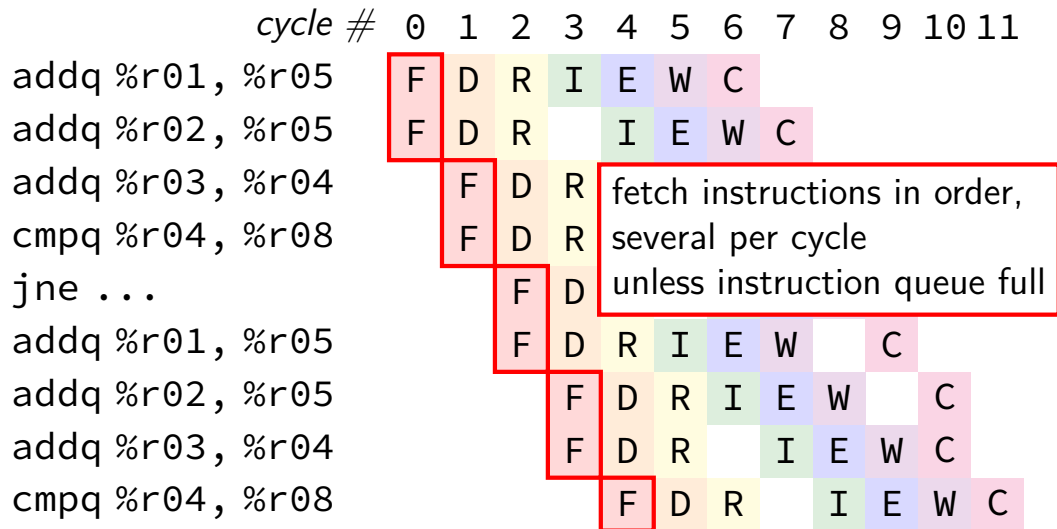


commit stage also handles branch misprediction  
*reorder buffer* tracks enough information to undo mispredicted instrs.  
also used to undo instructions after segfault, etc.

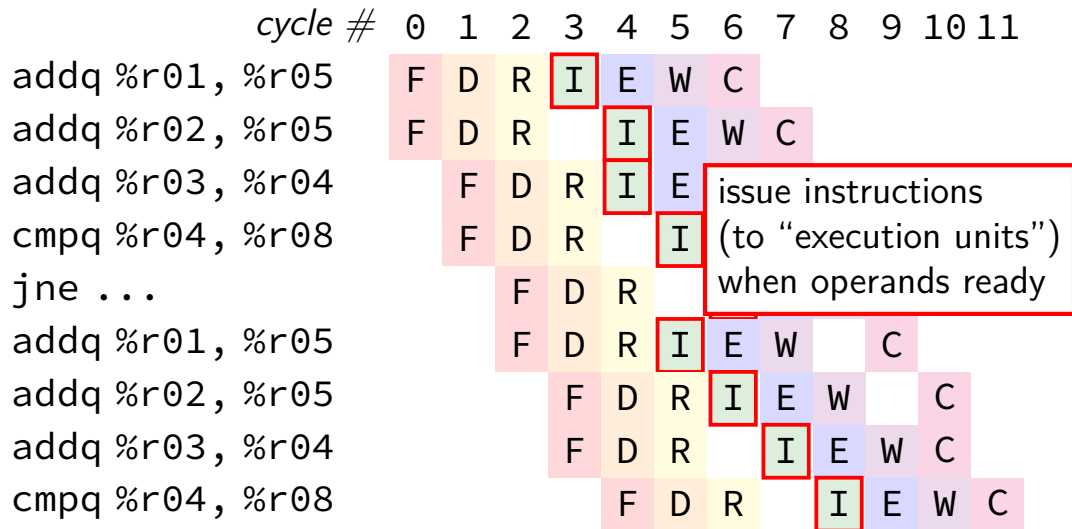
# an OOO pipeline diagram

|                 | <i>cycle #</i> | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|-----------------|----------------|---|---|---|---|---|---|---|---|---|---|----|----|
| addq %r01, %r05 |                | F | D | R | I | E | W | C |   |   |   |    |    |
| addq %r02, %r05 |                | F | D | R |   | I | E | W | C |   |   |    |    |
| addq %r03, %r04 |                |   | F | D | R | I | E | W | C |   |   |    |    |
| cmpq %r04, %r08 |                |   | F | D | R |   | I | E | W | C |   |    |    |
| jne ...         |                |   |   | F | D | R |   | I | E | W | C |    |    |
| addq %r01, %r05 |                |   |   | F | D | R | I | E | W |   | C |    |    |
| addq %r02, %r05 |                |   |   |   | F | D | R | I | E | W |   | C  |    |
| addq %r03, %r04 |                |   |   |   | F | D | R |   | I | E | W | C  |    |
| cmpq %r04, %r08 |                |   |   |   |   | F | D | R |   | I | E | W  | C  |

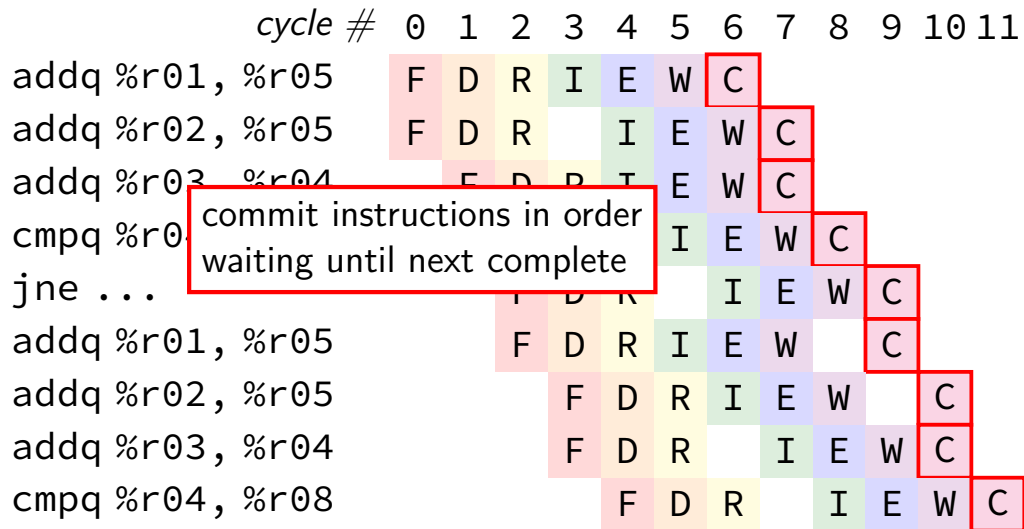
# an OOO pipeline diagram



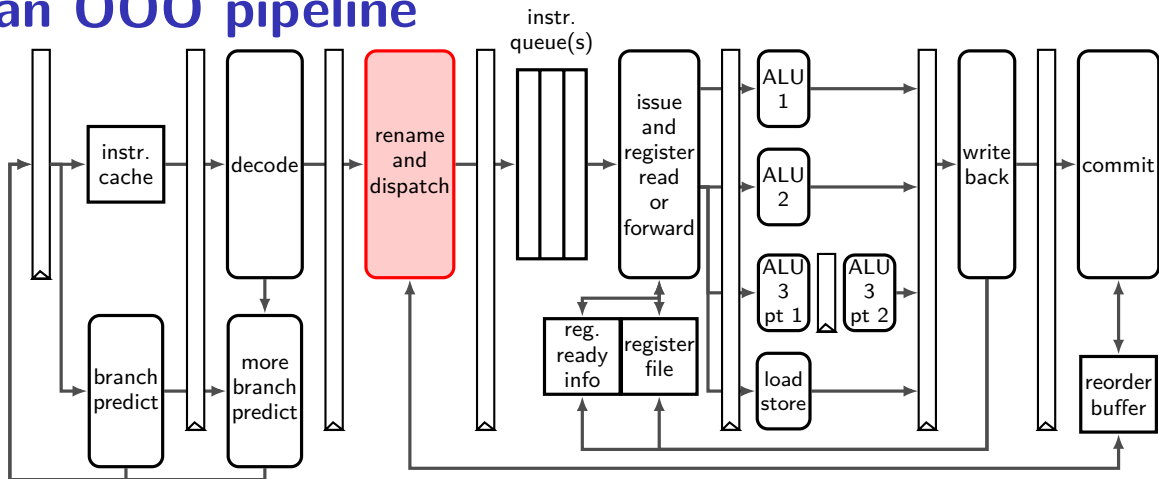
# an OOO pipeline diagram



# an OOO pipeline diagram



# an OOO pipeline



# register renaming

rename *architectural registers* to *physical registers*

architectural = part of instruction set architecture

different name for each version of architectural register

# register renaming state

| original          | renamed |
|-------------------|---------|
| add %r10, %r8 ... |         |
| add %r11, %r8 ... |         |
| add %r12, %r8 ... |         |

arch → phys register map

|      |      |
|------|------|
| %rax | %x04 |
| %rcx | %x09 |
| ...  | ...  |
| %r8  | %x13 |
| %r9  | %x17 |
| %r10 | %x19 |
| %r11 | %x07 |
| %r12 | %x05 |
| ...  | ...  |

free reg list

|      |
|------|
| %x18 |
| %x20 |
| %x21 |
| %x23 |
| %x24 |
| ...  |

# register renaming state

original

```
add %r10, %r8 ...  
add %r11, %r8 ...  
add %r12, %r8 ...
```

renamed

table for architectural (external)  
and physical (internal) name  
(for next instr. to process)

arch → phys register map

|      |      |
|------|------|
| %rax | %x04 |
| %rcx | %x09 |
| ...  | ...  |
| %r8  | %x13 |
| %r9  | %x17 |
| %r10 | %x19 |
| %r11 | %x07 |
| %r12 | %x05 |
| ...  | ...  |

free reg list

|      |
|------|
| %x18 |
| %x20 |
| %x21 |
| %x23 |
| %x24 |
| ...  |

# register renaming state

original  
add %r10, %r8 ...  
add %r11, %r8 ...  
add %r12, %r8 ...

renamed

list of available physical registers  
added to as instructions finish

arch → phys register map

|      |      |
|------|------|
| %rax | %x04 |
| %rcx | %x09 |
| ...  | ...  |
| %r8  | %x13 |
| %r9  | %x17 |
| %r10 | %x19 |
| %r11 | %x07 |
| %r12 | %x05 |
| ...  | ...  |

free reg list

|      |
|------|
| %x18 |
| %x20 |
| %x21 |
| %x23 |
| %x24 |
| ...  |

# register renaming example (1)

original

```
add %r10, %r8  
add %r11, %r8  
add %r12, %r8
```

renamed

arch → phys register map

|      |      |
|------|------|
| %rax | %x04 |
| %rcx | %x09 |
| ...  | ...  |
| %r8  | %x13 |
| %r9  | %x17 |
| %r10 | %x19 |
| %r11 | %x07 |
| %r12 | %x05 |
| ...  | ...  |

free reg list

|      |
|------|
| %x18 |
| %x20 |
| %x21 |
| %x23 |
| %x24 |
| ...  |

# register renaming example (1)

| original      | renamed               |
|---------------|-----------------------|
| add %r10, %r8 | add %x19, %x13 → %x18 |
| add %r11, %r8 |                       |
| add %r12, %r8 |                       |

arch → phys register map

|      |                      |
|------|----------------------|
| %rax | %x04                 |
| %rcx | %x09                 |
| ...  | ...                  |
| %r8  | <del>%x13</del> %x18 |
| %r9  | %x17                 |
| %r10 | %x19                 |
| %r11 | %x07                 |
| %r12 | %x05                 |
| ...  | ...                  |

free reg list

|                 |
|-----------------|
| <del>%x18</del> |
| %x20            |
| %x21            |
| %x23            |
| %x24            |
| ...             |

# register renaming example (1)

| original      | renamed               |
|---------------|-----------------------|
| add %r10, %r8 | add %x19, %x13 → %x18 |
| add %r11, %r8 | add %x07, %x18 → %x20 |
| add %r12, %r8 |                       |

arch → phys register map

|      |                           |
|------|---------------------------|
| %rax | %x04                      |
| %rcx | %x09                      |
| ...  | ...                       |
| %r8  | %x13% <del>x18</del> %x20 |
| %r9  | %x17                      |
| %r10 | %x19                      |
| %r11 | %x07                      |
| %r12 | %x05                      |
| ...  | ...                       |

free reg list

|                 |
|-----------------|
| %x18            |
| <del>%x20</del> |
| %x21            |
| %x23            |
| %x24            |
| ...             |

# register renaming example (1)

| original      | renamed               |
|---------------|-----------------------|
| add %r10, %r8 | add %x19, %x13 → %x18 |
| add %r11, %r8 | add %x07, %x18 → %x20 |
| add %r12, %r8 | add %x05, %x20 → %x21 |

arch → phys register map

|      |                               |
|------|-------------------------------|
| %rax | %x04                          |
| %rcx | %x09                          |
| ...  | ...                           |
| %r8  | %x13%x18% <del>x20</del> %x21 |
| %r9  | %x17                          |
| %r10 | %x19                          |
| %r11 | %x07                          |
| %r12 | %x05                          |
| ...  | ...                           |

free reg list

|      |
|------|
| %x18 |
| %x20 |
| %x21 |
| %x23 |
| %x24 |
| ...  |

# register renaming example (1)

| original      | renamed               |
|---------------|-----------------------|
| add %r10, %r8 | add %x19, %x13 → %x18 |
| add %r11, %r8 | add %x07, %x18 → %x20 |
| add %r12, %r8 | add %x05, %x20 → %x21 |

arch → phys register map

|      |                  |
|------|------------------|
| %rax | %x04             |
| %rcx | %x09             |
| ...  | ...              |
| %r8  | %x13%x18%x20%x21 |
| %r9  | %x17             |
| %r10 | %x19             |
| %r11 | %x07             |
| %r12 | %x05             |
| ...  | ...              |

free reg list

|      |
|------|
| %x18 |
| %x20 |
| %x21 |
| %x23 |
| %x24 |
| ...  |

## register renaming example (2)

original

renamed

```
addq %r10, %r8
movq %r8, (%rax)
subq %r8, %r11
movq 8(%r11), %r11
movq $100, %r8
addq %r11, %r8
```

arch → phys register map

|      |      |
|------|------|
| %rax | %x04 |
| %rcx | %x09 |
| ...  | ...  |
| %r8  | %x13 |
| %r9  | %x17 |
| %r10 | %x19 |
| %r11 | %x07 |
| %r12 | %x05 |
| %r13 | %x02 |
| ...  | ...  |

free  
regs

|      |
|------|
| %x18 |
| %x20 |
| %x21 |
| %x23 |
| %x24 |
| ...  |

## register renaming example (2)

original

```
addq %r10, %r8
movq %r8, (%rax)
subq %r8, %r11
movq 8(%r11), %r11
movq $100, %r8
addq %r11, %r8
```

renamed

```
addq %x19, %x13 → %x18
```

arch → phys register map

|      |                      |
|------|----------------------|
| %rax | %x04                 |
| %rcx | %x09                 |
| ...  | ...                  |
| %r8  | <del>%x13</del> %x18 |
| %r9  | %x17                 |
| %r10 | %x19                 |
| %r11 | %x07                 |
| %r12 | %x05                 |
| %r13 | %x02                 |
| ...  | ...                  |

free  
regs

|                 |
|-----------------|
| <del>%x18</del> |
| %x20            |
| %x21            |
| %x23            |
| %x24            |
| ...             |

## register renaming example (2)

original

```
addq %r10, %r8
movq %r8, (%rax)
subq %r8, %r11
movq 8(%r11), %r11
movq $100, %r8
addq %r11, %r8
```

renamed

```
addq %x19, %x13 → %x18
```

```
movq %x18, (%x04) → (memory)
```

arch → phys register map

|      |          |
|------|----------|
| %rax | %x04     |
| %rcx | %x09     |
| ...  | ...      |
| %r8  | %x13%x18 |
| %r9  | %x17     |
| %r10 | %x19     |
| %r11 | %x07     |
| %r12 | %x05     |
| %r13 | %x02     |
| ...  | ...      |

free  
regs

|      |
|------|
| %x18 |
| %x20 |
| %x21 |
| %x23 |
| %x24 |
| ...  |

## register renaming example (2)

| original                        | renamed                                   |
|---------------------------------|-------------------------------------------|
| <code>addq %r10, %r8</code>     | <code>addq %x19, %x13 → %x18</code>       |
| <code>movq %r8, (%rax)</code>   | <code>movq %x18, (%x04) → (memory)</code> |
| <code>subq %r8, %r11</code>     |                                           |
| <code>movq 8(%r11), %r11</code> |                                           |
| <code>movq \$100, %r8</code>    |                                           |
| <code>addq %r11, %r8</code>     |                                           |

arch → phys register map

|      |          |
|------|----------|
| %rax | %x04     |
| %rcx | %x09     |
| ...  | ...      |
| %r8  | %x13%x18 |
| %r9  | %x17     |
| %r10 | %x19     |
| %r11 | %x07     |
| %r12 | %x05     |
| %r13 | %x02     |
| ...  | ...      |

could be that  $\%rax = 8 + \%r11$   
could load before value written!  
possible data hazard!

not handled via register renaming

option 1: run load+stores in order

option 2: compare load/store addresses

|      |
|------|
| %x21 |
| %x23 |
| %x24 |
| ...  |

## register renaming example (2)

original

```
addq %r10, %r8
movq %r8, (%rax)
subq %r8, %r11
movq 8(%r11), %r11
movq $100, %r8
addq %r11, %r8
```

renamed

```
addq %x19, %x13 → %x18
movq %x18, (%x04) → (memory)
subq %x18, %x07 → %x20
```

arch → phys register map

|      |                      |
|------|----------------------|
| %rax | %x04                 |
| %rcx | %x09                 |
| ...  | ...                  |
| %r8  | %x13%x18             |
| %r9  | %x17                 |
| %r10 | %x19                 |
| %r11 | <del>%x07</del> %x20 |
| %r12 | %x05                 |
| %r13 | %x02                 |
| ...  | ...                  |

free  
regs

|                 |
|-----------------|
| %x18            |
| <del>%x20</del> |
| %x21            |
| %x23            |
| %x24            |
| ...             |

## register renaming example (2)

original

```
addq %r10, %r8
movq %r8, (%rax)
subq %r8, %r11
movq 8(%r11), %r11
movq $100, %r8
addq %r11, %r8
```

renamed

```
addq %x19, %x13 → %x18
movq %x18, (%x04) → (memory)
subq %x18, %x07 → %x20
movq 8(%x20), (memory) → %x21

```

arch → phys register map

|      |              |
|------|--------------|
| %rax | %x04         |
| %rcx | %x09         |
| ...  | ...          |
| %r8  | %x13%x18     |
| %r9  | %x17         |
| %r10 | %x19         |
| %r11 | %x07%x20%x21 |
| %r12 | %x05         |
| %r13 | %x02         |
| ...  | ...          |

free  
regs

|      |
|------|
| %x18 |
| %x20 |
| %x21 |
| %x23 |
| %x24 |
| ...  |

## register renaming example (2)

original

```
addq %r10, %r8
movq %r8, (%rax)
subq %r8, %r11
movq 8(%r11), %r11
movq $100, %r8
addq %r11, %r8
```

renamed

```
addq %x19, %x13 → %x18
movq %x18, (%x04) → (memory)
subq %x18, %x07 → %x20
movq 8(%x20), (memory) → %x21
movq $100 → %x23
```

arch → phys register map

|      |                           |
|------|---------------------------|
| %rax | %x04                      |
| %rcx | %x09                      |
| ...  | ...                       |
| %r8  | %x13 <del>%x18</del> %x23 |
| %r9  | %x17                      |
| %r10 | %x19                      |
| %r11 | %x07%x20%x21              |
| %r12 | %x05                      |
| %r13 | %x02                      |
| ...  | ...                       |

free  
regs

|                 |
|-----------------|
| %x18            |
| %x20            |
| %x21            |
| <del>%x23</del> |
| %x24            |
| ...             |

## register renaming example (2)

| original                        | renamed                                    |
|---------------------------------|--------------------------------------------|
| <code>addq %r10, %r8</code>     | <code>addq %x19, %x13 → %x18</code>        |
| <code>movq %r8, (%rax)</code>   | <code>movq %x18, (%x04) → (memory)</code>  |
| <code>subq %r8, %r11</code>     | <code>subq %x18, %x07 → %x20</code>        |
| <code>movq 8(%r11), %r11</code> | <code>movq 8(%x20), (memory) → %x21</code> |
| <code>movq \$100, %r8</code>    | <code>movq \$100 → %x23</code>             |
| <code>addq %r11, %r8</code>     | <code>addq %x21, %x23 → %x24</code>        |

arch → phys register map

|      |                               |
|------|-------------------------------|
| %rax | %x04                          |
| %rcx | %x09                          |
| ...  | ...                           |
| %r8  | %x13%x18% <del>x23</del> %x24 |
| %r9  | %x17                          |
| %r10 | %x19                          |
| %r11 | %x07%x20%x21                  |
| %r12 | %x05                          |
| %r13 | %x02                          |
| ...  | ...                           |

free  
regs

|                 |
|-----------------|
| %x18            |
| %x20            |
| %x21            |
| %x23            |
| <del>%x24</del> |
| ...             |

# register renaming exercise

original

```
addq %r8, %r9
movq $100, %r10
subq %r10, %r8
xorq %r8, %r9
andq %rax, %r9
```

arch → phys

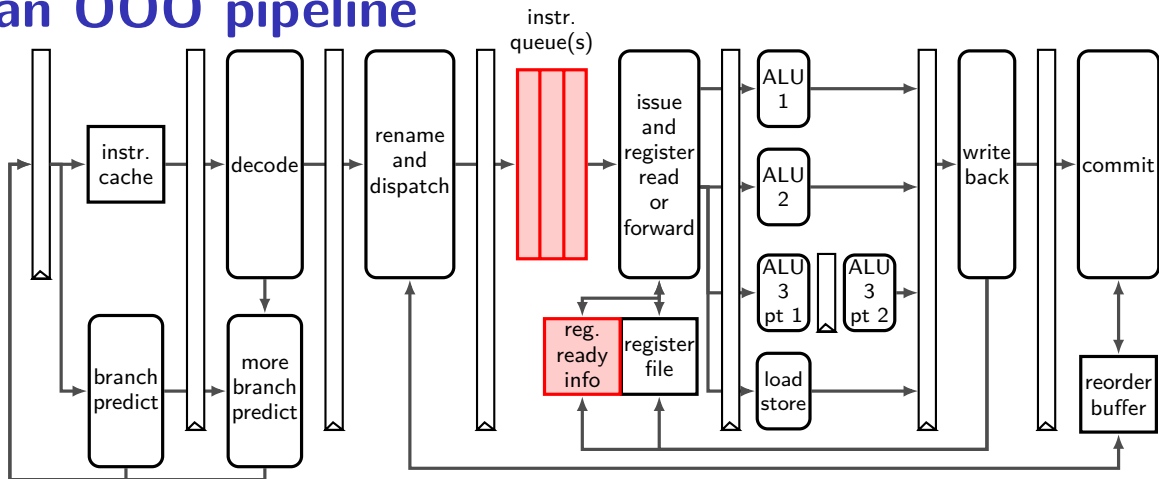
|      |      |
|------|------|
| %rax | %x04 |
| %rcx | %x09 |
| ...  | ...  |
| %r8  | %x13 |
| %r9  | %x17 |
| %r10 | %x19 |
| %r11 | %x29 |
| %r12 | %x05 |
| %r13 | %x02 |
| ...  | ...  |

renamed

free  
regs

|      |
|------|
| %x18 |
| %x20 |
| %x21 |
| %x23 |
| %x24 |
| ...  |

# an OOO pipeline



# instruction queue and dispatch

## instruction queue

| # | instruction                      |
|---|----------------------------------|
| 1 | <b>addq</b> %x01, %x05 → %x06    |
| 2 | <b>addq</b> %x02, %x06 → %x07    |
| 3 | <b>addq</b> %x03, %x07 → %x08    |
| 4 | <b>cmpq</b> %x04, %x08 → %x09.cc |
| 5 | <b>jne</b> %x09.cc, ...          |
| 6 | <b>addq</b> %x01, %x08 → %x10    |
| 7 | <b>addq</b> %x02, %x10 → %x11    |
| 8 | <b>addq</b> %x03, %x11 → %x12    |
| 9 | <b>cmpq</b> %x04, %x12 → %x13.cc |

... ..

*execution unit*

ALU 1

ALU 2

## scoreboard

| reg  | status  |
|------|---------|
| %x01 | ready   |
| %x02 | ready   |
| %x03 | ready   |
| %x04 | ready   |
| %x05 | ready   |
| %x06 | pending |
| %x07 | pending |
| %x08 | pending |
| %x09 | pending |
| %x10 | pending |
| %x11 | pending |
| %x12 | pending |
| %x13 | pending |
| ...  | ...     |

...

# instruction queue and dispatch

## instruction queue

| # | instruction                      |
|---|----------------------------------|
| 1 | <b>addq</b> %x01, %x05 → %x06    |
| 2 | <b>addq</b> %x02, %x06 → %x07    |
| 3 | <b>addq</b> %x03, %x07 → %x08    |
| 4 | <b>cmpq</b> %x04, %x08 → %x09.cc |
| 5 | <b>jne</b> %x09.cc, ...          |
| 6 | <b>addq</b> %x01, %x08 → %x10    |
| 7 | <b>addq</b> %x02, %x10 → %x11    |
| 8 | <b>addq</b> %x03, %x11 → %x12    |
| 9 | <b>cmpq</b> %x04, %x12 → %x13.cc |

... ..

## scoreboard

| reg  | status  |
|------|---------|
| %x01 | ready   |
| %x02 | ready   |
| %x03 | ready   |
| %x04 | ready   |
| %x05 | ready   |
| %x06 | pending |
| %x07 | pending |
| %x08 | pending |
| %x09 | pending |
| %x10 | pending |
| %x11 | pending |
| %x12 | pending |
| %x13 | pending |
| ...  | ...     |

execution unit    cycle# 1

ALU 1            1

ALU 2

...

# instruction queue and dispatch

## instruction queue

| # | instruction                            |
|---|----------------------------------------|
| 1 | <code>addq %x01, %x05 → %x06</code>    |
| 2 | <code>addq %x02, %x06 → %x07</code>    |
| 3 | <code>addq %x03, %x07 → %x08</code>    |
| 4 | <code>cmpq %x04, %x08 → %x09.cc</code> |
| 5 | <code>jne %x09.cc, ...</code>          |
| 6 | <code>addq %x01, %x08 → %x10</code>    |
| 7 | <code>addq %x02, %x10 → %x11</code>    |
| 8 | <code>addq %x03, %x11 → %x12</code>    |
| 9 | <code>cmpq %x04, %x12 → %x13.cc</code> |

... ..

## scoreboard

| reg  | status  |
|------|---------|
| %x01 | ready   |
| %x02 | ready   |
| %x03 | ready   |
| %x04 | ready   |
| %x05 | ready   |
| %x06 | pending |
| %x07 | pending |
| %x08 | pending |
| %x09 | pending |
| %x10 | pending |
| %x11 | pending |
| %x12 | pending |
| %x13 | pending |
| ...  | ...     |

*execution unit*    *cycle#* 1  
 ALU 1                    1  
 ALU 2

...

# instruction queue and dispatch

## instruction queue

| # | instruction                      |
|---|----------------------------------|
| 1 | <b>addq</b> %x01, %x05 → %x06    |
| 2 | <b>addq</b> %x02, %x06 → %x07    |
| 3 | <b>addq</b> %x03, %x07 → %x08    |
| 4 | <b>cmpq</b> %x04, %x08 → %x09.cc |
| 5 | <b>jne</b> %x09.cc, ...          |
| 6 | <b>addq</b> %x01, %x08 → %x10    |
| 7 | <b>addq</b> %x02, %x10 → %x11    |
| 8 | <b>addq</b> %x03, %x11 → %x12    |
| 9 | <b>cmpq</b> %x04, %x12 → %x13.cc |

... ..

## scoreboard

| reg  | status                   |
|------|--------------------------|
| %x01 | ready                    |
| %x02 | ready                    |
| %x03 | ready                    |
| %x04 | ready                    |
| %x05 | ready                    |
| %x06 | <del>pending</del> ready |
| %x07 | pending                  |
| %x08 | pending                  |
| %x09 | pending                  |
| %x10 | pending                  |
| %x11 | pending                  |
| %x12 | pending                  |
| %x13 | pending                  |
| ...  | ...                      |

execution unit    cycle# 1

ALU 1                    1

ALU 2                    —

...

# instruction queue and dispatch

## instruction queue

| #   | instruction                       |
|-----|-----------------------------------|
| 1   | <del>addq %x01, %x05 → %x06</del> |
| 2   | addq %x02, %x06 → %x07            |
| 3   | addq %x03, %x07 → %x08            |
| 4   | cmpq %x04, %x08 → %x09.cc         |
| 5   | jne %x09.cc, ...                  |
| 6   | addq %x01, %x08 → %x10            |
| 7   | addq %x02, %x10 → %x11            |
| 8   | addq %x03, %x11 → %x12            |
| 9   | cmpq %x04, %x12 → %x13.cc         |
| ... | ...                               |

## scoreboard

| reg  | status        |
|------|---------------|
| %x01 | ready         |
| %x02 | ready         |
| %x03 | ready         |
| %x04 | ready         |
| %x05 | ready         |
| %x06 | pending ready |
| %x07 | pending ready |
| %x08 | pending       |
| %x09 | pending       |
| %x10 | pending       |
| %x11 | pending       |
| %x12 | pending       |
| %x13 | pending       |
| ...  | ...           |

| execution unit | cycle# | 1 | 2 |
|----------------|--------|---|---|
| ALU 1          |        | 1 | 2 |
| ALU 2          |        | — | — |

...

# instruction queue and dispatch

instruction queue

| #   | instruction                       |
|-----|-----------------------------------|
| 1   | <del>addq %x01, %x05 → %x06</del> |
| 2   | <del>addq %x02, %x06 → %x07</del> |
| 3   | addq %x03, %x07 → %x08            |
| 4   | cmpq %x04, %x08 → %x09.cc         |
| 5   | jne %x09.cc, ...                  |
| 6   | addq %x01, %x08 → %x10            |
| 7   | addq %x02, %x10 → %x11            |
| 8   | addq %x03, %x11 → %x12            |
| 9   | cmpq %x04, %x12 → %x13.cc         |
| ... | ...                               |

scoreboard

| reg  | status        |
|------|---------------|
| %x01 | ready         |
| %x02 | ready         |
| %x03 | ready         |
| %x04 | ready         |
| %x05 | ready         |
| %x06 | pending ready |
| %x07 | pending ready |
| %x08 | pending ready |
| %x09 | pending       |
| %x10 | pending       |
| %x11 | pending       |
| %x12 | pending       |
| %x13 | pending       |
| ...  | ...           |

| execution unit | cycle# 1 | 2 | 3 | ... |
|----------------|----------|---|---|-----|
| ALU 1          | 1        | 2 | 3 |     |
| ALU 2          | —        | — | — |     |

# instruction queue and dispatch

instruction queue

| #   | instruction                       |
|-----|-----------------------------------|
| 1   | <del>addq %x01, %x05 → %x06</del> |
| 2   | <del>addq %x02, %x06 → %x07</del> |
| 3   | <del>addq %x03, %x07 → %x08</del> |
| 4   | cmpq %x04, %x08 → %x09.cc         |
| 5   | jne %x09.cc, ...                  |
| 6   | addq %x01, %x08 → %x10            |
| 7   | addq %x02, %x10 → %x11            |
| 8   | addq %x03, %x11 → %x12            |
| 9   | cmpq %x04, %x12 → %x13.cc         |
| ... | ...                               |

scoreboard

| reg  | status                   |
|------|--------------------------|
| %x01 | ready                    |
| %x02 | ready                    |
| %x03 | ready                    |
| %x04 | ready                    |
| %x05 | ready                    |
| %x06 | <del>pending</del> ready |
| %x07 | <del>pending</del> ready |
| %x08 | <del>pending</del> ready |
| %x09 | pending                  |
| %x10 | pending                  |
| %x11 | pending                  |
| %x12 | pending                  |
| %x13 | pending                  |
| ...  | ...                      |

| execution unit | cycle# | 1 | 2 | 3 | ... |
|----------------|--------|---|---|---|-----|
| ALU 1          |        | 1 | 2 | 3 |     |
| ALU 2          |        | — | — | — |     |

# instruction queue and dispatch

instruction queue

| #   | instruction                       |
|-----|-----------------------------------|
| 1   | <del>addq %x01, %x05 → %x06</del> |
| 2   | <del>addq %x02, %x06 → %x07</del> |
| 3   | <del>addq %x03, %x07 → %x08</del> |
| 4   | cmpq %x04, %x08 → %x09.cc         |
| 5   | jne %x09.cc, ...                  |
| 6   | addq %x01, %x08 → %x10            |
| 7   | addq %x02, %x10 → %x11            |
| 8   | addq %x03, %x11 → %x12            |
| 9   | cmpq %x04, %x12 → %x13.cc         |
| ... | ...                               |

scoreboard

| reg  | status        |
|------|---------------|
| %x01 | ready         |
| %x02 | ready         |
| %x03 | ready         |
| %x04 | ready         |
| %x05 | ready         |
| %x06 | pending ready |
| %x07 | pending ready |
| %x08 | pending ready |
| %x09 | pending ready |
| %x10 | pending ready |
| %x11 | pending       |
| %x12 | pending       |
| %x13 | pending       |
| ...  | ...           |

| execution unit | cycle# | 1 | 2 | 3 | 4 | ... |
|----------------|--------|---|---|---|---|-----|
| ALU 1          |        | 1 | 2 | 3 | 4 |     |
| ALU 2          |        | — | — | — | 6 |     |

# instruction queue and dispatch

instruction queue

| #   | instruction                          |
|-----|--------------------------------------|
| 1   | <del>addq %x01, %x05 → %x06</del>    |
| 2   | <del>addq %x02, %x06 → %x07</del>    |
| 3   | <del>addq %x03, %x07 → %x08</del>    |
| 4   | <del>cmpq %x04, %x08 → %x09.cc</del> |
| 5   | jne %x09.cc, ...                     |
| 6   | <del>addq %x01, %x08 → %x10</del>    |
| 7   | addq %x02, %x10 → %x11               |
| 8   | addq %x03, %x11 → %x12               |
| 9   | cmpq %x04, %x12 → %x13.cc            |
| ... | ...                                  |

scoreboard

| reg  | status        |
|------|---------------|
| %x01 | ready         |
| %x02 | ready         |
| %x03 | ready         |
| %x04 | ready         |
| %x05 | ready         |
| %x06 | pending ready |
| %x07 | pending ready |
| %x08 | pending ready |
| %x09 | pending ready |
| %x10 | pending ready |
| %x11 | pending       |
| %x12 | pending       |
| %x13 | pending       |
| ...  | ...           |

| execution unit | cycle# | 1 | 2 | 3 | 4 | ... |
|----------------|--------|---|---|---|---|-----|
| ALU 1          |        | 1 | 2 | 3 | 4 |     |
| ALU 2          |        | — | — | — | 6 |     |

# instruction queue and dispatch

instruction queue

| #   | instruction                          |
|-----|--------------------------------------|
| 1   | <del>addq %x01, %x05 → %x06</del>    |
| 2   | <del>addq %x02, %x06 → %x07</del>    |
| 3   | <del>addq %x03, %x07 → %x08</del>    |
| 4   | <del>cmpq %x04, %x08 → %x09.cc</del> |
| 5   | <del>jne %x09.cc, ...</del>          |
| 6   | <del>addq %x01, %x08 → %x10</del>    |
| 7   | <del>addq %x02, %x10 → %x11</del>    |
| 8   | addq %x03, %x11 → %x12               |
| 9   | cmpq %x04, %x12 → %x13.cc            |
| ... | ...                                  |

scoreboard

| reg  | status               |
|------|----------------------|
| %x01 | ready                |
| %x02 | ready                |
| %x03 | ready                |
| %x04 | ready                |
| %x05 | ready                |
| %x06 | pending ready        |
| %x07 | pending ready        |
| %x08 | pending ready        |
| %x09 | pending ready        |
| %x10 | pending ready        |
| %x11 | pending <b>ready</b> |
| %x12 | pending              |
| %x13 | pending              |
| ...  | ...                  |

| execution unit | cycle# | 1 | 2 | 3 | 4 | 5        | ... |
|----------------|--------|---|---|---|---|----------|-----|
| ALU 1          |        | 1 | 2 | 3 | 4 | <b>5</b> |     |
| ALU 2          |        | — | — | — | 6 | <b>7</b> |     |

# instruction queue and dispatch

instruction queue

| #   | instruction                          |
|-----|--------------------------------------|
| 1   | <del>addq %x01, %x05 → %x06</del>    |
| 2   | <del>addq %x02, %x06 → %x07</del>    |
| 3   | <del>addq %x03, %x07 → %x08</del>    |
| 4   | <del>cmpq %x04, %x08 → %x09.cc</del> |
| 5   | <del>jne %x09.cc, ...</del>          |
| 6   | <del>addq %x01, %x08 → %x10</del>    |
| 7   | <del>addq %x02, %x10 → %x11</del>    |
| 8   | <del>addq %x03, %x11 → %x12</del>    |
| 9   | cmpq %x04, %x12 → %x13.cc            |
| ... | ...                                  |

scoreboard

| reg  | status        |
|------|---------------|
| %x01 | ready         |
| %x02 | ready         |
| %x03 | ready         |
| %x04 | ready         |
| %x05 | ready         |
| %x06 | pending ready |
| %x07 | pending ready |
| %x08 | pending ready |
| %x09 | pending ready |
| %x10 | pending ready |
| %x11 | pending ready |
| %x12 | pending       |
| %x13 | pending       |
| ...  | ...           |

| execution unit | cycle# | 1 | 2 | 3 | 4 | 5 | 6 | ... |
|----------------|--------|---|---|---|---|---|---|-----|
| ALU 1          |        | 1 | 2 | 3 | 4 | 5 | 8 |     |
| ALU 2          |        | — | — | — | 6 | 7 | — |     |

# instruction queue and dispatch

instruction queue

| #   | instruction                          |
|-----|--------------------------------------|
| 1   | <del>addq %x01, %x05 → %x06</del>    |
| 2   | <del>addq %x02, %x06 → %x07</del>    |
| 3   | <del>addq %x03, %x07 → %x08</del>    |
| 4   | <del>cmpq %x04, %x08 → %x09.cc</del> |
| 5   | <del>jne %x09.cc, ...</del>          |
| 6   | <del>addq %x01, %x08 → %x10</del>    |
| 7   | <del>addq %x02, %x10 → %x11</del>    |
| 8   | <del>addq %x03, %x11 → %x12</del>    |
| 9   | <del>cmpq %x04, %x12 → %x13.cc</del> |
| ... | ...                                  |

scoreboard

| reg  | status               |
|------|----------------------|
| %x01 | ready                |
| %x02 | ready                |
| %x03 | ready                |
| %x04 | ready                |
| %x05 | ready                |
| %x06 | pending ready        |
| %x07 | pending ready        |
| %x08 | pending ready        |
| %x09 | pending ready        |
| %x10 | pending ready        |
| %x11 | pending ready        |
| %x12 | pending <b>ready</b> |
| %x13 | pending              |
| ...  | ...                  |

| execution unit | cycle# | 1 | 2 | 3 | 4 | 5 | 6 | 7          | ... |
|----------------|--------|---|---|---|---|---|---|------------|-----|
| ALU 1          |        | 1 | 2 | 3 | 4 | 5 | 8 | <b>9</b>   |     |
| ALU 2          |        | — | — | — | 6 | 7 | — | <b>...</b> |     |

# instruction queue and dispatch

instruction queue

| # | instruction                          |
|---|--------------------------------------|
| 1 | <del>addq %x01, %x05 → %x06</del>    |
| 2 | <del>addq %x02, %x06 → %x07</del>    |
| 3 | <del>addq %x03, %x07 → %x08</del>    |
| 4 | <del>cmpq %x04, %x08 → %x09.cc</del> |
| 5 | <del>jne %x09.cc, ...</del>          |
| 6 | <del>addq %x01, %x08 → %x10</del>    |
| 7 | <del>addq %x02, %x10 → %x11</del>    |
| 8 | <del>addq %x03, %x11 → %x12</del>    |
| 9 | <del>cmpq %x04, %x12 → %x13.cc</del> |

... ..

scoreboard

| reg  | status        |
|------|---------------|
| %x01 | ready         |
| %x02 | ready         |
| %x03 | ready         |
| %x04 | ready         |
| %x05 | ready         |
| %x06 | pending ready |
| %x07 | pending ready |
| %x08 | pending ready |
| %x09 | pending ready |
| %x10 | pending ready |
| %x11 | pending ready |
| %x12 | pending ready |
| %x13 | pending ready |
| ...  | ...           |

| execution unit | cycle# | 1 | 2 | 3 | 4 | 5 | 6 | 7   | ... |
|----------------|--------|---|---|---|---|---|---|-----|-----|
| ALU 1          |        | 1 | 2 | 3 | 4 | 5 | 8 | 9   |     |
| ALU 2          |        | — | — | — | 6 | 7 | — | ... |     |

# instruction queue and dispatch

instruction queue

| # | instruction                   |
|---|-------------------------------|
| 1 | <b>movq</b> (%x04) → %x06     |
| 2 | <b>movq</b> (%x05) → %x07     |
| 3 | <b>addq</b> %x01, %x02 → %x08 |
| 4 | <b>addq</b> %x01, %x06 → %x09 |
| 5 | <b>addq</b> %x01, %x07 → %x10 |

... ..

scoreboard

| reg  | status |
|------|--------|
| %x01 | ready  |
| %x02 | ready  |
| %x03 | ready  |
| %x04 | ready  |
| %x05 | ready  |
| %x06 |        |
| %x07 |        |
| %x08 |        |
| %x09 |        |
| %x10 |        |
| ...  | ...    |

execution unit  
ALU

data cache

↑  
assume

1 cycle/access

cycle# 1

2

3

4

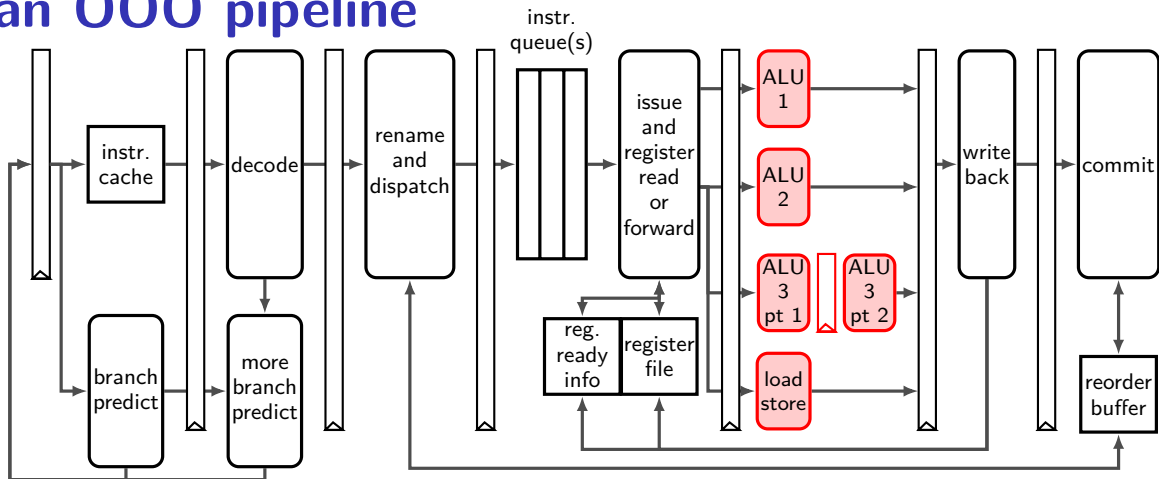
5

6

7

...

# an OOO pipeline



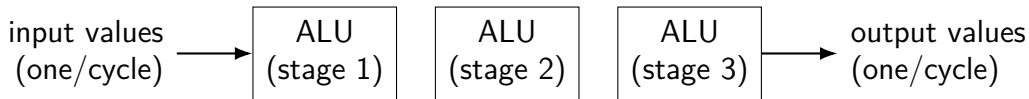
# execution units AKA functional units (1)

where actual work of instruction is done

e.g. the actual ALU, or data cache

sometimes pipelined:

(here: 1 op/cycle; 3 cycle latency)



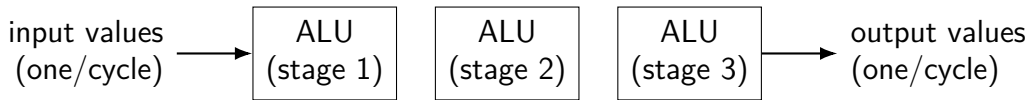
# execution units AKA functional units (1)

where actual work of instruction is done

e.g. the actual ALU, or data cache

sometimes pipelined:

(here: 1 op/cycle; 3 cycle latency)



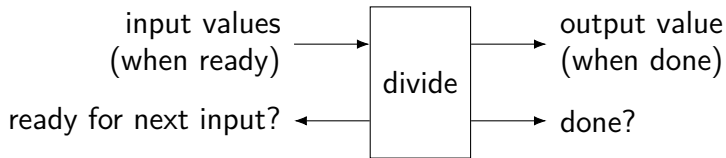
exercise: how long to compute  $A \times (B \times (C \times D))$ ?

## execution units AKA functional units (2)

where actual work of instruction is done

e.g. the actual ALU, or data cache

sometimes unpipelined:



# instruction queue and dispatch (multicycle)

scoreboard

instruction queue

| #  | instruction                     |
|----|---------------------------------|
| 1  | <b>add</b> %x01, %x02 → %x03    |
| 2  | <b>imul</b> %x04, %x05 → %x06   |
| 3  | <b>imul</b> %x03, %x07 → %x08   |
| 4  | <b>cmp</b> %x03, %x08 → %x09.cc |
| 5  | <b>jle</b> %x09.cc, ...         |
| 6  | <b>add</b> %x01, %x03 → %x11    |
| 7  | <b>imul</b> %x04, %x06 → %x12   |
| 8  | <b>imul</b> %x03, %x08 → %x13   |
| 9  | <b>cmp</b> %x11, %x13 → %x14.cc |
| 10 | <b>jle</b> %x14.cc, ...         |

... ..

execution unit

ALU 1 (add, cmp, jxx)

ALU 2 (add, cmp, jxx)

ALU 3 (mul) start

ALU 3 (mul) end

| reg  | status  |
|------|---------|
| %x01 | ready   |
| %x02 | ready   |
| %x03 | pending |
| %x04 | ready   |
| %x05 | ready   |
| %x06 | pending |
| %x07 | ready   |
| %x08 | pending |
| %x09 | pending |
| %x10 | pending |
| %x11 | pending |
| %x12 | pending |
| %x13 | pending |
| %x14 | pending |
| ...  | ... ..  |

# instruction queue and dispatch (multicycle)

scoreboard

instruction queue

| #  | instruction                     |
|----|---------------------------------|
| 1  | <b>add</b> %x01, %x02 → %x03    |
| 2  | <b>imul</b> %x04, %x05 → %x06   |
| 3  | <b>imul</b> %x03, %x07 → %x08   |
| 4  | <b>cmp</b> %x03, %x08 → %x09.cc |
| 5  | <b>jle</b> %x09.cc, ...         |
| 6  | <b>add</b> %x01, %x03 → %x11    |
| 7  | <b>imul</b> %x04, %x06 → %x12   |
| 8  | <b>imul</b> %x03, %x08 → %x13   |
| 9  | <b>cmp</b> %x11, %x13 → %x14.cc |
| 10 | <b>jle</b> %x14.cc, ...         |

... ..

execution unit

ALU 1 (add, cmp, jxx)

ALU 2 (add, cmp, jxx)

ALU 3 (mul) start

ALU 3 (mul) end

| reg  | status  |
|------|---------|
| %x01 | ready   |
| %x02 | ready   |
| %x03 | pending |
| %x04 | ready   |
| %x05 | ready   |
| %x06 | pending |
| %x07 | ready   |
| %x08 | pending |
| %x09 | pending |
| %x10 | pending |
| %x11 | pending |
| %x12 | pending |
| %x13 | pending |
| %x14 | pending |
| ...  | ... ..  |

# instruction queue and dispatch (multicycle)

scoreboard

instruction queue

| #  | instruction                     |
|----|---------------------------------|
| 1  | <b>add</b> %x01, %x02 → %x03    |
| 2  | <b>imul</b> %x04, %x05 → %x06   |
| 3  | <b>imul</b> %x03, %x07 → %x08   |
| 4  | <b>cmp</b> %x03, %x08 → %x09.cc |
| 5  | <b>jle</b> %x09.cc, ...         |
| 6  | <b>add</b> %x01, %x03 → %x11    |
| 7  | <b>imul</b> %x04, %x06 → %x12   |
| 8  | <b>imul</b> %x03, %x08 → %x13   |
| 9  | <b>cmp</b> %x11, %x13 → %x14.cc |
| 10 | <b>jle</b> %x14.cc, ...         |

... ..

| execution unit        | cycle# |
|-----------------------|--------|
| ALU 1 (add, cmp, jxx) | 1      |
| ALU 2 (add, cmp, jxx) | —      |
| ALU 3 (mul) start     | 2      |
| ALU 3 (mul) end       | 2      |

| reg  | status  |
|------|---------|
| %x01 | ready   |
| %x02 | ready   |
| %x03 | pending |
| %x04 | ready   |
| %x05 | ready   |
| %x06 | pending |
| %x07 | ready   |
| %x08 | pending |
| %x09 | pending |
| %x10 | pending |
| %x11 | pending |
| %x12 | pending |
| %x13 | pending |
| %x14 | pending |
| ...  | ... ..  |

# instruction queue and dispatch (multicycle)

scoreboard

instruction queue

| #  | instruction                       |
|----|-----------------------------------|
| 1  | <del>add %x01, %x02 → %x03</del>  |
| 2  | <del>imul %x04, %x05 → %x06</del> |
| 3  | imul %x03, %x07 → %x08            |
| 4  | cmp %x03, %x08 → %x09.cc          |
| 5  | jle %x09.cc, ...                  |
| 6  | add %x01, %x03 → %x11             |
| 7  | imul %x04, %x06 → %x12            |
| 8  | imul %x03, %x08 → %x13            |
| 9  | cmp %x11, %x13 → %x14.cc          |
| 10 | jle %x14.cc, ...                  |

... ..

| execution unit        | cycle# | 1 | 2 |
|-----------------------|--------|---|---|
| ALU 1 (add, cmp, jxx) | 1      | 6 |   |
| ALU 2 (add, cmp, jxx) | —      | — |   |
| ALU 3 (mul) start     | 2      | 3 |   |
| ALU 3 (mul) end       |        | 2 | 3 |

| reg  | status          |
|------|-----------------|
| %x01 | ready           |
| %x02 | ready           |
| %x03 | pending ready   |
| %x04 | ready           |
| %x05 | ready           |
| %x06 | pending (still) |
| %x07 | ready           |
| %x08 | pending         |
| %x09 | pending         |
| %x10 | pending         |
| %x11 | pending         |
| %x12 | pending         |
| %x13 | pending         |
| %x14 | pending         |
| ...  | ... ..          |

# instruction queue and dispatch (multicycle)

scoreboard

instruction queue

| #  | instruction                       |
|----|-----------------------------------|
| 1  | <del>add %x01, %x02 → %x03</del>  |
| 2  | <del>imul %x04, %x05 → %x06</del> |
| 3  | <del>imul %x03, %x07 → %x08</del> |
| 4  | cmp %x03, %x08 → %x09.cc          |
| 5  | jle %x09.cc, ...                  |
| 6  | <del>add %x01, %x03 → %x11</del>  |
| 7  | imul %x04, %x06 → %x12            |
| 8  | imul %x03, %x08 → %x13            |
| 9  | cmp %x11, %x13 → %x14.cc          |
| 10 | jle %x14.cc, ...                  |

... ..

| execution unit        | cycle# | 1 | 2 | 3 |
|-----------------------|--------|---|---|---|
| ALU 1 (add, cmp, jxx) | 1      | 6 | — | — |
| ALU 2 (add, cmp, jxx) | —      | — | — | — |
| ALU 3 (mul) start     | 2      | 3 | 7 | — |
| ALU 3 (mul) end       |        | 2 | 3 | 7 |

| reg  | status          |
|------|-----------------|
| %x01 | ready           |
| %x02 | ready           |
| %x03 | pending ready   |
| %x04 | ready           |
| %x05 | ready           |
| %x06 | pending ready   |
| %x07 | ready           |
| %x08 | pending (still) |
| %x09 | pending         |
| %x10 | pending         |
| %x11 | pending ready   |
| %x12 | pending         |
| %x13 | pending         |
| %x14 | pending         |
| ...  | ... ..          |

# instruction queue and dispatch (multicycle)

scoreboard

instruction queue

| #  | instruction                         |
|----|-------------------------------------|
| 1  | <del>add %x01, %x02 → %x03</del>    |
| 2  | <del>imul %x04, %x05 → %x06</del>   |
| 3  | <del>imul %x03, %x07 → %x08</del>   |
| 4  | <del>cmp %x03, %x08 → %x09.cc</del> |
| 5  | jle %x09.cc, ...                    |
| 6  | <del>add %x01, %x03 → %x11</del>    |
| 7  | <del>imul %x04, %x06 → %x12</del>   |
| 8  | imul %x03, %x08 → %x13              |
| 9  | cmp %x11, %x13 → %x14.cc            |
| 10 | jle %x14.cc, ...                    |

... ..

| execution unit        | cycle# | 1 | 2 | 3 | 4 |
|-----------------------|--------|---|---|---|---|
| ALU 1 (add, cmp, jxx) | 1      | 6 | — | — | 4 |
| ALU 2 (add, cmp, jxx) | —      | — | — | — | — |
| ALU 3 (mul) start     | 2      | 3 | 7 | 8 |   |
| ALU 3 (mul) end       |        | 2 | 3 | 7 | 8 |

| reg  | status          |
|------|-----------------|
| %x01 | ready           |
| %x02 | ready           |
| %x03 | pending ready   |
| %x04 | ready           |
| %x05 | ready           |
| %x06 | pending ready   |
| %x07 | ready           |
| %x08 | pending ready   |
| %x09 | pending ready   |
| %x10 | pending         |
| %x11 | pending ready   |
| %x12 | pending (still) |
| %x13 | pending         |
| %x14 | pending         |
| ...  | ... ..          |

# instruction queue and dispatch (multicycle)

scoreboard

instruction queue

| #  | instruction                         |
|----|-------------------------------------|
| 1  | <del>add %x01, %x02 → %x03</del>    |
| 2  | <del>imul %x04, %x05 → %x06</del>   |
| 3  | <del>imul %x03, %x07 → %x08</del>   |
| 4  | <del>cmp %x03, %x08 → %x09.cc</del> |
| 5  | <del>jle %x09.cc, ...</del>         |
| 6  | <del>add %x01, %x03 → %x11</del>    |
| 7  | <del>imul %x04, %x06 → %x12</del>   |
| 8  | imul %x03, %x08 → %x13              |
| 9  | cmp %x11, %x13 → %x14.cc            |
| 10 | jle %x14.cc, ...                    |

... ..

| execution unit        | cycle# | 1 | 2 | 3 | 4 | 5 |
|-----------------------|--------|---|---|---|---|---|
| ALU 1 (add, cmp, jxx) | 1      | 6 | — | 4 | 5 | 5 |
| ALU 2 (add, cmp, jxx) | —      | — | — | — | — | — |
| ALU 3 (mul) start     | 2      | 3 | 7 | 8 | — | — |
| ALU 3 (mul) end       |        | 2 | 3 | 7 | 8 |   |

| reg  | status          |
|------|-----------------|
| %x01 | ready           |
| %x02 | ready           |
| %x03 | pending ready   |
| %x04 | ready           |
| %x05 | ready           |
| %x06 | pending ready   |
| %x07 | ready           |
| %x08 | pending ready   |
| %x09 | pending ready   |
| %x10 | pending         |
| %x11 | pending ready   |
| %x12 | pending ready   |
| %x13 | pending (still) |
| %x14 | pending         |
| ...  | ... ..          |

# instruction queue and dispatch (multicycle)

scoreboard

instruction queue

| #  | instruction                         |
|----|-------------------------------------|
| 1  | <del>add %x01, %x02 → %x03</del>    |
| 2  | <del>imul %x04, %x05 → %x06</del>   |
| 3  | <del>imul %x03, %x07 → %x08</del>   |
| 4  | <del>cmp %x03, %x08 → %x09.cc</del> |
| 5  | <del>jle %x09.cc, ...</del>         |
| 6  | <del>add %x01, %x03 → %x11</del>    |
| 7  | <del>imul %x04, %x06 → %x12</del>   |
| 8  | <del>imul %x03, %x08 → %x13</del>   |
| 9  | cmp %x11, %x13 → %x14.cc            |
| 10 | jle %x14.cc, ...                    |

... ..

| execution unit        | cycle# | 1 | 2 | 3 | 4 | 5 |
|-----------------------|--------|---|---|---|---|---|
| ALU 1 (add, cmp, jxx) | 1      | 6 | — | 4 | 5 |   |
| ALU 2 (add, cmp, jxx) | —      | — | — | — | — |   |
| ALU 3 (mul) start     | 2      | 3 | 7 | 8 | — |   |
| ALU 3 (mul) end       |        | 2 | 3 | 7 | 8 |   |

| reg  | status        |
|------|---------------|
| %x01 | ready         |
| %x02 | ready         |
| %x03 | pending ready |
| %x04 | ready         |
| %x05 | ready         |
| %x06 | pending ready |
| %x07 | ready         |
| %x08 | pending ready |
| %x09 | pending ready |
| %x10 | pending       |
| %x11 | pending ready |
| %x12 | pending ready |
| %x13 | pending ready |
| %x14 | pending       |
| ...  | ... ..        |

# instruction queue and dispatch (multicycle)

scoreboard

instruction queue

| #  | instruction                         |
|----|-------------------------------------|
| 1  | <del>add %x01, %x02 → %x03</del>    |
| 2  | <del>imul %x04, %x05 → %x06</del>   |
| 3  | <del>imul %x03, %x07 → %x08</del>   |
| 4  | <del>cmp %x03, %x08 → %x09.cc</del> |
| 5  | <del>jle %x09.cc, ...</del>         |
| 6  | add %x01, %x03 → %x11               |
| 7  | imul %x04, %x06 → %x12              |
| 8  | imul %x03, %x08 → %x13              |
| 9  | cmp %x11, %x13 → %x14.cc            |
| 10 | jle %x14.cc, ...                    |

... ..

| execution unit        | cycle# | 1 | 2 | 3 | 4 | 5 |
|-----------------------|--------|---|---|---|---|---|
| ALU 1 (add, cmp, jxx) |        | 1 | 6 | — | 4 | 5 |
| ALU 2 (add, cmp, jxx) |        | — | — | — | — | — |
| ALU 3 (mul) start     |        | 2 | 3 | 7 | 8 | — |
| ALU 3 (mul) end       |        |   | 2 | 3 | 7 | 8 |

| reg  | status        |
|------|---------------|
| %x01 | ready         |
| %x02 | ready         |
| %x03 | pending ready |
| %x04 | ready         |
| %x05 | ready         |
| %x06 | pending ready |
| %x07 | ready         |
| %x08 | pending ready |
| %x09 | pending ready |
| %x10 | pending       |
| %x11 | pending ready |
| %x12 | pending ready |
| %x13 | pending ready |
| %x14 | pending ready |
| ...  | ...           |

6  
9  
—

# instruction queue and dispatch (multicycle)

scoreboard

instruction queue

| #  | instruction                         |
|----|-------------------------------------|
| 1  | <del>add %x01, %x02 → %x03</del>    |
| 2  | <del>imul %x04, %x05 → %x06</del>   |
| 3  | <del>imul %x03, %x07 → %x08</del>   |
| 4  | <del>cmp %x03, %x08 → %x09.cc</del> |
| 5  | <del>jle %x09.cc, ...</del>         |
| 6  | <del>add %x01, %x03 → %x11</del>    |
| 7  | <del>imul %x04, %x06 → %x12</del>   |
| 8  | <del>imul %x03, %x08 → %x13</del>   |
| 9  | <del>cmp %x11, %x13 → %x14.cc</del> |
| 10 | <del>jle %x14.cc, ...</del>         |

... ..

| execution unit        | cycle# | 1 | 2 | 3 | 4 | 5 |
|-----------------------|--------|---|---|---|---|---|
| ALU 1 (add, cmp, jxx) | 1      | 6 | — | 4 | 5 |   |
| ALU 2 (add, cmp, jxx) | —      | — | — | — | — |   |
| ALU 3 (mul) start     | 2      | 3 | 7 | 8 | — |   |
| ALU 3 (mul) end       |        | 2 | 3 | 7 | 8 |   |

| reg  | status        |
|------|---------------|
| %x01 | ready         |
| %x02 | ready         |
| %x03 | pending ready |
| %x04 | ready         |
| %x05 | ready         |
| %x06 | pending ready |
| %x07 | ready         |
| %x08 | pending ready |
| %x09 | pending ready |
| %x10 | pending       |
| %x11 | pending ready |
| %x12 | pending ready |
| %x13 | pending ready |
| %x14 | pending ready |
| 6    | 7... ..       |

9 10  
— —

# register renaming: missing pieces

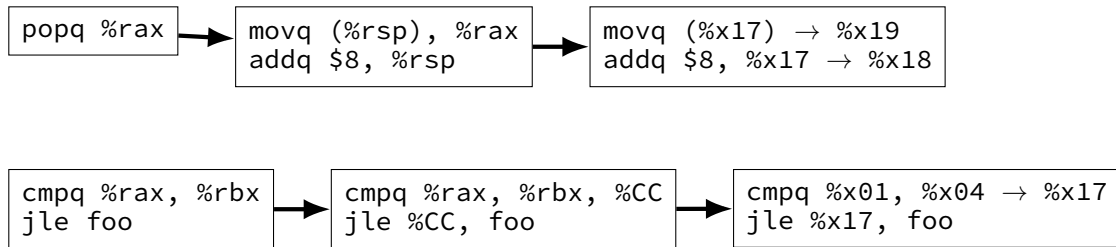
what about “hidden” inputs like `%rsp`, condition codes?

one solution: translate to instructions with additional register parameters

- making `%rsp` explicit parameter

- turning hidden condition codes into operands!

bonus: can also translate complex instructions to simpler ones



# OOO limitations

- can't always find instructions to run

  - plenty of instructions, but all depend on unfinished ones

  - programmer can adjust program to help this

- need to track all uncommitted instructions

  - can only go so far ahead

  - e.g. Intel Skylake: 224-entry reorder buffer, 168 physical registers

- branch misprediction has a big cost (relative to pipelined)

  - e.g. Intel Skylake: up to approx. 16 cycles (v. 2 for simple pipelined CPU)

# OOO limitations

can't always find instructions to run

plenty of instructions, but all depend on unfinished ones

programmer can adjust program to help this

need to track all uncommitted instructions

can only go so far ahead

e.g. Intel Skylake: 224-entry reorder buffer, 168 physical registers

branch misprediction has a big cost (relative to pipelined)

e.g. Intel Skylake: up to approx. 16 cycles (v. 2 for simple pipelined CPU)

## some performance examples

```
example1:  
    movq $1000000000000, %rax  
loop1:  
    addq %rbx, %rcx  
    decq %rax  
    jge loop1  
    ret
```

about 30B instructions  
my desktop: approx 2.65 sec

```
example2:  
    movq $1000000000000, %rax  
loop2:  
    addq %rbx, %rcx  
    addq %r8, %r9  
    decq %rax  
    jge loop2  
    ret
```

about 40B instructions  
my desktop: approx 2.65 sec

## some performance examples

```
example1:  
    movq $1000000000000, %rax  
loop1:  
    addq %rbx, %rcx  
    decq %rax  
    jge loop1  
    ret
```

about 30B instructions

my desktop: approx 2.65 sec

```
example2:  
    movq $1000000000000, %rax  
loop2:  
    addq %rbx, %rcx  
    addq %r8, %r9  
    decq %rax  
    jge loop2  
    ret
```

about 40B instructions

my desktop: approx 2.65 sec

# static branch prediction

forward (target > PC) not taken; backward taken

intuition: loops:

```
LOOP: ...
```

```
...
```

```
je LOOP
```

```
LOOP: ...
```

```
jne SKIP_LOOP
```

```
...
```

```
jmp LOOP
```

```
SKIP_LOOP:
```

# predict: repeat last

PC of branch

0x40042A

hash function

*index*    *prediction/  
last result?*

0

taken (1)

1

not taken (0)

2

taken (1)

3

taken (1)

...

...

14

not taken (0)

15

taken (1)

# predict: repeat last

PC of branch

0x40042A

hash function

index    *prediction/  
last result?*

0    taken (1)

1    not taken (0)

2    taken (1)

3    taken (1)

...

14    not taken (0)

typical choice: some bits of branch address  
for our example: will use bits 4-7

# predict: repeat last

PC of branch

0x40042A

hash function

| <i>index</i> | <i>prediction/<br/>last result?</i> |
|--------------|-------------------------------------|
| 0            | taken (1)                           |
| 1            | not taken (0)                       |
| 2            | taken (1)                           |
| 3            | taken (1)                           |
| ...          | ...                                 |
| 14           | not taken (0)                       |
| 15           | taken (1)                           |

# predict: repeat last

PC of branch

0x40042A

hash function

| <i>index</i> | <i>prediction/<br/>last result?</i> |
|--------------|-------------------------------------|
| 0            | taken (1)                           |
| 1            | not taken (0)                       |
| 2            | taken (1)                           |
| 3            | taken (1)                           |
| ...          | ...                                 |
| 14           | not taken (0)                       |
| 15           | taken (1)                           |

prediction  
to fetch stage

# predict: repeat last

PC of branch

0x40042A

hash function

index prediction/  
last result?

0

taken (1)

1

not taken (0)

2

taken (1)

3

taken (1)

...

...

14

not taken (0)

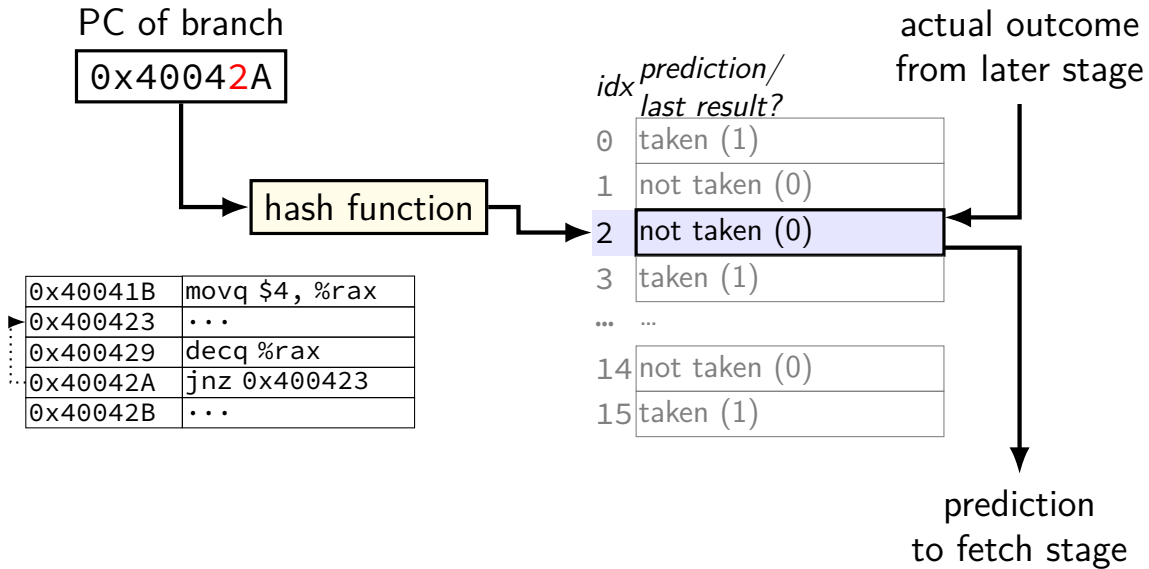
15

taken (1)

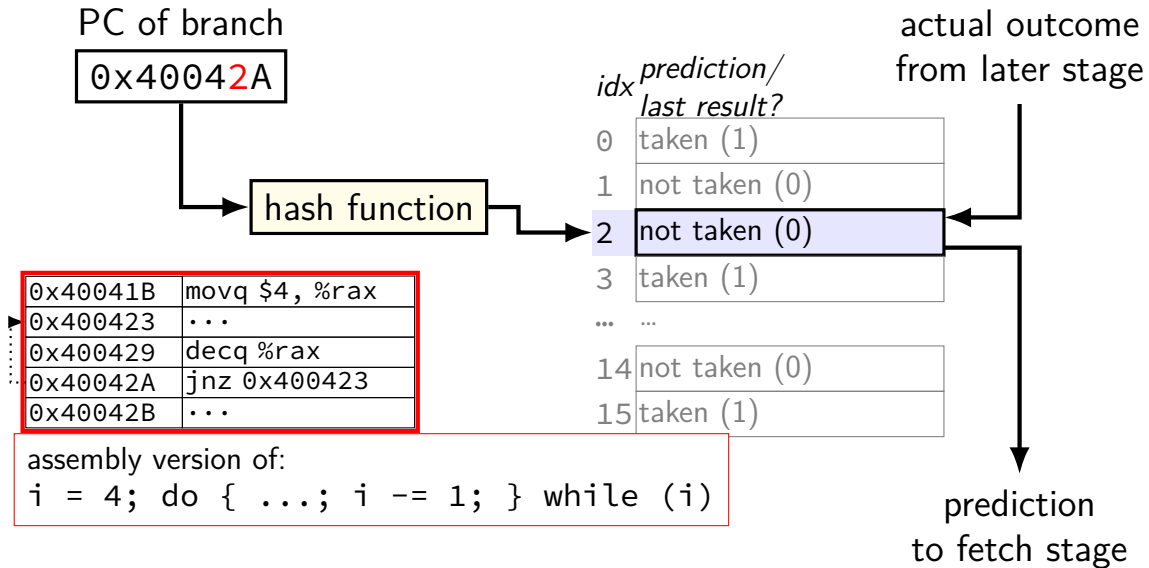
actual outcome  
(from later stage)

prediction  
to fetch stage

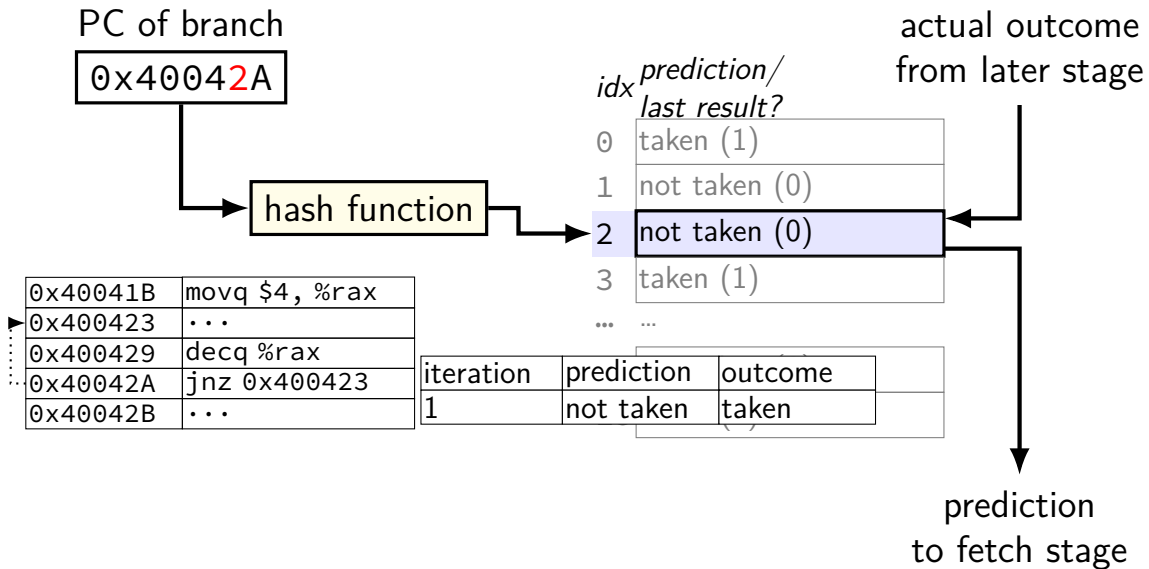
# example



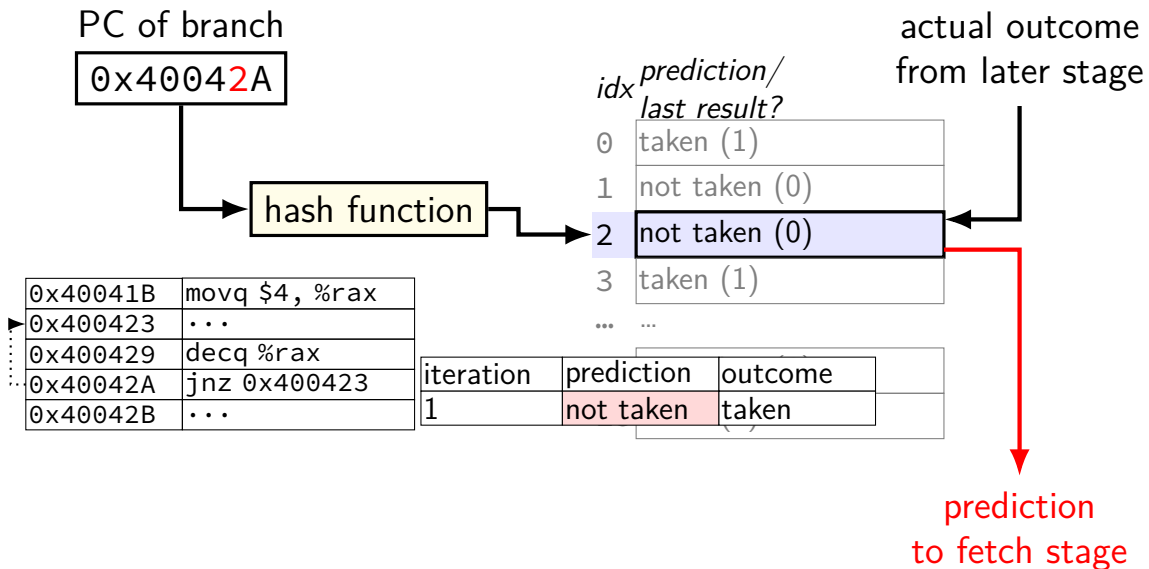
# example



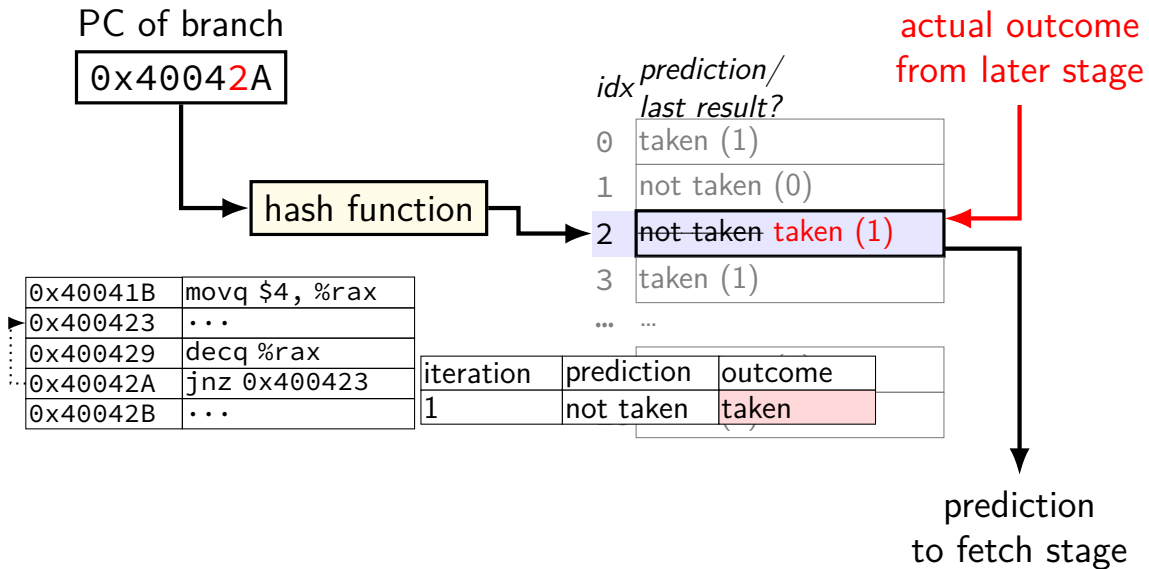
# example



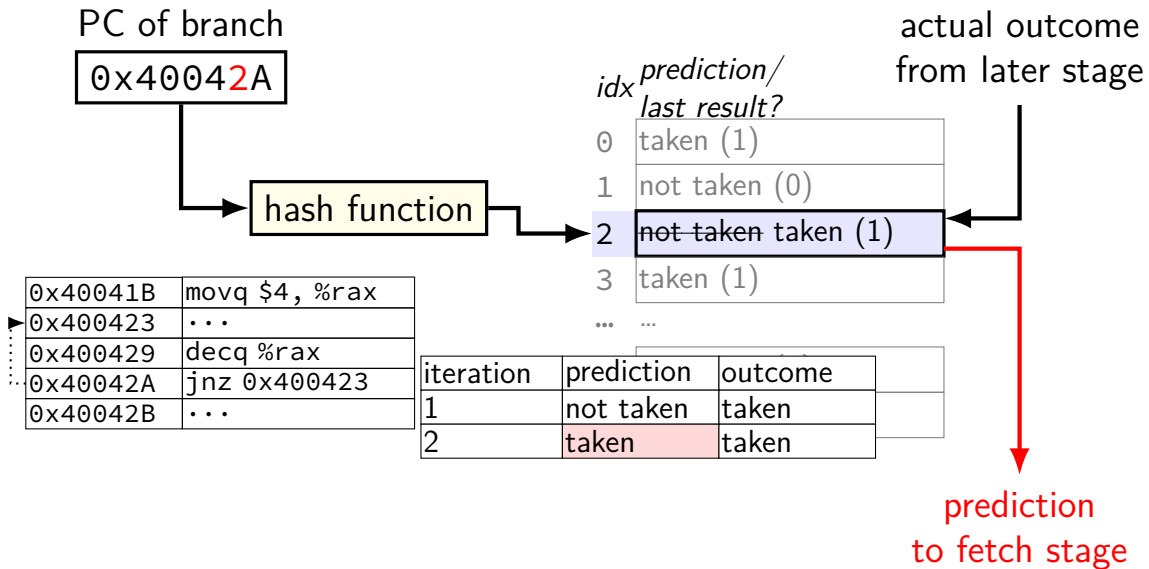
# example



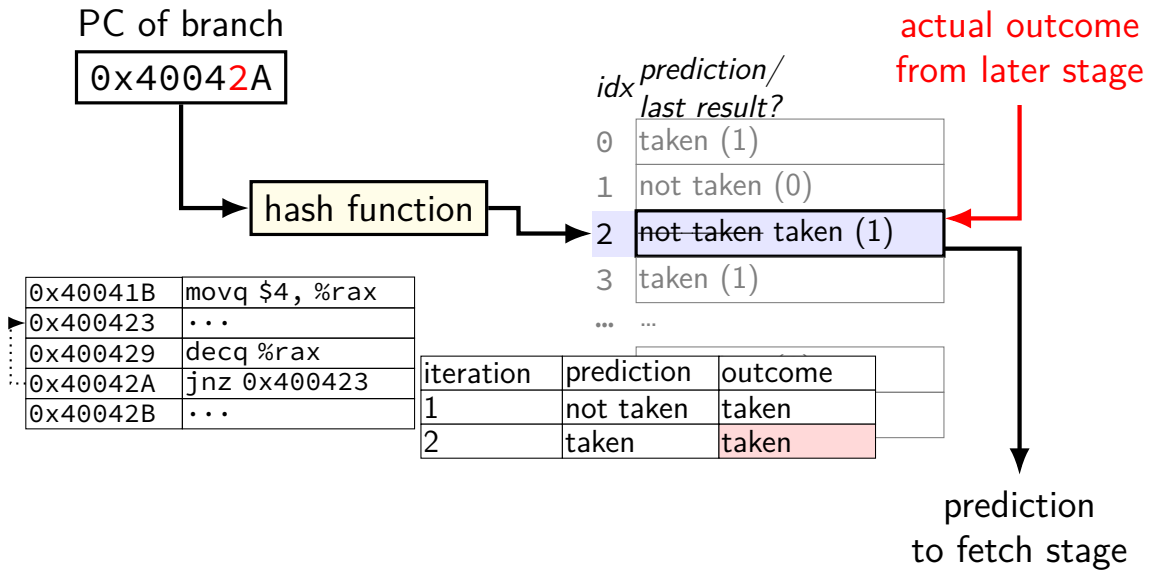
# example



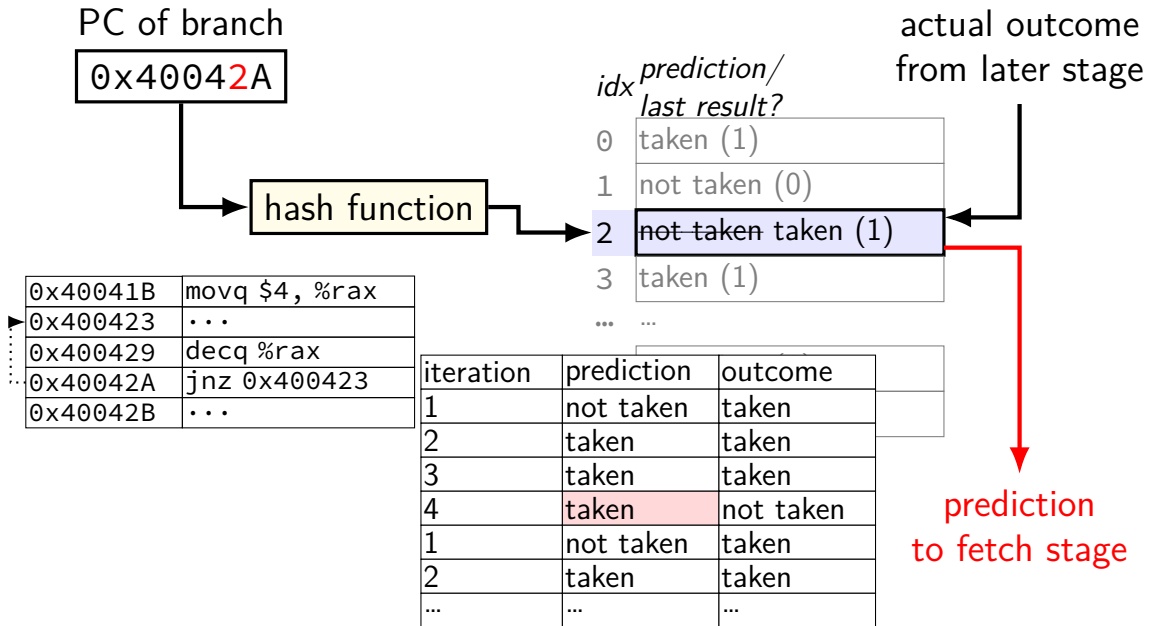
# example



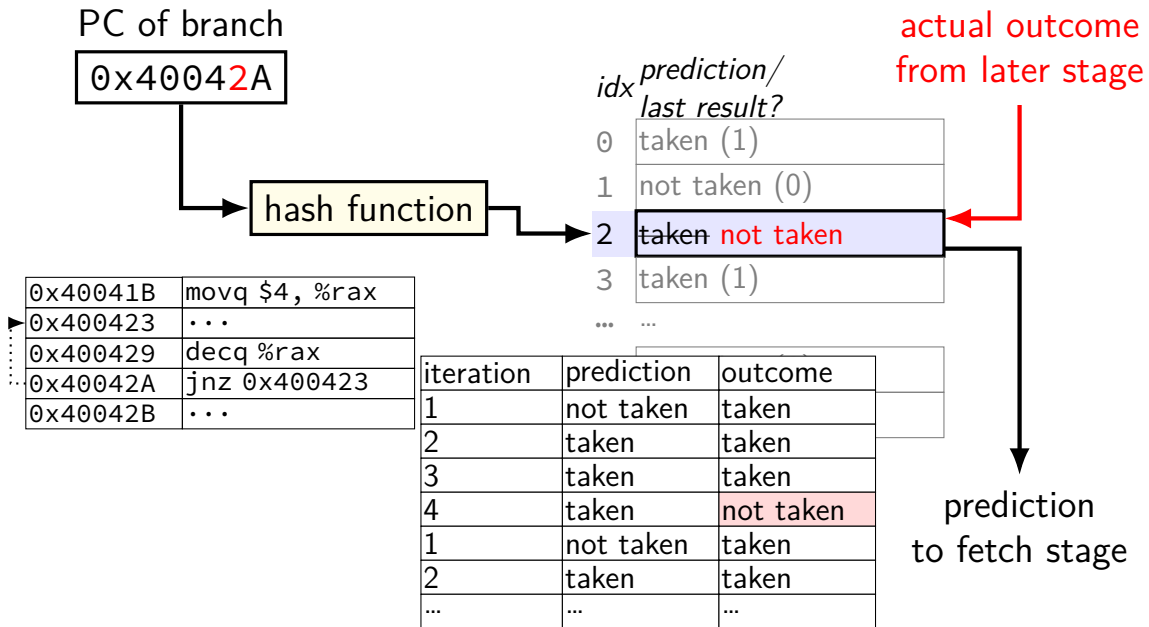
# example



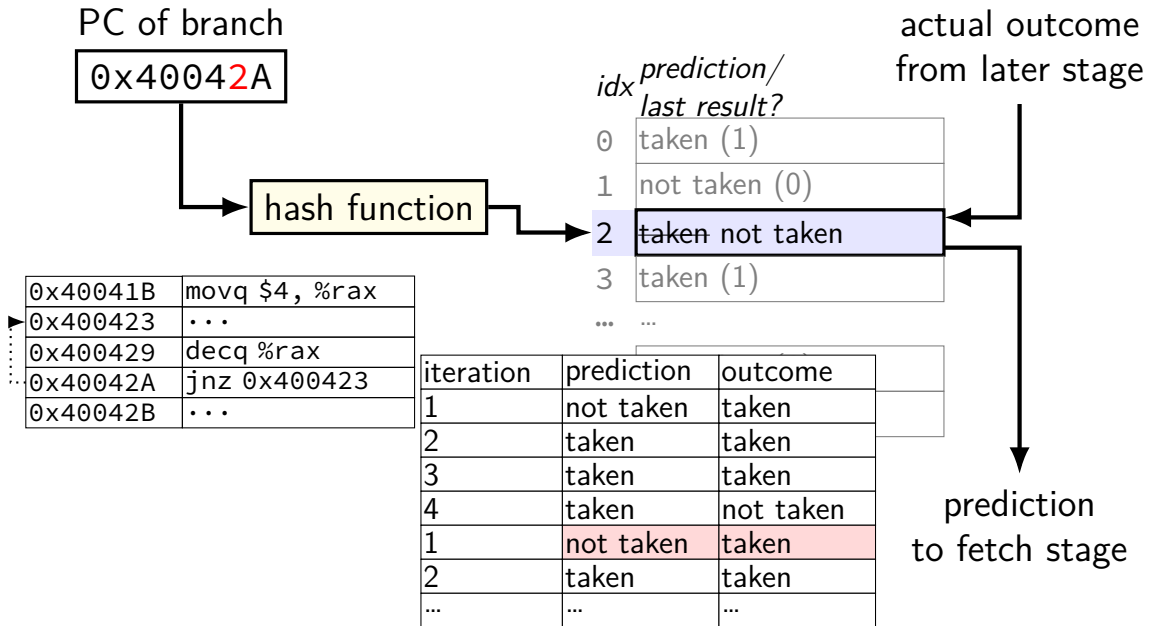
# example



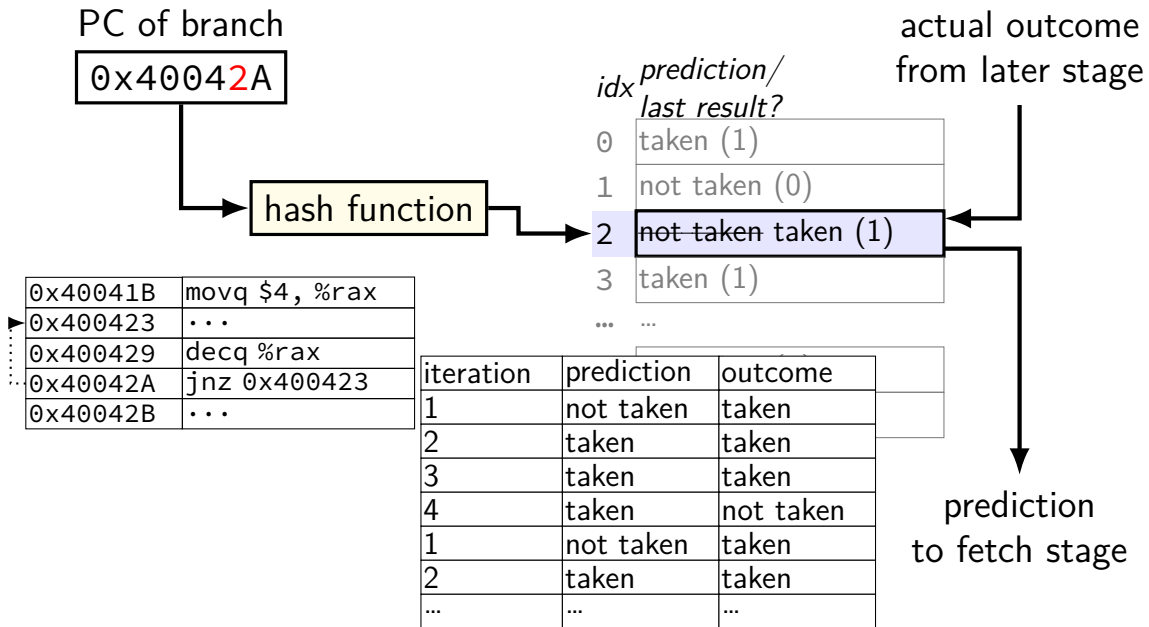
# example



# example



# example



# collisions?

two branches could have same hashed PC

nothing in table tells us about this

versus direct-mapped cache: had *tag bits* to tell

is it worth it?

adding tag bits makes table *much* larger and/or slower

but does anything go wrong when there's a collision?

## collision results

possibility 1: both branches usually taken

no actual conflict — prediction is better(!)

possibility 2: both branches usually not taken

no actual conflict — prediction is better(!)

possibility 3: one branch taken, one not taken

performance probably worse

# 1-bit predictor for loops

predicts first and last iteration wrong

example: branch to beginning — but same for branch from beginning to end

everything else correct

## exercise (pt 1)

use 1-bit predictor on this loop

executed in outer loop (not shown) many, many times

what is the conditional jump misprediction rate for `i % 3 == 0`?

```
int i = 0;
while (true) {
    if (i % 3 == 0)
        goto next;
    ...
next:
    i += 1;
    if (i == 50)
        break;
}
```

## exercise (pt 1)

use 1-bit predictor on this loop

executed in outer loop (not shown) many, many times

what is the conditional jump misprediction rate for  $i \% 3 == 0$ ?

```
int i = 0;
while (true) {
    if (i % 3 == 0)
        goto next;
    ...
next:
    i += 1;
    if (i == 50)
        break;
}
```

| i = | branch | pred | outcome | correct? |
|-----|--------|------|---------|----------|
| 0   | mod 3  | ???  | T       | ???      |
| 1   | mod 3  | T    | F       | no       |
| 2   | mod 3  | F    | F       | yes      |
| 3   | mod 3  | F    | T       | no       |
| ... | ...    | ...  | ...     | ...      |

## exercise (pt 1)

use 1-bit predictor on this loop

executed in outer loop (not shown) many, many times

what is the conditional jump misprediction rate for  $i \% 3 == 0$ ?

```
int i = 0;
while (true) {
    if (i % 3 == 0)
        goto next;
    ...
next:
    i += 1;
    if (i == 50)
        break;
}
```

| i = | branch | pred | outcome | correct? |
|-----|--------|------|---------|----------|
| 0   | mod 3  | ???  | T       | ???      |
| 1   | mod 3  | T    | F       | no       |
| 2   | mod 3  | F    | F       | yes      |
| 3   | mod 3  | F    | T       | no       |
| ... | ...    | ...  | ...     | ...      |

## exercise (pt 1)

use 1-bit predictor on this loop

executed in outer loop (not shown) many, many times

what is the conditional jump misprediction rate for  $i \% 3 == 0$ ?

```
int i = 0;
while (true) {
    if (i % 3 == 0)
        goto next;
    ...
next:
    i += 1;
    if (i == 50)
        break;
}
```

| i = | branch | pred     | outcome  | correct? |
|-----|--------|----------|----------|----------|
| 0   | mod 3  | ???      | T        | ???      |
| 1   | mod 3  | T        | F        | no       |
| 2   | mod 3  | F        | <b>F</b> | yes      |
| 3   | mod 3  | <b>F</b> | T        | no       |
| ... | ...    | ...      | ...      | ...      |

## exercise (pt 2)

use 1-bit predictor on this loop

executed in outer loop (not shown) many, many times

what is the conditional jump misprediction rate for `i == 50`?

```
int i = 0;
while (true) {
    if (i % 3 == 0)
        goto next;
    ...
next:
    i += 1;
    if (i == 50)
        break;
}
```

## exercise (full)

use 1-bit predictor on this loop

executed in outer loop (not shown) many, many times

what is the conditional jump misprediction rate?

```
int i = 0;
while (true) {
    if (i % 3 == 0)
        goto next;
    ...
next:
    i += 1;
    if (i == 50)
        break;
}
```

## exercise (full)

use 1-bit predictor on this loop

executed in outer loop (not shown) many, many times

what is the conditional jump misprediction rate?

```
int i = 0;
while (true) {
    if (i % 3 == 0)
        goto next;
    ...
next:
    i += 1;
    if (i == 50)
        break;
}
```

| i = | branch | pred | outcome | correct? |
|-----|--------|------|---------|----------|
| 0   | mod 3  | ???  | T       | ???      |
| 1   | == 50  | ???  | F       | ???      |
| 1   | mod 3  | T    | F       | —        |
| 2   | == 50  | F    | F       | ✓        |
| ... | ...    | ...  | ...     | ...      |

## exercise (full)

use 1-bit predictor on this loop

executed in outer loop (not shown) many, many times

what is the conditional jump misprediction rate?

```
int i = 0;
while (true) {
    if (i % 3 == 0)
        goto next;
    ...
next:
    i += 1;
    if (i == 50)
        break;
}
```

| i = | branch | pred | outcome | correct? |
|-----|--------|------|---------|----------|
| 0   | mod 3  | ???  | T       | ???      |
| 1   | == 50  | ???  | F       | ???      |
| 1   | mod 3  | T    | F       | —        |
| 2   | == 50  | F    | F       | ✓        |
| ... | ...    | ...  | ...     | ...      |

## branch target buffer

what if we can't decode LABEL from machine code for `jmp LABEL` or `jle LABEL` fast?

will happen in more complex pipelines

what if we can't decode that there's a `RET`, `CALL`, etc. fast?

# BTB: cache for branch targets

| idx  | valid | tag   | ofst | type | target   | (more info?) |
|------|-------|-------|------|------|----------|--------------|
| 0x00 | 1     | 0x400 | 5    | Jxx  | 0x3FFFF3 | ...          |
| 0x01 | 1     | 0x401 | C    | JMP  | 0x401035 | ----         |
| 0x02 | 0     | ---   | ---  | ---  | ---      | ----         |
| 0x03 | 1     | 0x400 | 9    | RET  | ---      | ...          |
| ...  | ...   | ...   | ...  | ...  | ...      | ...          |
| 0xFF | 1     | 0x3FF | 8    | CALL | 0x404033 | ...          |

| valid | ... |
|-------|-----|
| 1     | ... |
| 0     | ... |
| 0     | ... |
| 0     | ... |
| ...   | ... |
| 0     | ... |

```
0x3FFFF3:  movq %rax, %rsi
0x3FFFF7:  pushq %rbx
0x3FFFF8:  call 0x404033
0x400001:  popq %rbx
0x400003:  cmpq %rbx, %rax
0x400005:  jle 0x3FFFF3
...
0x400031:  ret
...
```

# BTB: cache for branch targets

| idx  | valid | tag   | ofst | type | target   | (more info?) |
|------|-------|-------|------|------|----------|--------------|
| 0x00 | 1     | 0x400 | 5    | Jxx  | 0x3FFFF3 | ...          |
| 0x01 | 1     | 0x401 | C    | JMP  | 0x401035 | ----         |
| 0x02 | 0     | ---   | ---  | ---  | ---      | ----         |
| 0x03 | 1     | 0x400 | 9    | RET  | ---      | ...          |
| ...  | ...   | ...   | ...  | ...  | ...      | ...          |
| 0xFF | 1     | 0x3FF | 8    | CALL | 0x404033 | ...          |

| valid | ... |
|-------|-----|
| 1     | ... |
| 0     | ... |
| 0     | ... |
| 0     | ... |
| ...   | ... |
| 0     | ... |

```
0x3FFFF3:  movq %rax, %rsi
0x3FFFF7:  pushq %rbx
0x3FFFF8:  call 0x404033
0x400001:  popq %rbx
0x400003:  cmpq %rbx, %rax
0x400005:  jle 0x3FFFF3
...
0x400031:  ret
...
```

# BTB: cache for branch targets

| idx  | valid | tag   | ofst | type | target   | (more info?) |
|------|-------|-------|------|------|----------|--------------|
| 0x00 | 1     | 0x400 | 5    | Jxx  | 0x3FFFF3 | ...          |
| 0x01 | 1     | 0x401 | C    | JMP  | 0x401035 | ---          |
| 0x02 | 0     | ---   | ---  | ---  | ---      | ---          |
| 0x03 | 1     | 0x400 | 9    | RET  | ---      | ...          |
| ...  | ...   | ...   | ...  | ...  | ...      | ...          |
| 0xFF | 1     | 0x3FF | 8    | CALL | 0x404033 | ...          |

| valid | ... |
|-------|-----|
| 1     | ... |
| 0     | ... |
| 0     | ... |
| 0     | ... |
| ...   | ... |
| 0     | ... |

```

0x3FFFF3:  movq %rax, %rsi
0x3FFFF7:  pushq %rbx
0x3FFFF8:  call 0x404033
0x400001:  popq %rbx
0x400003:  cmpq %rbx, %rax
0x400005:  jle 0x3FFFF3
...
0x400031:  ret
...
```

# indirect branch prediction

`jmp *%rax` or `jmp *(%rax, %rcx, 8)`

BTB can provide a prediction

but can do better with more context

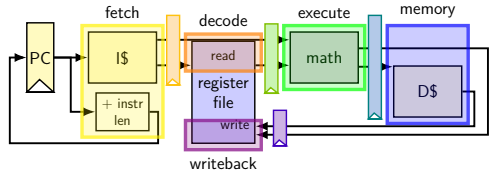
example—predict based on other recent computed jumps  
good for polymorphic method calls

table lookup with `Hash(last few jmps)`  
instead of `Hash(this jmp)`

**backup slides**

# a data hazard

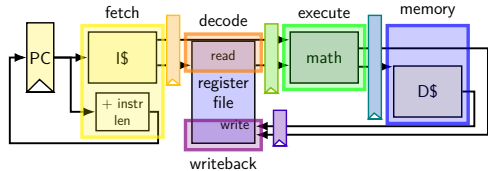
```
// initially %r8 = 800,
//                %r9 = 900, etc.
addq %r8, %r9    // R8 + R9 -> R9
addq %r9, %r8    // R9 + R8 -> R9
addq ...
addq ...
```



|       | fetch | fetch/decode |    | decode/execute |       |    | execute/memory |    | memory/writeback |    |
|-------|-------|--------------|----|----------------|-------|----|----------------|----|------------------|----|
| cycle | PC    | rA           | rB | R[rB]          | R[rB] | rB | sum            | rB | sum              | rB |
| 0     | 0x0   |              |    |                |       |    |                |    |                  |    |
| 1     | 0x2   | 8            | 9  |                |       |    |                |    |                  |    |
| 2     |       | 9            | 8  | 800            | 900   | 9  |                |    |                  |    |
| 3     |       |              |    | 900            | 800   | 8  | 1700           | 9  |                  |    |
| 4     |       |              |    |                |       |    | 1700           | 8  | 1700             | 9  |
| 5     |       |              |    |                |       |    |                |    | 1700             | 8  |

# a data hazard

```
// initially %r8 = 800,
//                %r9 = 900, etc.
addq %r8, %r9    // R8 + R9 -> R9
addq %r9, %r8    // R9 + R8 -> R9
addq ...
addq ...
```



|       | fetch | fetch/decode |    | decode/execute |       |    | execute/memory |    | memory/writeback |    |
|-------|-------|--------------|----|----------------|-------|----|----------------|----|------------------|----|
| cycle | PC    | rA           | rB | R[rB]          | R[rB] | rB | sum            | rB | sum              | rB |
| 0     | 0x0   |              |    |                |       |    |                |    |                  |    |
| 1     | 0x2   | 8            | 9  |                |       |    |                |    |                  |    |
| 2     |       | 9            | 8  | 800            | 900   | 9  |                |    |                  |    |
| 3     |       |              |    | 900            | 800   | 8  | 1700           | 9  |                  |    |
| 4     |       |              |    |                |       |    | 1700           | 8  | 1700             | 9  |
| 5     |       |              |    |                |       |    |                |    | 1700             | 8  |

should be 1700

# data hazard

addq %r8, %r9 // (1)

addq %r9, %r8 // (2)

| step# | pipeline implementation | ISA specification   |
|-------|-------------------------|---------------------|
| 1     | read r8, r9 for (1)     | read r8, r9 for (1) |
| 2     | read r9, r8 for (2)     | write r9 for (1)    |
| 3     | write r9 for (1)        | read r9, r8 for (2) |
| 4     | write r8 for (2)        | write r8 for (2)    |

pipeline reads **older value**...

instead of value ISA says was just written

# data hazard compiler solution

```
addq %r8, %r9  
nop  
nop  
addq %r9, %r8
```

one solution: **change the ISA**

all addqs take effect **three instructions later**

(assuming can read register value while it is being written back)

make it **compiler's job**

problem: recompile everytime processor changes?

# data hazard compiler solution

```
addq %r8, %r9  
nop  
nop  
addq %r9, %r8
```

one solution: **change the ISA**

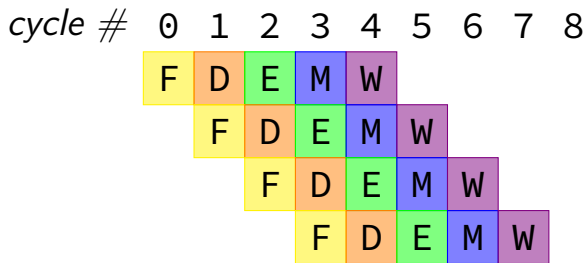
all addqs take effect **three instructions later**  
(assuming can read register value while it is being written back)

make it **compiler's job**

problem: recompile everytime processor changes?

# stalling/nop pipeline diagram (1)

add %r8, %r9  
nop  
nop  
addq %r9, %r8



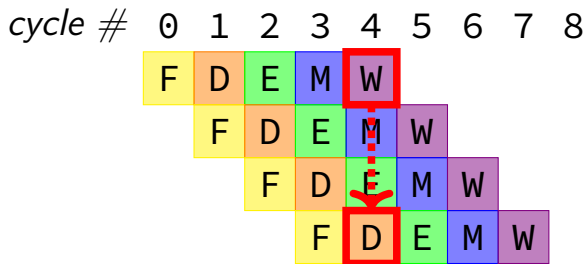
# stalling/nop pipeline diagram (1)

add %r8, %r9

nop

nop

addq %r9, %r8



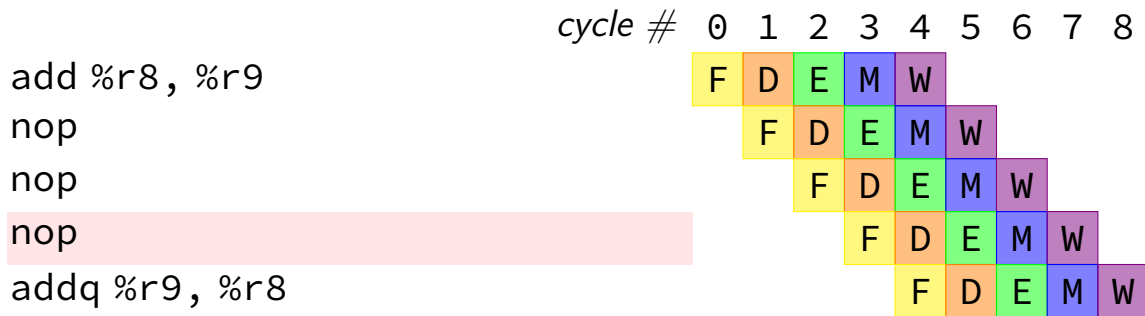
assumption:

if writing register value

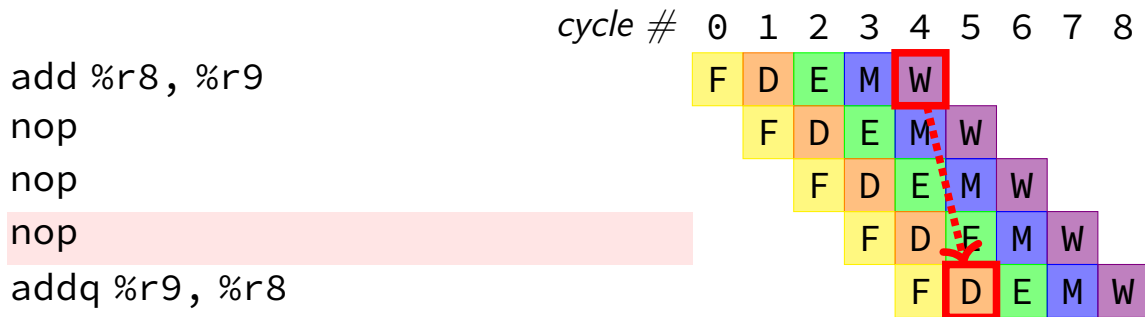
register file will return that value for reads

not actually way register file worked in single-cycle CPU  
(e.g. can read old %r9 while writing new %r9)

## stalling/nop pipeline diagram (2)



## stalling/nop pipeline diagram (2)



if we didn't modify the register file, we'd need an extra cycle

# data hazard hardware solution

```
addq %r8, %r9  
// hardware inserts: nop  
// hardware inserts: nop  
addq %r9, %r8
```

how about hardware add nops?

called **stalling**

extra logic:

- sometimes don't change PC

- sometimes put do-nothing values in pipeline registers

# opportunity

```
// initially %r8 = 800,  
//                %r9 = 900, etc.
```

```
0x0: addq %r8, %r9
```

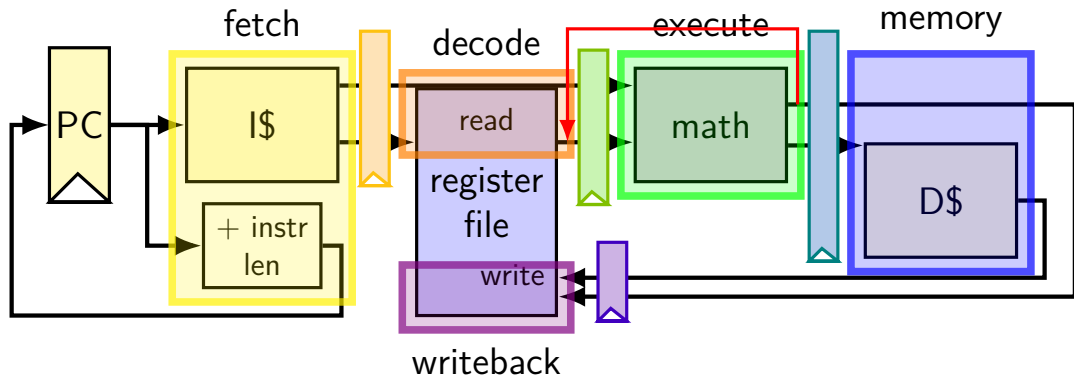
```
0x2: addq %r9, %r8
```

...

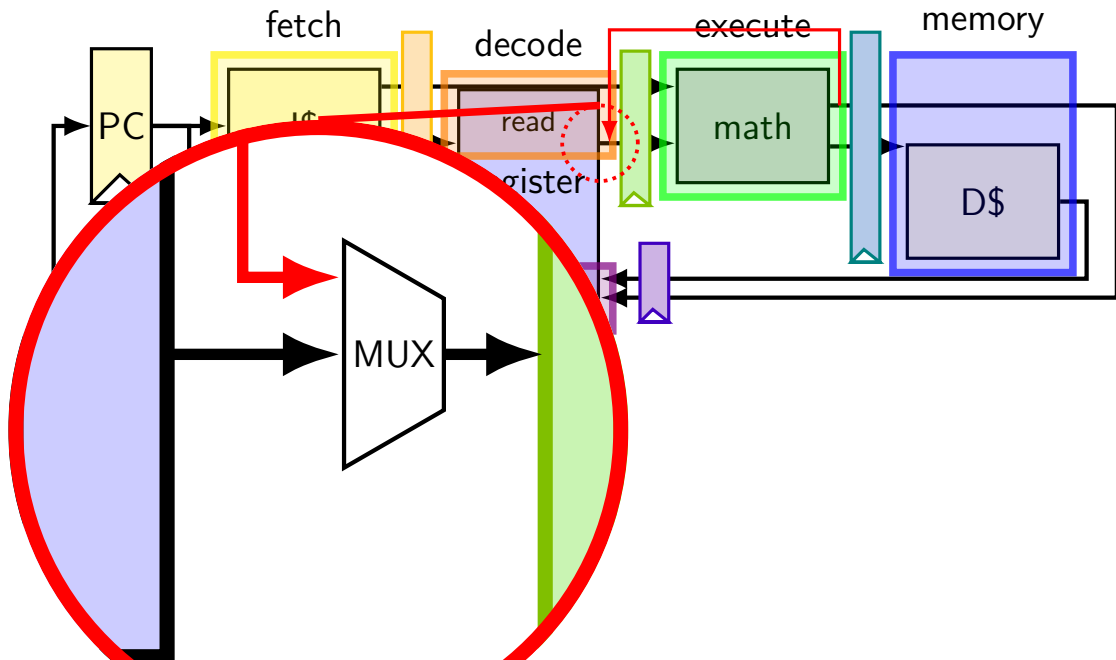
|       | fetch | fetch/decode |    | decode/execute |       |    | execute/memory |    | memory/writeback |    |
|-------|-------|--------------|----|----------------|-------|----|----------------|----|------------------|----|
| cycle | PC    | rA           | rB | R[rB]          | R[rB] | rB | sum            | rB | sum              | rB |
| 0     | 0x0   |              |    |                |       |    |                |    |                  |    |
| 1     | 0x2   | 8            | 9  |                |       |    |                |    |                  |    |
| 2     |       | 9            | 8  | 800            | 900   | 9  |                |    |                  |    |
| 3     |       |              |    | 900            | 800   | 8  | 1700           | 9  |                  |    |
| 4     |       |              |    |                |       |    | 1700           | 8  | 1700             | 9  |
| 5     |       |              |    |                |       |    |                |    | 1700             | 8  |

should be 1700

# exploiting the opportunity



# exploiting the opportunity



# opportunity 2

```
// initially %r8 = 800,  
//                %r9 = 900, etc.
```

```
0x0: addq %r8, %r9
```

```
0x2: nop
```

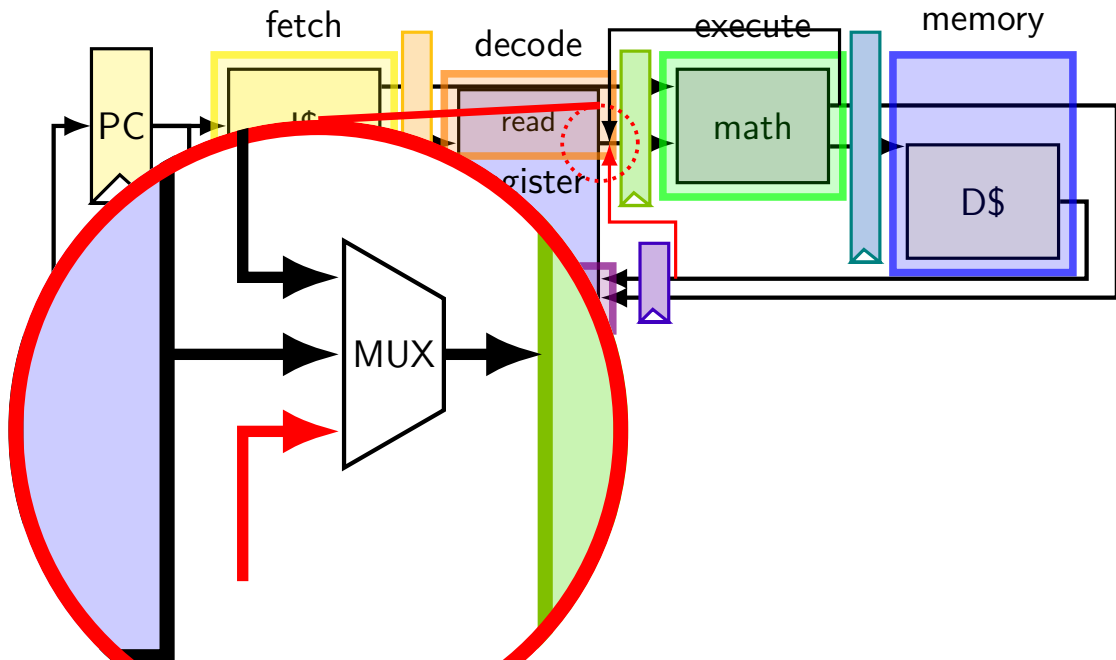
```
0x3: addq %r9, %r8
```

```
...
```

|       | fetch | fetch/decode |     | decode/execute |       |     | execute/memory |     | memory/writeback |     |
|-------|-------|--------------|-----|----------------|-------|-----|----------------|-----|------------------|-----|
| cycle | PC    | rA           | rB  | R[rB]          | R[rB] | rB  | sum            | rB  | sum              | rB  |
| 0     | 0x0   |              |     |                |       |     |                |     |                  |     |
| 1     | 0x2   | 8            | 9   |                |       |     |                |     |                  |     |
| 2     | 0x3   | ---          | --- | 800            | 900   | 9   |                |     |                  |     |
| 3     |       | 9            | 8   | ---            | ---   | --- | 1700           | 9   |                  |     |
| 4     |       |              |     | 900            | 800   | 8   | ---            | --- | 1700             | 9   |
| 5     |       |              |     |                |       |     | 1700           | 9   | ---              | --- |
| 6     |       |              |     |                |       |     |                |     | 1700             | 9   |

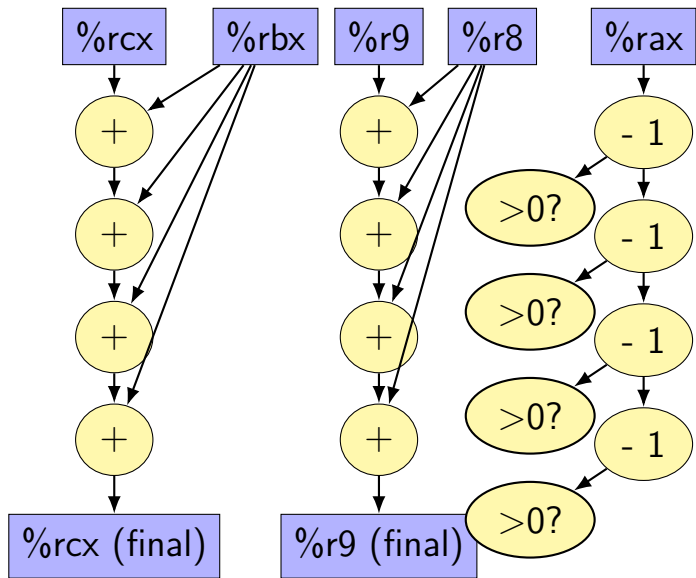
should be 1700

# exploiting the opportunity



**backup slides**

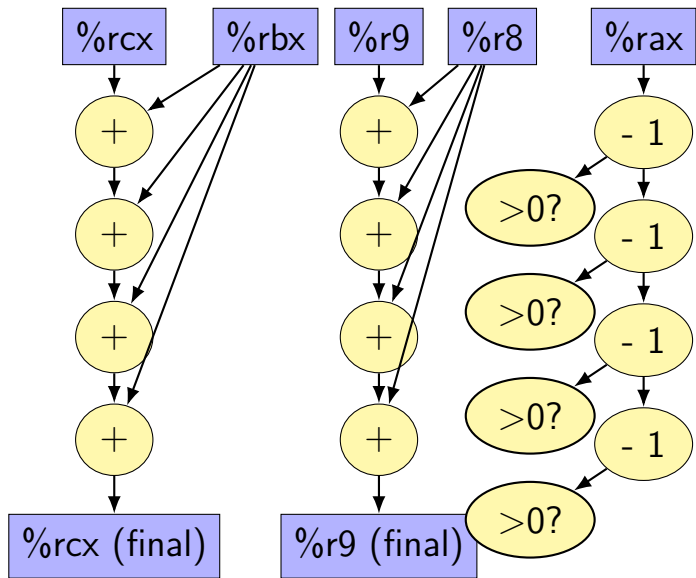
# data flow model and limits (1)



loop2:

```
addq %rbx, %rcx  
addq %r8, %r9  
decq %rax  
jge loop2
```

# data flow model and limits (1)



each yellow box =  
instruction

arrows = dependencies

instructions only executed  
when dependencies ready

# reassociation

with pipelined, 5-cycle latency multiplier; how long does each take to compute?

$$((a \times b) \times c) \times d$$

```
imulq %rbx, %rax  
imulq %rcx, %rax  
imulq %rdx, %rax
```

$$(a \times b) \times (c \times d)$$

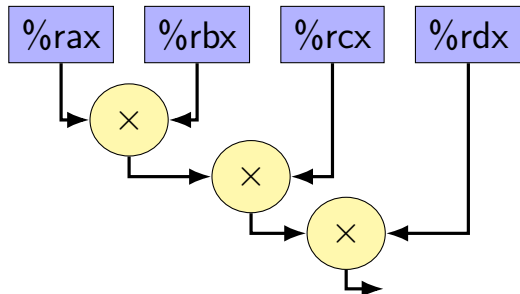
```
imulq %rbx, %rax  
imulq %rcx, %rdx  
imulq %rdx, %rax
```

# reassociation

with pipelined, 5-cycle latency multiplier; how long does each take to compute?

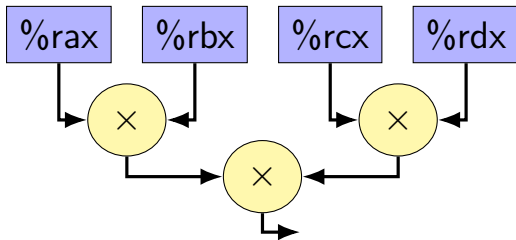
$$((a \times b) \times c) \times d$$

```
imulq %rbx, %rax  
imulq %rcx, %rax  
imulq %rdx, %rax
```



$$(a \times b) \times (c \times d)$$

```
imulq %rbx, %rax  
imulq %rcx, %rdx  
imulq %rdx, %rax
```



# Intel Skylake OOO design

2015 Intel design — codename 'Skylake'

94-entry instruction queue-equivalent

168 physical integer registers

168 physical floating point registers

4 ALU functional units

but some can handle more/different types of operations than others

2 load functional units

but pipelined: supports multiple pending cache misses in parallel

1 store functional unit

224-entry reorder buffer

determines how far ahead branch mispredictions, etc. can happen

# indirect branch prediction

`jmp *%rax` or `jmp *(%rax, %rcx, 8)`

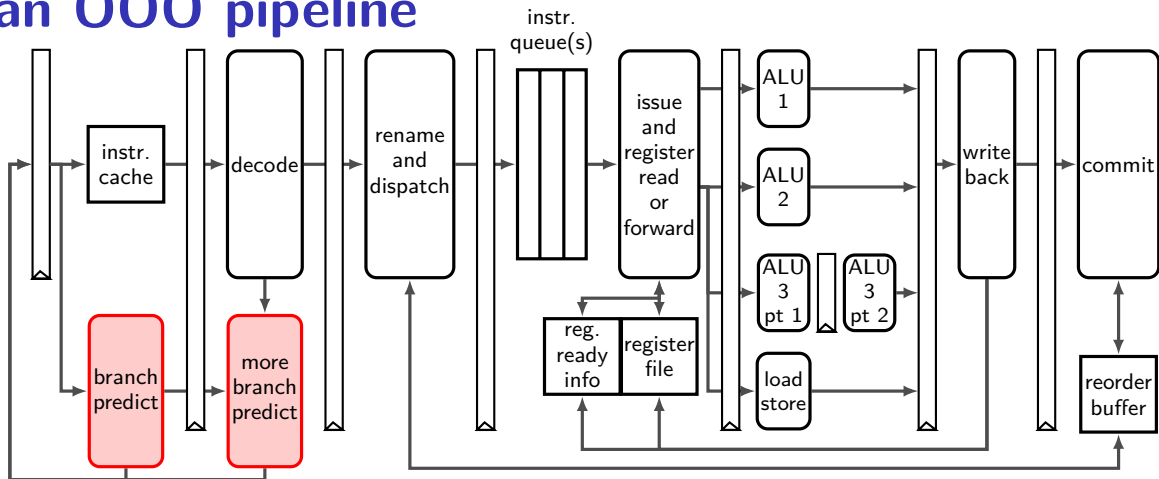
BTB can provide a prediction

but can do better with more context

example—predict based on other recent computed jumps  
good for polymorphic method calls

table lookup with `Hash(last few jmps)`  
instead of `Hash(this jmp)`

# an OOO pipeline



# reorder buffer: on rename

arch → phys reg  
for new instrs

| arch.<br>reg | phys.<br>reg |
|--------------|--------------|
| %rax         | %x12         |
| %rcx         | %x17         |
| %rbx         | %x13         |
| %rdx         | %x07         |
| ...          | ...          |

free list

|      |
|------|
| %x19 |
| %x23 |
| ...  |
| ...  |

# reorder buffer: on rename

arch → phys reg  
for new instrs

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x12      |
| %rcx      | %x17      |
| %rbx      | %x13      |
| %rdx      | %x07      |
| ...       | ...       |

free list

|      |
|------|
| %x19 |
| %x23 |
| ...  |
| ...  |

reorder buffer (ROB)

| instr num. | PC     | dest. reg   | done? | mispred? / except? |
|------------|--------|-------------|-------|--------------------|
| 14         | 0x1233 | %rbx / %x23 |       |                    |
| 15         | 0x1239 | %rax / %x30 |       |                    |
| 16         | 0x1242 | %rcx / %x31 |       |                    |
| 17         | 0x1244 | %rcx / %x32 |       |                    |
| 18         | 0x1248 | %rdx / %x34 |       |                    |
| 19         | 0x1249 | %rax / %x38 |       |                    |
| 20         | 0x1254 | PC          |       |                    |
| 21         | 0x1260 | %rcx / %x17 |       |                    |
| ...        | ...    | ...         | ...   | ...                |
| 31         | 0x129f | %rax / %x12 |       |                    |
|            |        |             |       |                    |
|            |        |             |       |                    |

reorder buffer contains instructions started,  
but not fully finished new entries created on rename  
(not enough space? stall rename stage)

# reorder buffer: on rename

arch → phys reg  
for new instrs

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x12      |
| %rcx      | %x17      |
| %rbx      | %x13      |
| %rdx      | %x07      |
| ...       | ...       |

free list

|      |
|------|
| %x19 |
| %x23 |
| ...  |
| ...  |

reorder buffer (ROB)

remove  
here  
on commit



add here  
on rename



| instr num. | PC     | dest. reg   | done? | mispred? / except? |
|------------|--------|-------------|-------|--------------------|
| 14         | 0x1233 | %rbx / %x23 |       |                    |
| 15         | 0x1239 | %rax / %x30 |       |                    |
| 16         | 0x1242 | %rcx / %x31 |       |                    |
| 17         | 0x1244 | %rcx / %x32 |       |                    |
| 18         | 0x1248 | %rdx / %x34 |       |                    |
| 19         | 0x1249 | %rax / %x38 |       |                    |
| 20         | 0x1254 | PC          |       |                    |
| 21         | 0x1260 | %rcx / %x17 |       |                    |
| ...        | ...    | ...         | ...   | ...                |
| 31         | 0x129f | %rax / %x12 |       |                    |
|            |        |             |       |                    |
|            |        |             |       |                    |

place newly started instruction at end of buffer  
remember at least its destination register  
(both architectural and physical versions)

# reorder buffer: on rename

arch → phys reg  
for new instrs

| arch. reg | phys. reg            |
|-----------|----------------------|
| %rax      | %x12                 |
| %rcx      | %x17                 |
| %rbx      | %x13                 |
| %rdx      | <del>%x07</del> %x19 |
| ...       | ...                  |

free list

|      |
|------|
| %x19 |
| %x23 |
| ...  |
| ...  |

reorder buffer (ROB)

remove  
here  
on commit



add here  
on rename



| instr num. | PC     | dest. reg   | done? | mispred? / except? |
|------------|--------|-------------|-------|--------------------|
| 14         | 0x1233 | %rbx / %x23 |       |                    |
| 15         | 0x1239 | %rax / %x30 |       |                    |
| 16         | 0x1242 | %rcx / %x31 |       |                    |
| 17         | 0x1244 | %rcx / %x32 |       |                    |
| 18         | 0x1248 | %rdx / %x34 |       |                    |
| 19         | 0x1249 | %rax / %x38 |       |                    |
| 20         | 0x1254 | PC          |       |                    |
| 21         | 0x1260 | %rcx / %x17 |       |                    |
| ...        | ...    | ...         | ...   | ...                |
| 31         | 0x129f | %rax / %x12 |       |                    |
| 32         | 0x1230 | %rdx / %x19 |       |                    |
|            |        |             |       |                    |

next renamed instruction goes in next slot, etc.

# reorder buffer: on rename

arch → phys reg  
for new instrs

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x12      |
| %rcx      | %x17      |
| %rbx      | %x13      |
| %rdx      | %x07 %x19 |
| ...       | ...       |

free list

|      |
|------|
| %x19 |
| %x23 |
| ...  |
| ...  |

reorder buffer (ROB)

remove  
here  
on commit



| instr num. | PC     | dest. reg   | done? | mispred? / except? |
|------------|--------|-------------|-------|--------------------|
| 14         | 0x1233 | %rbx / %x23 |       |                    |
| 15         | 0x1239 | %rax / %x30 |       |                    |
| 16         | 0x1242 | %rcx / %x31 |       |                    |
| 17         | 0x1244 | %rcx / %x32 |       |                    |
| 18         | 0x1248 | %rdx / %x34 |       |                    |
| 19         | 0x1249 | %rax / %x38 |       |                    |
| 20         | 0x1254 | PC          |       |                    |
| 21         | 0x1260 | %rcx / %x17 |       |                    |
| ...        | ...    | ...         | ...   | ...                |
| 31         | 0x129f | %rax / %x12 |       |                    |
| 32         | 0x1230 | %rdx / %x19 |       |                    |
|            |        |             |       |                    |

add here  
on rename



# reorder buffer: on commit

arch → phys. reg  
for new instrs

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x12      |
| %rcx      | %x17      |
| %rbx      | %x13      |
| %rdx      | %x07 %x19 |
| ...       | ...       |

free list

|      |
|------|
| %x19 |
| %x13 |
| ...  |
| ...  |

remove  
here →  
on commit

reorder buffer (ROB)

| instr num. | PC     | dest. reg   | done? | mispred? / except? |
|------------|--------|-------------|-------|--------------------|
| 14         | 0x1233 | %rbx / %x24 |       |                    |
| 15         | 0x1239 | %rax / %x30 |       |                    |
| 16         | 0x1242 | %rcx / %x31 |       |                    |
| 17         | 0x1244 | %rcx / %x32 |       |                    |
| 18         | 0x1248 | %rdx / %x34 |       |                    |
| 19         | 0x1249 | %rax / %x38 |       |                    |
| 20         | 0x1254 | PC          |       |                    |
| 21         | 0x1260 | %rcx / %x17 |       |                    |
| ...        | ...    | ...         | ...   | ...                |
| 31         | 0x129f | %rax / %x12 |       |                    |
|            |        |             |       |                    |
|            |        |             |       |                    |

# reorder buffer: on commit

arch → phys. reg  
for new instrs

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x12      |
| %rcx      | %x17      |
| %rbx      | %x13      |
| %rdx      | %x07 %x19 |
| ...       | ...       |

free list

|      |
|------|
| %x19 |
| %x13 |
| ...  |
| ...  |

remove  
here →  
on commit

reorder buffer (ROB)

| instr num. | PC     | dest. reg   | done? | mispred? / except? |
|------------|--------|-------------|-------|--------------------|
| 14         | 0x1233 | %rbx / %x24 |       |                    |
| 15         | 0x1239 | %rax / %x30 |       |                    |
| 16         | 0x1242 | %rcx / %x31 | ✓     |                    |
| 17         | 0x1244 | %rcx / %x32 |       |                    |
| 18         | 0x1248 | %rdx / %x34 | ✓     |                    |
| 19         | 0x1249 | %rax / %x38 | ✓     |                    |
| 20         | 0x1254 | PC          |       |                    |
| 21         | 0x1260 | %rcx / %x17 |       |                    |
| ...        | ...    | ...         | ...   | ...                |
| 31         | 0x129f | %rax / %x12 |       | ✓                  |
|            |        |             |       |                    |
|            |        |             |       |                    |

instructions marked done in reorder buffer when computed but not removed ('committed') yet

# reorder buffer: on commit

arch → phys. reg  
for new instrs

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x12      |
| %rcx      | %x17      |
| %rbx      | %x13      |
| %rdx      | %x07 %x19 |
| ...       | ...       |

free list

|      |
|------|
| %x19 |
| %x13 |
| ...  |
| ...  |

arch → phys reg  
for committed

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x30      |
| %rcx      | %x28      |
| %rbx      | %x23      |
| %rdx      | %x21      |
| ...       | ...       |

remove  
here →  
on commit

reorder buffer (ROB)

| instr num. | PC     | dest. reg   | done? | mispred? / except? |
|------------|--------|-------------|-------|--------------------|
| 14         | 0x1233 | %rbx / %x24 |       |                    |
| 15         | 0x1239 | %rax / %x30 |       |                    |
| 16         | 0x1242 | %rcx / %x31 | ✓     |                    |
| 17         | 0x1244 | %rcx / %x32 |       |                    |
| 18         | 0x1248 | %rdx / %x34 | ✓     |                    |
| 19         | 0x1249 | %rax / %x38 | ✓     |                    |
| 20         | 0x1254 | PC          |       |                    |
| 21         | 0x1260 | %rcx / %x17 |       |                    |
| ...        | ...    | ...         | ...   | ...                |
| 31         | 0x129f | %rax / %x12 |       | ✓                  |
|            |        |             |       |                    |
|            |        |             |       |                    |

commit stage tracks architectural to physical register map  
for committed instructions

# reorder buffer: on commit

arch → phys. reg  
for new instrs

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x12      |
| %rcx      | %x17      |
| %rbx      | %x13      |
| %rdx      | %x07 %x19 |
| ...       | ...       |

free list

|      |
|------|
| %x19 |
| %x13 |
| ...  |
| %x23 |

arch → phys reg  
for committed

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x30      |
| %rcx      | %x28      |
| %rbx      | %x23 %x24 |
| %rdx      | %x21      |
| ...       | ...       |

remove  
here →  
on commit

reorder buffer (ROB)

| instr num. | PC     | dest. reg   | done? | mispred? / except? |
|------------|--------|-------------|-------|--------------------|
| 14         | 0x1233 | %rbx / %x24 | ✓     |                    |
| 15         | 0x1239 | %rax / %x30 |       |                    |
| 16         | 0x1242 | %rcx / %x31 | ✓     |                    |
| 17         | 0x1244 | %rcx / %x32 |       |                    |
| 18         | 0x1248 | %rdx / %x34 | ✓     |                    |
| 19         | 0x1249 | %rax / %x38 | ✓     |                    |
| 20         | 0x1254 | PC          |       |                    |
| 21         | 0x1260 | %rcx / %x17 |       |                    |
| ...        | ...    | ...         | ...   | ...                |
| 31         | 0x129f | %rax / %x12 |       | ✓                  |
| 32         | 0x1230 | %rdx / %x19 |       |                    |

when next-to-commit instruction is done  
update this register map and free register list  
and remove instr. from reorder buffer

# reorder buffer: on commit

arch → phys. reg  
for new instrs

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x12      |
| %rcx      | %x17      |
| %rbx      | %x13      |
| %rdx      | %x07 %x19 |
| ...       | ...       |

free list

|      |
|------|
| %x19 |
| %x13 |
| ...  |
| %x23 |

arch → phys reg remove here  
for committed when committed

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x30      |
| %rcx      | %x28      |
| %rbx      | %x23 %x24 |
| %rdx      | %x21      |
| ...       | ...       |

reorder buffer (ROB)

| instr num. | PC     | dest. reg   | done? | mispred? / except? |
|------------|--------|-------------|-------|--------------------|
| 14         | 0x1233 | %rbx / %x24 | ✓     |                    |
| 15         | 0x1239 | %rax / %x30 |       |                    |
| 16         | 0x1242 | %rcx / %x31 | ✓     |                    |
| 17         | 0x1244 | %rcx / %x32 |       |                    |
| 18         | 0x1248 | %rdx / %x34 | ✓     |                    |
| 19         | 0x1249 | %rax / %x38 | ✓     |                    |
| 20         | 0x1254 | PC          |       |                    |
| 21         | 0x1260 | %rcx / %x17 |       |                    |
| ...        | ...    | ...         | ...   | ...                |
| 31         | 0x129f | %rax / %x12 |       | ✓                  |
| 32         | 0x1230 | %rdx / %x19 |       |                    |

when next-to-commit instruction is done  
update this register map and free register list  
and remove instr. from reorder buffer

# reorder buffer: commit mispredict (one way)

arch → phys reg  
for new instrs

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x12      |
| %rcx      | %x17      |
| %rbx      | %x13      |
| %rdx      | %x19      |
| ...       | ...       |

arch → phys reg  
for committed

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x30 %x38 |
| %rcx      | %x31 %x32 |
| %rbx      | %x23 %x24 |
| %rdx      | %x21 %x34 |
| ...       | ...       |

free list

|      |
|------|
| %x19 |
| %x13 |
| ...  |
| ...  |

reorder buffer (ROB)

| instr num.    | PC                | dest. reg              | done?        | mispred? / except? |
|---------------|-------------------|------------------------|--------------|--------------------|
| <del>14</del> | <del>0x1233</del> | <del>%rbx / %x24</del> | <del>✓</del> |                    |
| <del>15</del> | <del>0x1239</del> | <del>%rax / %x30</del> | <del>✓</del> |                    |
| <del>16</del> | <del>0x1242</del> | <del>%rcx / %x31</del> | <del>✓</del> |                    |
| <del>17</del> | <del>0x1244</del> | <del>%rcx / %x32</del> | <del>✓</del> |                    |
| <del>18</del> | <del>0x1248</del> | <del>%rdx / %x34</del> | <del>✓</del> |                    |
| <del>19</del> | <del>0x1249</del> | <del>%rax / %x38</del> | <del>✓</del> |                    |
| → 20          | 0x1254            | PC                     | ✓            | ✓                  |
| 21            | 0x1260            | %rcx / %x17            |              |                    |
| ...           | ...               | ...                    | ...          | ...                |
| 31            | 0x129f            | %rax / %x12            | ✓            |                    |
| 32            | 0x1230            | %rdx / %x19            |              |                    |
|               |                   |                        |              |                    |

# reorder buffer: commit mispredict (one way)

arch → phys reg  
for new instrs

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x12      |
| %rcx      | %x17      |
| %rbx      | %x13      |
| %rdx      | %x19      |
| ...       | ...       |

arch → phys reg  
for committed

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x30 %x38 |
| %rcx      | %x31 %x32 |
| %rbx      | %x23 %x24 |
| %rdx      | %x21 %x34 |
| ...       | ...       |

reorder buffer (ROB)

| instr num.    | PC                | dest. reg              | done?        | mispred? / except? |
|---------------|-------------------|------------------------|--------------|--------------------|
| <del>14</del> | <del>0x1233</del> | <del>%rbx / %x24</del> | <del>✓</del> |                    |
| <del>15</del> | <del>0x1239</del> | <del>%rax / %x30</del> | <del>✓</del> |                    |
| <del>16</del> | <del>0x1242</del> | <del>%rcx / %x31</del> | <del>✓</del> |                    |
| <del>17</del> | <del>0x1244</del> | <del>%rcx / %x32</del> | <del>✓</del> |                    |
| <del>18</del> | <del>0x1248</del> | <del>%rdx / %x34</del> | <del>✓</del> |                    |
| <del>19</del> | <del>0x1249</del> | <del>%rax / %x38</del> | <del>✓</del> |                    |
| → 20          | 0x1254            | PC                     | ✓            | ✓                  |
| 21            | 0x1260            | %rcx / %x17            |              |                    |
| ...           | ...               | ...                    | ...          | ...                |
| 31            | 0x129f            | %rax / %x12            | ✓            |                    |
| 32            | 0x1230            | %rdx / %x19            |              |                    |

free list

|      |
|------|
| %x19 |
| %x13 |
| ...  |
| ...  |

when committing a mispredicted instruction...  
this is where we undo mispredicted instructions

# reorder buffer: commit mispredict (one way)

arch → phys reg  
for new instrs

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x38      |
| %rcx      | %x32      |
| %rbx      | %x24      |
| %rdx      | %x34      |
| ...       | ...       |

arch → phys reg  
for committed

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x30 %x38 |
| %rcx      | %x31 %x32 |
| %rbx      | %x23 %x24 |
| %rdx      | %x21 %x34 |
| ...       | ...       |

reorder buffer (ROB)

| instr num. | PC     | dest. reg   | done? | mispred? / except? |
|------------|--------|-------------|-------|--------------------|
| 14         | 0x1233 | %rbx / %x24 | ✓     |                    |
| 15         | 0x1239 | %rax / %x30 | ✓     |                    |
| 16         | 0x1242 | %rcx / %x31 | ✓     |                    |
| 17         | 0x1244 | %rcx / %x32 | ✓     |                    |
| 18         | 0x1248 | %rdx / %x34 | ✓     |                    |
| 19         | 0x1249 | %rax / %x38 | ✓     |                    |
| 20         | 0x1254 | PC          | ✓     | ✓                  |
| 21         | 0x1260 | %rcx / %x17 |       |                    |
| ...        | ...    | ...         | ...   | ...                |
| 31         | 0x129f | %rax / %x12 | ✓     |                    |
| 32         | 0x1230 | %rdx / %x19 |       |                    |

free list

|      |
|------|
| %x19 |
| %x13 |
| ...  |
| ...  |

copy commit register map into rename register map  
so we can start fetching from the correct PC

# reorder buffer: commit mispredict (one way)

arch → phys reg  
for new instrs

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x38      |
| %rcx      | %x32      |
| %rbx      | %x24      |
| %rdx      | %x34      |
| ...       | ...       |

arch → phys reg  
for committed

| arch. reg | phys. reg |
|-----------|-----------|
| %rax      | %x30 %x38 |
| %rcx      | %x31 %x32 |
| %rbx      | %x23 %x24 |
| %rdx      | %x21 %x34 |
| ...       | ...       |



free list

|      |
|------|
| %x19 |
| %x13 |
| ...  |
| ...  |

reorder buffer (ROB)

| instr num. | PC     | dest. reg   | done? | mispred? / except? |
|------------|--------|-------------|-------|--------------------|
| 14         | 0x1233 | %rbx / %x24 | ✓     |                    |
| 15         | 0x1239 | %rax / %x30 | ✓     |                    |
| 16         | 0x1242 | %rcx / %x31 | ✓     |                    |
| 17         | 0x1244 | %rcx / %x32 | ✓     |                    |
| 18         | 0x1248 | %rdx / %x34 | ✓     |                    |
| 19         | 0x1249 | %rax / %x38 | ✓     |                    |
| 20         | 0x1254 | PC          | ✓     | ✓                  |
| 21         | 0x1260 | %rcx / %x17 |       |                    |
| ...        | ...    | ...         | ...   | ...                |
| 31         | 0x129f | %rax / %x12 | ✓     |                    |
| 32         | 0x1230 | %rdx / %x19 |       |                    |



...and discard all the mispredicted instructions  
(without committing them)

## better? alternatives

- can take snapshots of register map on each branch

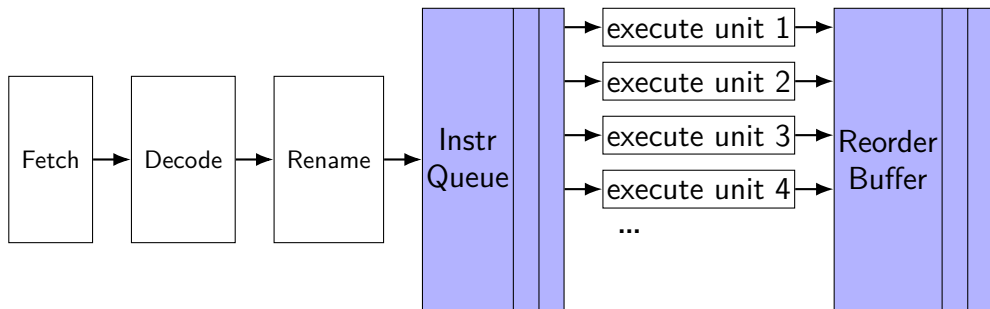
  - don't need to reconstruct the table  
(but how to efficiently store them)

- can reconstruct register map before we commit the branch instruction

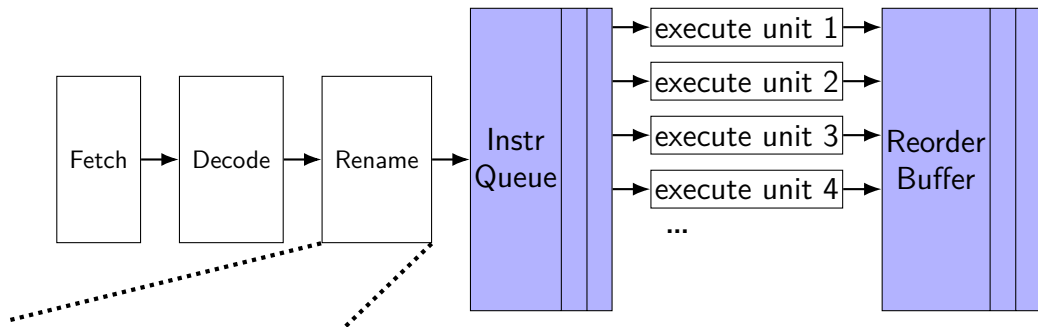
  - need to let reorder buffer be accessed even more?

- can track more/different information in reorder buffer

# exceptions and OOO (one strategy)



# exceptions and OOO (one strategy)

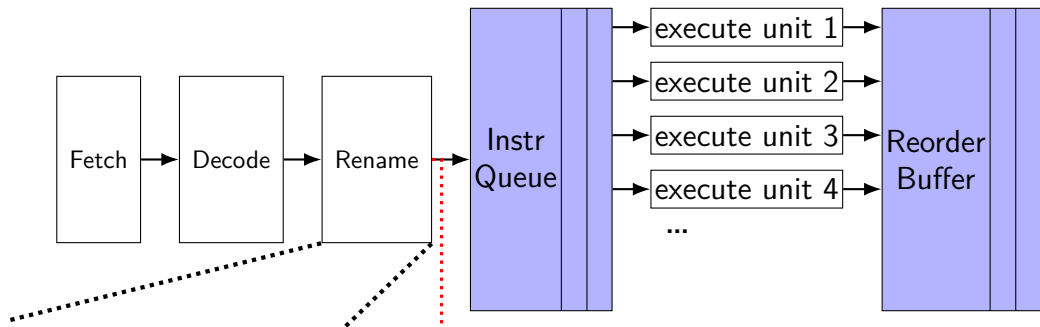


free regs for new instrs

|     |
|-----|
| X19 |
| X23 |
| ... |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X15       |
| RCX       | X17       |
| RBX       | X13       |
| RBX       | X07       |
| ...       | ...       |

# exceptions and OOO (one strategy)



free regs for new instrs

|     |
|-----|
| X19 |
| X23 |
| ... |

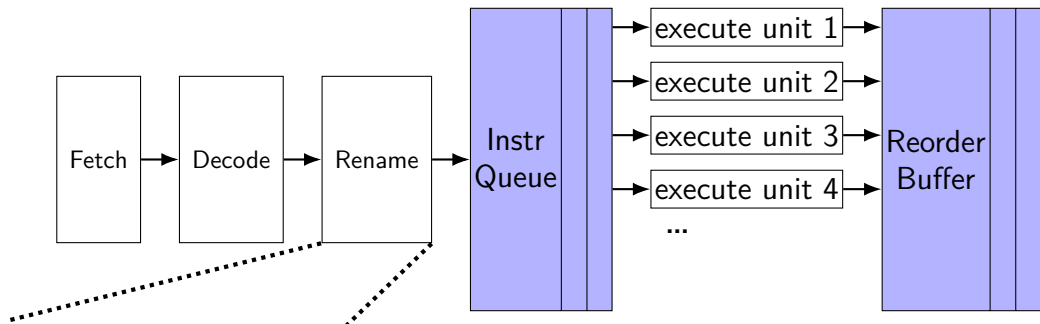
| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X15       |
| RCX       | X17       |
| RBX       | X13       |
| RBX       | X07       |
| ...       | ...       |

done instrs  
committed in order

new instrs added

| instr num. | PC     | dest. reg | done? | except? |
|------------|--------|-----------|-------|---------|
| ...        | ...    | ...       | ...   | ...     |
| 17         | 0x1244 | RCX / X32 |       |         |
| 18         | 0x1248 | RDY / X34 |       |         |
| 19         | 0x1249 | RAX / X38 | ✓     |         |
| 20         | 0x1254 | R8 / X05  |       |         |
| 21         | 0x1260 | R8 / X06  |       |         |
| ...        | ...    | ...       | ...   | ...     |

# exceptions and OOO (one strategy)



free regs for new instrs for complete instrs

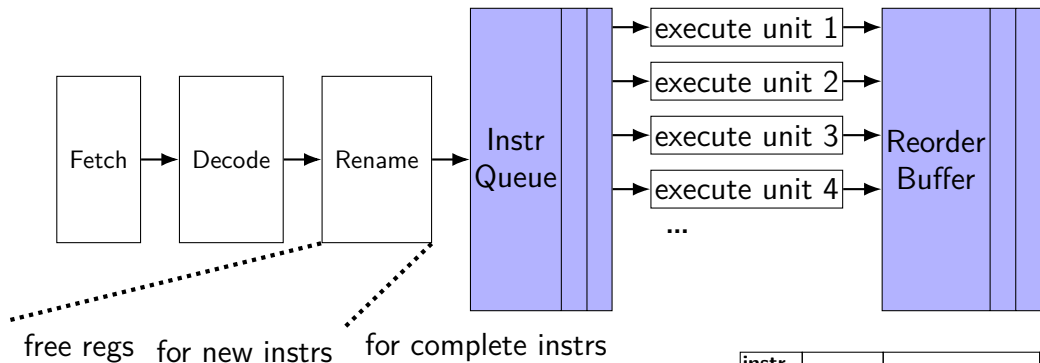
|     |
|-----|
| X19 |
| X23 |
| ... |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X15       |
| RCX       | X17       |
| RBX       | X13       |
| RBX       | X07       |
| ...       | ...       |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X21       |
| RCX       | X2 X32    |
| RBX       | X48       |
| RDX       | X37       |
| ...       | ...       |

| instr num. | PC     | dest. reg | done? | except? |
|------------|--------|-----------|-------|---------|
| ...        | ...    | ...       | ...   | ...     |
| 17         | 0x1244 | RCX / X32 | ✓     |         |
| 18         | 0x1248 | RDX / X34 |       |         |
| 19         | 0x1249 | RAX / X38 | ✓     |         |
| 20         | 0x1254 | R8 / X05  |       |         |
| 21         | 0x1260 | R8 / X06  |       |         |
| ...        | ...    | ...       | ...   | ...     |

# exceptions and OOO (one strategy)



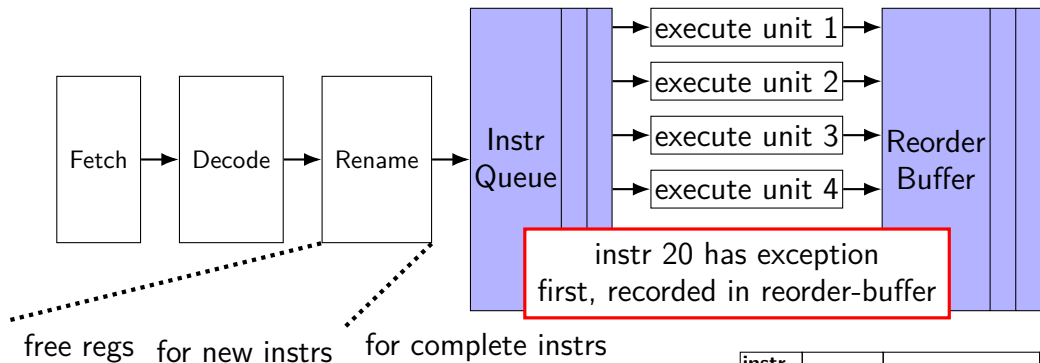
|     |
|-----|
| X19 |
| X23 |
| ... |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X15       |
| RCX       | X17       |
| RBX       | X13       |
| RBX       | X07       |
| ...       | ...       |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X21       |
| RCX       | X2 X32    |
| RBX       | X48       |
| RDX       | X37       |
| ...       | ...       |

| instr num. | PC     | dest. reg | done? | except? |
|------------|--------|-----------|-------|---------|
| ...        | ...    | ...       | ...   | ...     |
| 17         | 0x1244 | RCX / X32 | ✓     |         |
| 18         | 0x1248 | RDX / X34 |       |         |
| 19         | 0x1249 | RAX / X38 | ✓     |         |
| 20         | 0x1254 | R8 / X05  |       |         |
| 21         | 0x1260 | R8 / X06  |       |         |
| ...        | ...    | ...       | ...   | ...     |

# exceptions and OOO (one strategy)



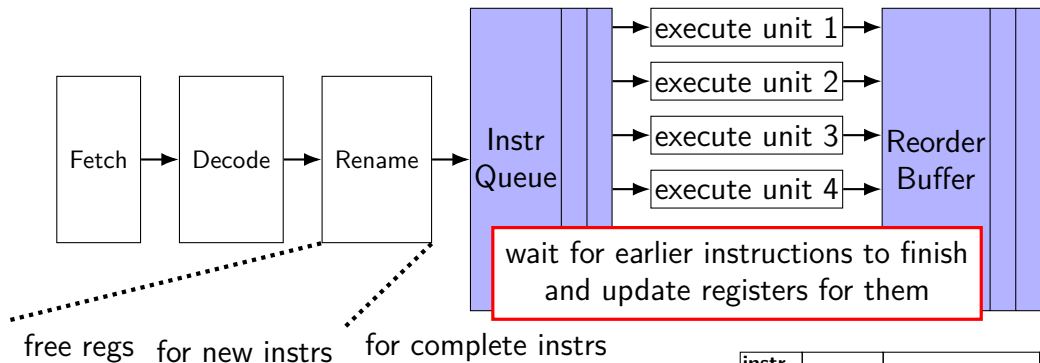
|     |
|-----|
| X19 |
| X23 |
| ... |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X15       |
| RCX       | X17       |
| RBX       | X13       |
| RBX       | X07       |
| ...       | ...       |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X21       |
| RCX       | X2 X32    |
| RBX       | X48       |
| RDX       | X37       |
| ...       | ...       |

| instr num. | PC     | dest. reg | done? | except? |
|------------|--------|-----------|-------|---------|
| ...        | ...    | ...       | ...   | ...     |
| 17         | 0x1244 | RCX / X32 | ✓     |         |
| 18         | 0x1248 | RDX / X34 |       |         |
| 19         | 0x1249 | RAX / X38 | ✓     |         |
| 20         | 0x1254 | R8 / X05  | ✓     | ✓       |
| 21         | 0x1260 | R8 / X06  |       |         |
| ...        | ...    | ...       | ...   | ...     |

# exceptions and OOO (one strategy)



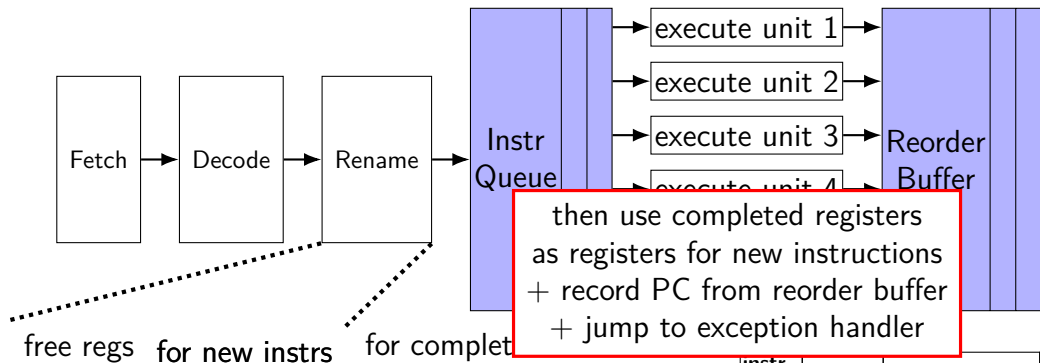
|     |
|-----|
| X19 |
| X23 |
| ... |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X15       |
| RCX       | X17       |
| RBX       | X13       |
| RBX       | X07       |
| ...       | ...       |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X21 X38   |
| RCX       | X2 X32    |
| RBX       | X48       |
| RDX       | X37 X34   |
| ...       | ...       |

| instr num. | PC     | dest. reg | done? | except? |
|------------|--------|-----------|-------|---------|
| ...        | ...    | ...       | ...   | ...     |
| 17         | 0x1244 | RCX / X32 | ✓     |         |
| 18         | 0x1248 | RDX / X34 | ✓     |         |
| 19         | 0x1249 | RAX / X38 | ✓     |         |
| 20         | 0x1254 | R8 / X05  | ✓     | ✓       |
| 21         | 0x1260 | R8 / X06  |       |         |
| ...        | ...    | ...       | ...   | ...     |

# exceptions and OOO (one strategy)



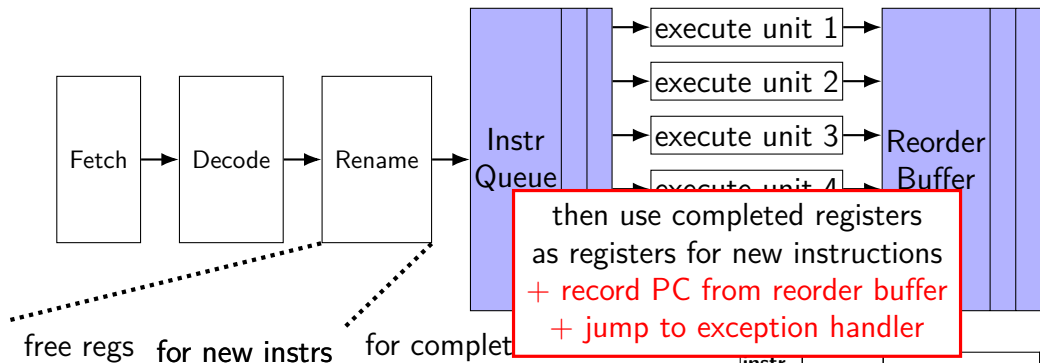
|     |
|-----|
| X19 |
| X23 |
| ... |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X38       |
| RCX       | X32       |
| RBX       | X48       |
| RBX       | X34       |
| ...       | ...       |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X21 X38   |
| RCX       | X2 X32    |
| RBX       | X48       |
| RDX       | X37 X34   |
| ...       | ...       |

| instr num. | PC     | dest. reg | done? | except? |
|------------|--------|-----------|-------|---------|
| ...        | ...    | ...       | ...   | ...     |
| 17         | 0x1244 | RCX / X32 | ✓     |         |
| 18         | 0x1248 | RDX / X34 | ✓     |         |
| 19         | 0x1249 | RAX / X38 | ✓     |         |
| 20         | 0x1254 | R8 / X05  | ✓     | ✓       |
| 21         | 0x1260 | R8 / X06  |       |         |
| ...        | ...    | ...       | ...   | ...     |

# exceptions and OOO (one strategy)



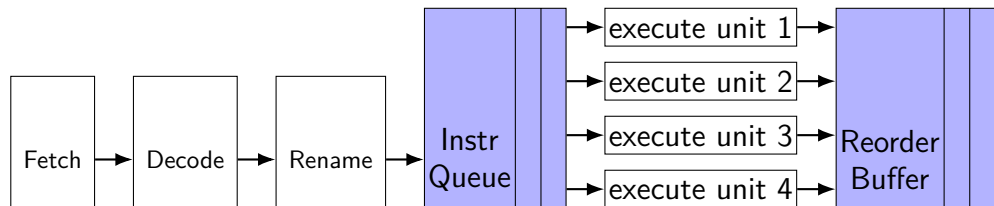
|     |
|-----|
| X19 |
| X23 |
| ... |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X38       |
| RCX       | X32       |
| RBX       | X48       |
| RBX       | X34       |
| ...       | ...       |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X21 X38   |
| RCX       | X2 X32    |
| RBX       | X48       |
| RDX       | X37 X34   |
| ...       | ...       |

| instr num. | PC     | dest. reg | done? | except? |
|------------|--------|-----------|-------|---------|
| ...        | ...    | ...       | ...   | ...     |
| 17         | 0x1244 | RCX / X32 | ✓     |         |
| 18         | 0x1248 | RDX / X34 | ✓     |         |
| 19         | 0x1249 | RAX / X38 | ✓     |         |
| 20         | 0x1254 | R8 / X05  | ✓     | ✓       |
| 21         | 0x1260 | R8 / X06  |       |         |
| ...        | ...    | ...       | ...   | ...     |

# exceptions and OOO (one strategy)



variation: could store architectural reg. values instead of mapping for completed instrs.  
(and copy values instead of mapping on exception)

free regs for new instrs      for complete instrs

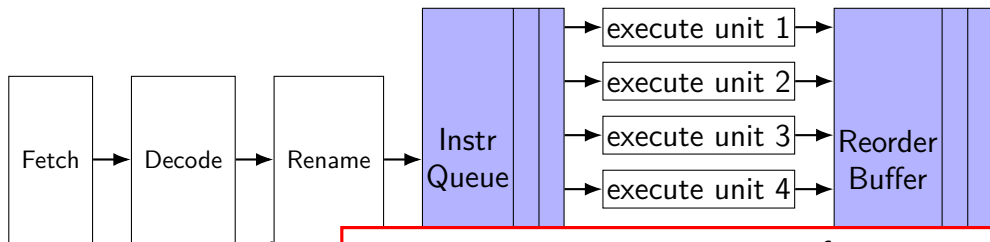
|     |
|-----|
| X19 |
| X23 |
| ... |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X15       |
| RCX       | X17       |
| RBX       | X13       |
| RBX       | X07       |
| ...       | ...       |

| arch. reg | value    |
|-----------|----------|
| RAX       | 0x12343  |
| RCX       | 0x234543 |
| RBX       | 0x56782  |
| RDX       | 0xF83A4  |
| ...       | ...      |

| instr num. | PC     | dest. reg | done? | except? |
|------------|--------|-----------|-------|---------|
| ...        | ...    | ...       | ...   | ...     |
| 17         | 0x1244 | RCX / X32 | ✓     |         |
| 18         | 0x1248 | RDX / X34 | ✓     |         |
| 19         | 0x1249 | RAX / X38 | ✓     |         |
| 20         | 0x1254 | R8 / X05  | ✓     | ✓       |
| 21         | 0x1260 | R8 / X06  |       |         |
| ...        | ...    | ...       | ...   | ...     |

# exceptions and OOO (one strategy)



stopping instructions in progress for exception  
similar to how 'squashing' mispredicted instructions

free regs for new instrs      for complete instrs

|     |
|-----|
| X19 |
| X23 |
| ... |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X15       |
| RCX       | X17       |
| RBX       | X13       |
| RBX       | X07       |
| ...       | ...       |

| arch. reg | phys. reg |
|-----------|-----------|
| RAX       | X21 X38   |
| RCX       | X2 X32    |
| RBX       | X48       |
| RDX       | X37 X34   |
| ...       | ...       |

| instr num. | PC     | dest. reg | done? | except? |
|------------|--------|-----------|-------|---------|
| ...        | ...    | ...       | ...   | ...     |
| 17         | 0x1244 | RCX / X32 | ✓     |         |
| 18         | 0x1248 | RDX / X34 | ✓     |         |
| 19         | 0x1249 | RAX / X38 | ✓     |         |
| 20         | 0x1254 | R8 / X05  | ✓     | ✓       |
| 21         | 0x1260 | R8 / X06  |       |         |
| ...        | ...    | ...       | ...   | ...     |

# handling memory accesses?

one idea:

list of done + uncommitted loads+stores

execute load early + double-check on commit

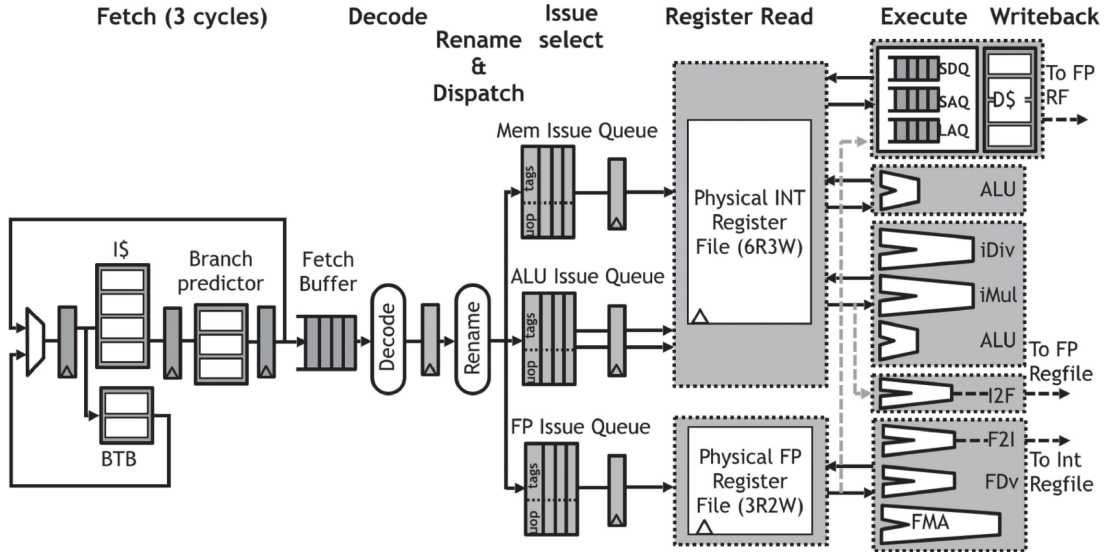
- have data cache watch for changes to addresses on list
- if changed, treat like branch misprediction

loads check list of stores so you read back own values

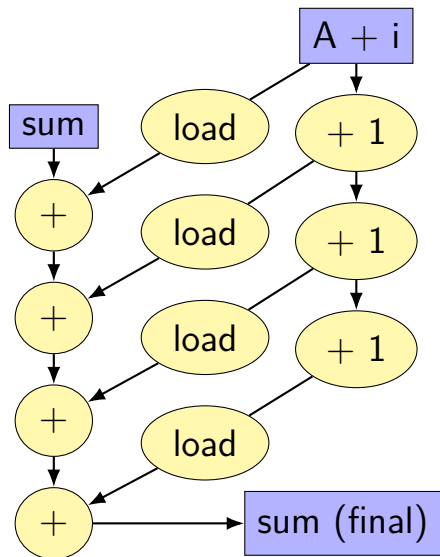
actually finish store on commit

- maybe treat like branch misprediction if conflict?

# the open-source BROOM pipeline

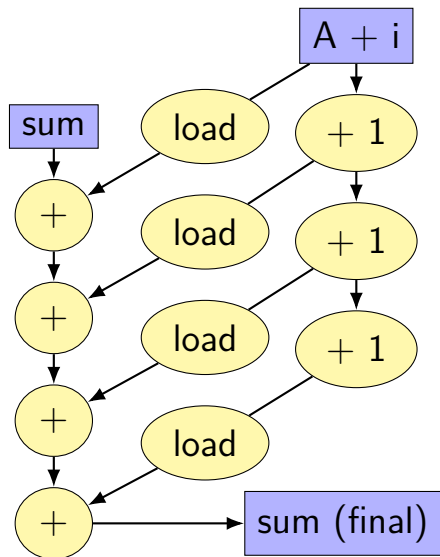


# data flow model and limits



```
for (int i = 0; i < N; i += K) {  
    sum += A[i];  
    sum += A[i+1];  
    ...  
}
```

# data flow model and limits

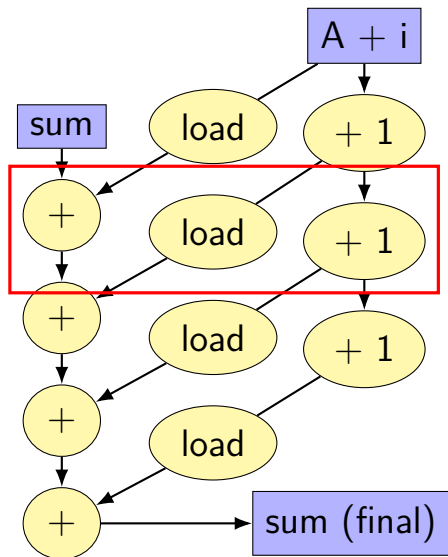


each yellow box = instruction

arrows = dependencies

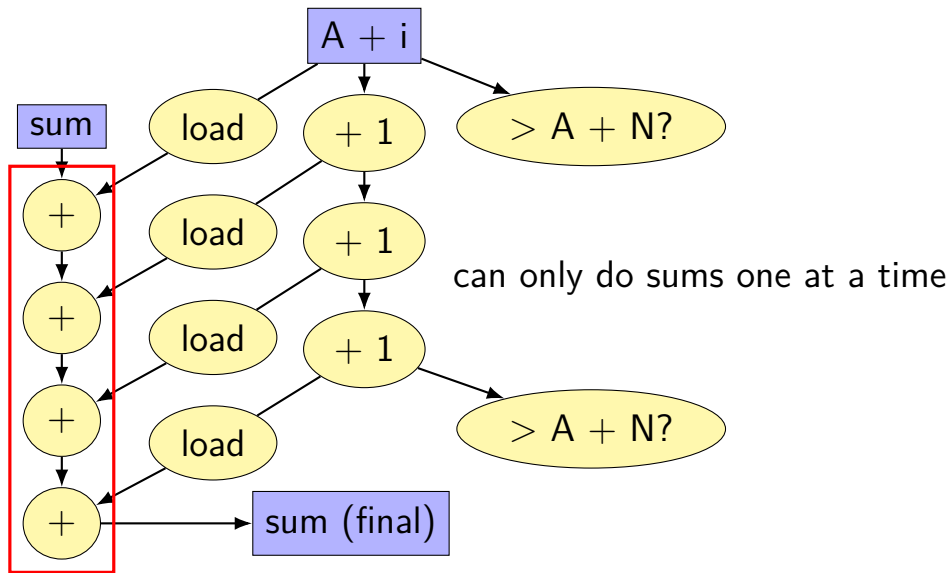
instructions only executed when dependencies

# data flow model and limits

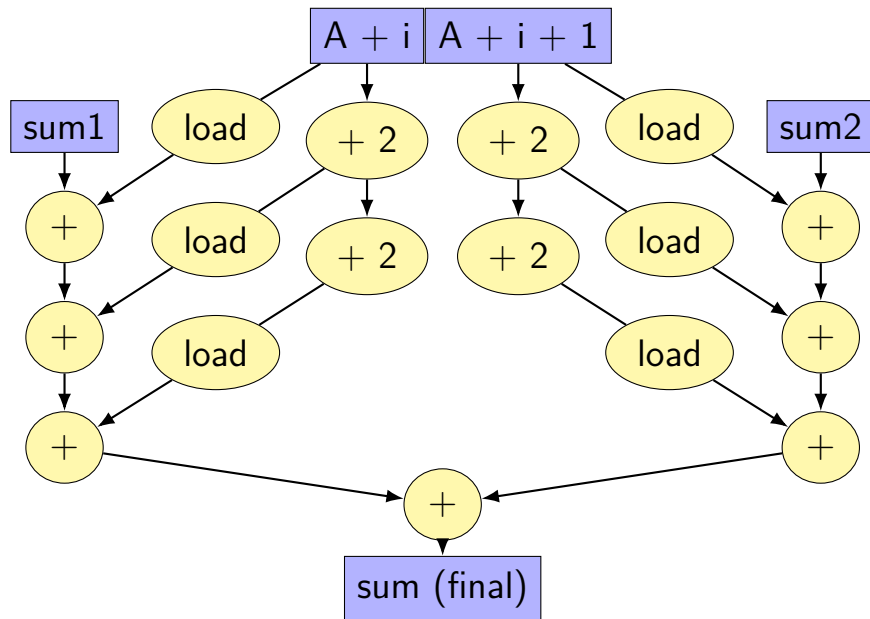


three ops/cycle (if each one cycle)

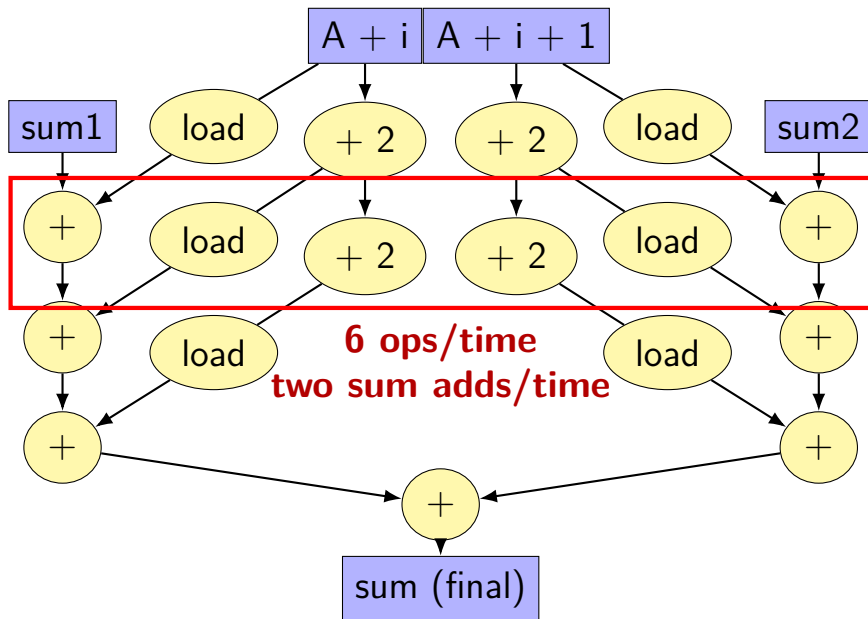
# data flow model and limits



## better data-flow



## better data-flow



## better data-flow

